The Zcash Arboretum

Crosslink Guide

The Crosslink 2 construction: trailing finality over proof of work

Marek
m@rek.onl

Abstract

This volume of *The Zcash Arboretum* describes Crosslink 2, the construction of the design book *Zcash Trailing Finality Layer* of N. Wilcox and D.-E. Hopwood, which adds a Byzantine-fault-tolerant finality protocol to Zcash's proof-of-work best chain. It states the availability-finality dilemma and exactly what the ebb-and-flow theorems of Neu, Tas and Tse prove of snap-and-chat; constructs Crosslink 2 from a best-chain protocol and a BFT protocol taken as black boxes with stated interfaces, with its cross-link fields, every validity and honest-behaviour rule, and Stalled Mode; and defines the finalised and bounded-available chains. It proves that the finalised chain never rolls back and agrees across all honest nodes and times whenever the best chain has Prefix Agreement at the confirmation depth or the BFT protocol has Final Agreement, the second with a corrected proof and a counterexample showing that the Linearity rule is necessary for it; proves, for a node whose finalised chain is a prefix of its best chain, the bounded-availability statement that the book leaves unproved, which, for some best-chain protocol of the interface, requires the confirmation depth not to exceed the finalisation gap bound, and leaves the other case open; and gives each liveness argument its exact status. It records which theorems survive Shielded Labs' design, describes the finaliser set as far as it is designed, states what "final" means to a client and how it differs from confirmation depth, and collects the status of every goal and the open problems. No ZIP specifies Crosslink, the design runs only on Shielded Labs' test network, and every component of Crosslink is designed but unspecified or an open problem. The volume assumes the *Math Guide*, the *Crypto Guide* and the *Consensus Guide*, and cites the *Ironwood Guide*, the *FlyClient Guide* and the *Wallet Guide*. It is non-normative: the Zcash Protocol Specification and the Zcash Improvement Proposals are authoritative, and neither specifies Crosslink.

Contents

1 Introduction **6**
1.1 Scope and status . . . 6
1.2 Epistemic classes . . . 7
1.3 Sources and conventions . . . 8

2 The ebb-and-flow model **9**
2.1 The availability-finality dilemma . . . 9
2.2 Execution model and environments . . . 12
2.3 Ledger security and ebb-and-flow protocols . . . 13
2.4 The snap-and-chat construction . . . 14
2.5 The proof-of-work setting . . . 17
2.6 From snap-and-chat to Crosslink 2 . . . 18

3 The model and the two sub-protocols **19**
3.1 The hybrid construction . . . 19
3.2 Executions, chains and prefix order . . . 20
3.3 The best-chain protocol . . . 21
3.4 The BFT protocol . . . 24

4 The Crosslink 2 construction **29**
4.1 Parameters . . . 29
4.2 Structural additions . . . 30
4.3 Validity rules of the BFT protocol . . . 32
4.4 Honest proposal and honest voting . . . 34
4.5 Validity rules of the best-chain protocol . . . 35
4.6 Honest block production and fork choice . . . 37
4.7 Stalled Mode . . . 39

5 The finalised and bounded-available chains **42**
5.1 The locally finalised chain . . . 42
5.2 The locally bounded-available chain . . . 44
5.3 Syncing and checkpoints . . . 45
5.4 Computation of the finalised prefix . . . 46

6 Safety **48**
6.1 Assured Finality . . . 48
6.2 The Prefix Agreement route . . . 49
6.3 The Final Agreement route . . . 50
6.4 Consequences of the two routes . . . 53
6.5 Safety of the bounded-available chain . . . 53

7 Liveness **55**
7.1 Liveness of the sub-protocols . . . 55
7.2 Progress of finalisation . . . 57

8 Bounded availability and finality overrides **58**
8.1 Bounded availability . . . 58
8.2 The bounded non-stalled run . . . 59
8.3 Finality overrides . . . 61

9 The Shielded Labs design **62**
9.1 Terms and ledger state . . . 62
9.2 Retained and removed rules . . . 63
9.3 Certificates and cross-link references . . . 64
9.4 Sticky fork choice . . . 65
9.5 Proposer behaviour . . . 66
9.6 Finality vocabulary . . . 67
9.7 Theorems under the Shielded Labs design . . . 68

10 The finaliser set **70**
10.1 The abstract finaliser model . . . 70
10.2 Finalisers and the roster . . . 71
10.3 Delegation bonds . . . 74
10.4 Staking windows and privacy . . . 75
10.5 Rewards . . . 76
10.6 Slashing . . . 78
10.7 Finaliser keys . . . 79
10.8 The mechanism-design review . . . 79

11 The client contract **80**
11.1 Finality as a status . . . 80
11.2 Finality and confirmation depth . . . 84
11.3 Ledger views and stalls . . . 85
11.4 Confirmation policy, anchors and expiry . . . 86
11.5 Light clients . . . 87

12 Formal models **88**
12.1 The Lean formalisation . . . 89
12.2 The Informal Systems Quint model . . . 89
12.3 The Manian Quint suite . . . 89
12.4 The Garcia TLA+ specification . . . 90
12.5 Coverage of the formal models . . . 91

13 Status and open problems **91**
13.1 Components a specification must fix 91
13.2 Status by goal and by rule 92

1 Introduction

This section fixes the subject of the volume and its status, the classes and status words that every later claim carries, the sources and the rule by which they are cited, and the notation, before any construction.

1.1 Scope and status

This volume of *The Zcash Arboretum* describes Crosslink 2, the construction of the design book *Zcash Trailing Finality Layer* of N. Wilcox and D.-E. Hopwood. Crosslink 2 is a hybrid of Zcash's proof-of-work best chain and a *Byzantine-fault-tolerant* (BFT) protocol: a protocol whose participants decide by votes and which remains safe while fewer than a fixed fraction of them deviate arbitrarily from it, a property that "The ebb-and-flow model" (§2) states formally. The book describes itself as "an early and incomplete protocol design proposal" (chapter "Status and Next Steps"), and its Crosslink 2 chapters are unchanged since 13 December 2024. The volume is non-normative: the Zcash Protocol Specification and the Zcash Improvement Proposals (ZIPs) are authoritative for every deployed rule, and no Crosslink rule is normative anywhere.

Every status statement of the volume is made as of 16 October 2026, in the present tense. As of that date, no ZIP specifies Crosslink: the main branch of the ZIP repository contains no Crosslink, trailing-finality or NU8 deployment ZIP. Draft Crosslink texts exist only in an open, unmerged draft pull request to that repository, opened on 17 March 2026, in which every text is marked "ZIP: Unassigned", and on branches of two forks of the repository; they are not ZIPs. These drafts weaken or drop several rules of the book, and the author of the pull request describes its texts as requiring refinement or rewriting (3 June 2026); the volume does not use any of them as design. The design runs only on Shielded Labs' test network, relaunched on 1 October 2026: there is no deployment on Mainnet or Testnet, no activation decision and no production value of any parameter, and the small parameter values of the prototype that runs the test network, which its authors describe as unverified, are not given here.

The only analyses of the design are the following. The ebb-and-flow paper of J. Neu, E. N. Tas and D. Tse proves its theorems for snap-and-chat, the construction from which Crosslink descends, in a permissioned model (§2). The book gives its own proofs and arguments. One unrefereed mechanism-design review examines a Shielded Labs design text of 11 December 2025, which predates the overview (§10.8). Four partial formal models exist, one of which, an unmerged Lean formalisation, leaves every Crosslink theorem unproved (§12). No security audit firm has reviewed the design, and no paper on arXiv or in the IACR ePrint archive analyses Crosslink.

The network upgrade NU7, whose deployment ZIP, ZIP 259, has status Draft, does not include Crosslink; ZIP 218 (Draft), which ZIP 259 lists, sets a 25-second block target spacing, so every Crosslink parameter counted in blocks covers a third of its earlier duration, and the volume gives no value of any such parameter. ZIP 218, "Motivation", states that it is complementary to, and does not compete with, finality mechanisms such as Crosslink, and since 16 October 2026 its footnote on Crosslink cites the book.

The volume assumes the *Math Guide*, the *Crypto Guide* and the *Consensus Guide*, and cites the *Ironwood Guide*, the *FlyClient Guide* and the *Wallet Guide* for the objects they construct, each by italic volume name and section title. From the *Math Guide*: probability and asymptotic notation

(§"Probability, asymptotics, and computation", with §"Asymptotic notation" and §"The security parameter"). From the *Crypto Guide*: signature schemes and strong unforgeability (§"Syntax and security goal", Definitions "Digital signature scheme" and "Strong unforgeability"). From the *Consensus Guide*: work, the best chain and confirmations (§"Work, and the best chain"), Proposition "No finite count is final" (§"Confirmation requirements"), rollback limits (§"Rollback limits, coinbase maturity, and finality"), and the boundary that leaves the policy of treating a transaction as final to the layers above it (§"Layer boundary"). From the *Ironwood Guide*: block chains and height (§"Scope and status", Definition "Block chain, transaction, consensus rules") and anchors (§"Anchors"). From the *FlyClient Guide*: the epistemic classes (§"Scope and status", Definition "Epistemic classes"), the block header (§"Header-only verification", Definition "Block header"), the persistence assumption (§"The variable-difficulty backbone model") and the commitment hook of the block header (§"The block commitment hash" and §"The specified commitment and its consumers"). From the *Wallet Guide*: the confirmation policy (§"The confirmation policy"), anchor height (§"Target and anchor heights"), expiry (§"Expiry (ZIP 203)") and terminal states (§"Status tracking and terminal states").

Shielded Labs' design departs from the book in several rules; "The Shielded Labs design" (§9) states the departures and which theorems survive them.

1.2 Epistemic classes

Definition 1.1 (Epistemic classes)**.** A *component* is a data structure, rule, parameter or check of a protocol, or a hypothesis of a result about it. A component is

(i) *specified* when normative text of the protocol specification or of a ZIP fixes it, whatever the status of that ZIP;

(ii) *designed but unspecified* when a published design with an analysis exists but no ZIP or section of the protocol specification fixes it for Zcash;

(iii) an *open problem* when no design exists.

Each result, assumption and classified remark carries a status line whose status word is one of the following, used verbatim:

(a) "proved (ebb-and-flow paper)", "proved (TFL book)" or "proved (this volume)", for an implication with a complete proof by the source named, followed by ", of a designed-but-unspecified construction" when its object is a component of Crosslink;

(b) "argued (TFL book)", "argued (Shielded Labs)" or "argued (ebb-and-flow paper)", for a statement that the source named supports by a prose argument only;

(c) "machine-checked fragment", for a statement that a machine checks on a formal model or a bounded instance, which covers part of a result;

(d) "open problem", for a statement that is neither proved nor argued, or a component with no design;

(e) "assumption", on every Assumption environment: a hypothesis that is neither argued nor open, on which the results that name it are conditional.

The three classes are those of the *FlyClient Guide*, §"Scope and status", Definition "Epistemic classes"; the class designed but unspecified has the sense of that volume, not the sense in which the *Ironwood Guide* uses the same words. A statement that a source makes without proof or argument is recorded as that source's statement in a remark and carries no status word. No Crosslink rule is specified: the class specified applies in this volume only to the lower layers on which Crosslink rests. A property proved of a designed-but-unspecified component establishes an implication about the design and does not make the component specified.

Unqualified, "Crosslink 2" denotes the construction of the book. Shielded Labs' design overview receives one section for its departures from the construction, "The Shielded Labs design" (§9), which states what it keeps, drops and adds, and which theorems survive; "The finaliser set" (§10) describes its finaliser set. Shielded Labs' document on finality semantics, the chapters of its design book that describe earlier designs, and the draft texts on fork branches are not cited as design.

1.3 Sources and conventions

Every external source is cited by author, title and date. A source whose title names an implementation, a function of code or the test network, or whose only title is the name of a repository, is cited instead by author, kind and date. The design book is attributed to its authors, not to an organisation.

The design book is N. Wilcox and D.-E. Hopwood, *Zcash Trailing Finality Layer*, whose Crosslink 2 chapters are unchanged since 13 December 2024; it is called *the book* below and is cited by chapter and section title, verbatim. *Crosslink 1* is the book's earlier construction. Its ledgers survive in the chapter "Security Analysis of Crosslink 2" below that chapter's marker "Not updated for Crosslink 2", and in the chapter "Potential Changes to Crosslink", which declares itself out of date. Only the ledgers of the chapter "The Crosslink 2 Construction", the locally finalised chain and the locally bounded-available chain (§5), are those of Crosslink 2.

The model source is J. Neu, E. N. Tas and D. Tse, "Ebb-and-Flow Protocols: A Resolution of the Availability-Finality Dilemma", arXiv 2009.04987, in its third revision of 4 February 2021, which appeared at the IEEE Symposium on Security and Privacy 2021. It is called *the paper* below and is cited by section, definition and theorem number. Its theorems are for snap-and-chat in a permissioned model, and its statement for proof of work is informal (§2.5).

Shielded Labs' design is cited from two documents: Shielded Labs, *Zcash Crosslink Design Overview*, draft 10 of 14 September 2026, edited to 5 October 2026, called *the overview* below and cited by section title; and Shielded Labs, *Rewards Distribution*, 22 September 2026. Shielded Labs' public *Crosslink FAQ*, modified 30 July 2026, is cited only for statements on rewards and the staking calendar (§10).

The mechanism-design review is a review by N. Della Penna of a Shielded Labs Crosslink design text of 11 December 2025, which predates the overview, with a companion document of formal proofs, both dated January 2026, published on 5 February 2026 and unrefereed. Its title

names an implementation, so it is cited by author, kind and date; it is called *the mechanism-design review* below and serves as framing only (§10.8). Shielded Labs states that it will seek assessments from security audit firms before activation; none has been announced. By the same rule, the blog post of J. McGee for Shielded Labs of 29 May 2026 and the Quint models of Informal Systems and of Z. Manian are cited by author, kind and date, and the Lean pull request to the book's repository and A. Garcia's TLA+ specification by author, title and date (§12).

ZIP text is that of the main branch of the ZIP repository on 16 October 2026. A ZIP is cited by its number and the title of a section; the protocol specification is cited as protocol specification, §"Title". The lower volumes predate that text: the *Consensus Guide* fixes its boundary at the draft NU7 rules of 6 October 2026, and later NU7 changes to ZIPs 218 and 237 are absent from the lower volumes. Where a statement of a lower volume differs from the current text of a ZIP, this volume cites the ZIP.

Prose uses the spellings "finaliser", "finalised" and "finalisation". Verbatim titles, rule names and quotations keep the spelling of their source, as in the book's section "Locally finalized chain". The PCZT roles "IO Finalizer" and "Spend Finalizer" of the *Wallet Guide* are proper names unrelated to the finalisers of this volume.

Each symbol has one meaning, and Table 1 indexes the symbols with the subsection that defines each; the table defines nothing. The letter σ denotes only the confirmation depth of Crosslink 2 (§4.1), and no signature is written σ. The paper's security parameter, which is also its confirmation time and which the paper writes σ, is written λ, as in the *Math Guide*, §"The security parameter", Definition "Security parameter"; λ never denotes a rate. The finalisation gap bound L is an italic capital, never calligraphic. The bounded-available depth μ keeps the book's meaning and is defined locally (§5.2). A variable x over the two chain kinds, bc and bft, replaces the book's star placeholder, since the series reserves the star for the encoding of curve points. The (c, L)-adversary of the *FlyClient Guide* does not occur. The word "anchor" means only a Zcash anchor (*Ironwood Guide*, §"Anchors"). The word "epoch" is always qualified, as in BFT epoch. The word "final", applied to a best-chain block or a transaction, is defined once, in "Finality as a status" (§11.1); it is distinct from the final treestate of a block (*Ironwood Guide*, §"Anchors", Definition "Treestate and anchor") and from the finality of a bft-block in a context (§3.4).

2 The ebb-and-flow model

This section states the model of the ebb-and-flow paper of Neu, Tas and Tse (§1.3), its security definitions, the snap-and-chat construction and exactly what its theorems prove and for which protocols, the paper's informal statement for proof of work, and four deficiencies of snap-and-chat, among those the TFL book records, from which it departs. No theorem of this section is about Crosslink 2 or about proof of work.

2.1 The availability-finality dilemma

A *ledger* is a finite sequence of transactions output by a node; a *ledger protocol* is a protocol in which every node outputs a ledger at every time. Informally, a ledger protocol is safe when the ledgers of honest nodes never conflict, one always being a prefix of the other, and live when ev-

Table 1. Notation: each symbol with its one meaning and the subsection that defines it. The table defines nothing; a letter bound inside a single definition, construction or proof keeps its meaning only there.

Symbol	Meaning	Defined in
λ	security parameter and confirmation time of the paper, which the paper writes σ; never a rate	§2.2
β, Δ, GST, GAT	adversarial fraction, delay bound and transitions of the paper; the paper's node and corruption counts have no symbol	§2.2
β_1, β_2	fractions tolerated under P1 and P2 by an ebb-and-flow protocol	§2.3
$(\mathcal{A}_k(\beta), \mathcal{Z}_k), \mathcal{A}_k^*$	environment of property Pk of the paper, $k \in \{1, 2\}$, and its worst case	§2.2
LOG, LOG_i^t, T_{confirm}	a ledger, the ledger of node i at time t, and the transaction confirmation time	§2.3
$\mathsf{LOG}_{\text{fin}}$	finalised ledger: of the paper's protocols, and, for Crosslink 2, the transaction sequence of fin	§2.3, §5.2
LOG_{da}	dynamically available ledger of the paper	§2.3
Π_{lc}, Π_{sac}, LOG_{lc}	longest-chain protocol of snap-and-chat, snap-and-chat itself, and the ledger of the longest-chain protocol	§2.4
Π_{origbc}, Π_{origbft}, Π_{bc}, Π_{bft}	the two black-box protocols and their Crosslink 2 transforms	§3.1
V_i^t	a view: the value of node i at time t	§3.2
x	variable over the chain kinds bc and bft, in place of the book's star placeholder	§3.2
$\mathcal{O}_x$	genesis block of the x-chains	§3.2
$C\lceil^k$	C without its last k blocks, floored at genesis (the book's notation)	§3.2
$B \preceq_x C$	B is a prefix of C, equality included	§3.2
$B \nsim C$	agreement: one of B and C is a prefix of the other; not transitive	§3.2
lca	last common ancestor	§3.2
height	number of predecessors (*Ironwood Guide*)	§3.2

Table 1. Notation, continued.

Symbol	Meaning	Defined in
ch_i^t	best chain of node i at time t	§3.3
score, work	best-chain order, accumulated work for Zcash; work of a block	§3.3
is-coinbase-only-block	coinbase-only predicate	§3.3
k	depth in Prefix Consistency and Prefix Agreement	§3.3
n, q, m	number of voting units of a BFT epoch, the quorum $\lceil 2n/3 \rceil$, and the number of units cast non-honestly; n has no other meaning	§3.4
proof_P	notarisation proof for a proposal P	§3.4
bft-last-final, origbft-last-final	last final ancestor of a valid bft-block, in Π_{bft} and in Π_{origbft}	§3.4
σ	bc-confirmation depth; never a signature	§4.1
L	finalisation gap bound	§4.1
is-stalled-block	stalled-block predicate	§4.1
context_bft, headers_bc	the two cross-link fields	§4.2
snapshot, LF, candidate, finality-depth	functions derived from the cross-link fields	§4.2
fin_i^t, update-fin	locally finalised chain of node i at time t; its update rule	§5.1
μ, $(\mathsf{ba}_\mu)_i^t$	bounded-available depth; locally bounded-available chain	§5.2
$\mathsf{LOG}_{\mathrm{ba}}$	for Crosslink 2, where the book's names are used, the transaction sequence of ba_μ	§5.2
B_{ckpt}	checkpoint bft-block of a release	§5.3
h_1, h_2	roster height and start height of the Shielded Labs bootstrap; no value is given	§10.2
N	active roster size of the Shielded Labs design; no value is given; the only meaning of italic N, the natural numbers being $\mathbb{N}$	§10.2
h_{exp}	expiry height of a non-coinbase transaction, 0 disabling expiry	§10.4, §11.4
removed	value removed from circulation at a height (ZIP 237)	§10.5
τ	threshold fraction, $\tau \geq 2/3$, of certificates in the mechanism-design review	§10.8
c_{tr}, c_{untr}, z	the confirmation policy of the *Wallet Guide*	§11.4

ery transaction given to honest nodes eventually enters the ledger of every honest node; Definition 2.5 makes both precise. A protocol has *dynamic participation* when honest nodes may stop and resume taking part, asleep and awake, without notice to the other nodes. A *network partition* is an interval of time during which messages between two sets of honest nodes are not delivered. A BFT protocol (§1.1) decides by the votes of a fixed set of participants and stays safe while fewer than a fixed fraction of them deviate arbitrarily from it.

The *availability-finality dilemma* is the statement that no single ledger protocol is both live under dynamic participation and safe under network partition: a node cannot distinguish asleep nodes from nodes cut off by a partition, so it either continues to extend its ledger, losing safety under partition, or waits for a quorum, losing liveness when many nodes sleep. The ebb-and-flow paper does not prove this statement; it attributes the formalisation to Pass and Shi, "The sleepy model of consensus" (ASIACRYPT 2017), and its connection with the CAP theorem of Gilbert and Lynch, "Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services" (SIGACT News, 2002), that no replicated service is both consistent and available under network partition, to Lewis-Pye and Roughgarden, "Resource pools and the CAP theorem" (2020) (Neu, Tas and Tse, Section I-A). The volume uses it with that attribution, not as a theorem.

The paper's resolution (its Section I-B) gives every node two ledgers: a *finalised* ledger, safe under network partition, and a *dynamically available* ledger, live under dynamic participation, of which the finalised ledger is a prefix. Each client chooses which of the two ledgers to trust.

2.2 Execution model and environments

The paper's security parameter is also its confirmation time, a number of slots, and the paper writes it σ. It is written λ here, as in the *Math Guide*, Definition "Security parameter" (§"The security parameter"); σ is reserved for the confirmation depth of Crosslink 2, and λ is never a rate. The paper's counts of nodes and of corrupted nodes are not given symbols: its model is stated here with fractions.

Definition 2.1 (Ebb-and-flow execution model)**.** The model has the following components (Neu, Tas and Tse, Section III-A).

(i) A fixed, finite set of nodes. Time proceeds in *slots*, and nodes have synchronised clocks; bounded clock offsets are counted as network delay.

(ii) A public-key infrastructure that gives each node a unique cryptographic identity, and a random oracle that is the source of the protocol's randomness.

(iii) An *adversary*, a probabilistic polynomial-time algorithm. Before the execution starts it corrupts a set of nodes, which are then *adversarial*: they surrender their state to the adversary and deviate arbitrarily under its control. The other nodes are *honest* and follow the protocol. Corruption is static: no node is corrupted later.

(iv) *Sleep.* For every slot and every honest node the adversary chooses whether the node is *awake* or *asleep*, subject to the environment's constraints. An awake honest node executes the protocol in that slot; an asleep one does not, and the messages that would have

reached it are queued and delivered in the first slot in which it is awake. Adversarial nodes are always awake.

(v) *Two transitions.* One asynchrony-to-synchrony transition, at the time GST, after which honest-to-honest messages arrive within a delay bound of Δ slots; and one sleep-to-awake transition, at the time GAT, after which every honest node is awake.

(vi) A finite time horizon polynomial in the security parameter λ.

(vii) The setting is permissioned: the identities of all nodes are common knowledge, while no honest node knows which nodes are awake.

The paper studies a single transition to synchrony, "we follow the long-standing practice in the BFT literature and study only a single such transition" (Section III-A), and leaves proof-of-stake and proof-of-work settings to later work (Section III).

Definition 2.2 (Environment for P1)**.** The pair $(\mathcal{A}_1(\beta), \mathcal{Z}_1)$ is the environment of the finality property P1: the adversary $\mathcal{A}_1$ corrupts a fraction β of the nodes; before GST it delays messages arbitrarily, and after GST it delivers every message between honest nodes within Δ slots; before GAT it decides which honest nodes are awake and when, and after GAT every honest node is awake. The adversary chooses GST and GAT; honest nodes do not know them, and they may be causal functions of the protocol's randomness.

Without the transition GAT this adversary falls under the CAP theorem, so that no protocol secure against it exists (Neu, Tas and Tse, footnote 5).

Definition 2.3 (Environment for P2)**.** The pair $(\mathcal{A}_2(\beta), \mathcal{Z}_2)$ is the environment of the dynamic availability property P2: at all times the adversary $\mathcal{A}_2$ delivers every message between honest nodes within Δ slots, and it decides which honest nodes are awake and when, subject to the constraint that at all times at most a fraction β of the awake nodes are adversarial and at least one honest node is awake.

Remark 2.4 (Strict thresholds). The paper writes $\mathcal{A}_1^* := \mathcal{A}_1(1/3)$ and $\mathcal{A}_2^* := \mathcal{A}_2(1/2)$ (Section III-C), and its appendix uses the first strictly: under $(\mathcal{A}_1^*, \mathcal{Z}_1)$ the corrupted fraction satisfies $\beta < 1/3$. Read in the same way, $(\mathcal{A}_2^*, \mathcal{Z}_2)$ requires that fewer than half of the awake nodes be adversarial at every time. The tolerable fractions are therefore strictly below one third and one half. A finite set of voters needs the one-third bound in integer form, which the volume gives for Crosslink 2 with the interface of its BFT protocol.

2.3 Ledger security and ebb-and-flow protocols

For ledgers LOG and LOG', the relation $\mathsf{LOG} \preceq \mathsf{LOG}'$ means that LOG is a prefix of LOG', equality included. For a ledger protocol, LOG_i^t is the ledger of node i at time t.

Definition 2.5 (Ledger security)**.** Let T_{confirm} be a polynomial function of λ. A ledger protocol outputting ledgers LOG is *secure after time* T, with transaction confirmation time T_{confirm}, if LOG satisfies the following (Neu, Tas and Tse, Definition 1).

(i) *Safety.* For all times $t \geq t' \geq T$ and all honest nodes i and j awake at t and t' respectively, $\mathsf{LOG}_i^t \preceq \mathsf{LOG}_j^{t'}$ or $\mathsf{LOG}_j^{t'} \preceq \mathsf{LOG}_i^t$.

(ii) *Liveness.* If a transaction is received by an awake honest node at a time $t \geq T$, then for every time $t' \geq t + T_{\text{confirm}}$ and every honest node j awake at t', the transaction is in $\mathsf{LOG}_j^{t'}$.

The ledger is *safe after* T or *live after* T if the respective property holds; it is *safe* or *live* if that property holds after $T = 0$.

Definition 2.6 (Flexible protocol)**.** A *flexible protocol* is a pair of ledger protocols that take the same input transactions and each output a ledger (Neu, Tas and Tse, Definition 2).

Definition 2.7 (Ebb-and-flow protocol)**.** A (β_1, β_2)*-secure ebb-and-flow protocol* is a flexible protocol whose two ledgers are an *available ledger* LOG_{da} and a *finalised ledger* $\mathsf{LOG}_{\text{fin}}$, written $\mathsf{LOG}_{\text{da},i}^t$ and $\mathsf{LOG}_{\text{fin},i}^t$ at node i and time t, such that, for the security parameter $\lambda := T_{\text{confirm}}$, the following hold (Neu, Tas and Tse, Definition 3).

(i) *P1, finality.* Under $(\mathcal{A}_1(\beta_1), \mathcal{Z}_1)$, the ledger $\mathsf{LOG}_{\text{fin}}$ is safe at all times, and there is a constant multiple of $\max\{\mathsf{GST}, \mathsf{GAT}\} + \lambda$ after which $\mathsf{LOG}_{\text{fin}}$ is live, except with probability $\mathsf{negl}(\lambda)$.

(ii) *P2, dynamic availability.* Under $(\mathcal{A}_2(\beta_2), \mathcal{Z}_2)$, the ledger LOG_{da} is secure, except with probability $\mathsf{negl}(\lambda)$.

(iii) *Prefix.* For every honest node i and every time t, $\mathsf{LOG}_{\text{fin},i}^t \preceq \mathsf{LOG}_{\text{da},i}^t$.

A function is $\mathsf{negl}(\lambda)$ in the sense of the *Math Guide*, Definition "Negligible function" (§"Polynomial, exponential, and negligible functions"): it is eventually below λ^{-c} for every constant $c > 0$, which is the paper's own definition.

Remark 2.8 (Classical regimes and optimal resilience). Setting $\mathsf{GAT} = 0$ in Definition 2.2 gives the classical partially synchronous network, in which every honest node is awake; setting $\mathsf{GST} = 0$ and $\mathsf{GAT} = \infty$ gives a synchronous network with dynamic participation. Both follow from the definitions. The paper adds that, for its snap-and-chat construction (§2.4), whose instance with permissioned Sleepy and Streamlet is $(1/3, 1/2)$-secure (Theorem 2.10), $\mathsf{LOG}_{\text{fin}}$, respectively LOG_{da}, then has the optimal resilience achievable in that environment (Section I-D), and states without proof, as a standard bound, that no ebb-and-flow protocol tolerates $\mathcal{A}_1(\beta_1)$ with $\beta_1 \geq 1/3$ in a partially synchronous network, or $\mathcal{A}_2(\beta_2)$ with $\beta_2 \geq 1/2$ in a synchronous network (Section III-C). These optimality statements are recorded as the paper makes them; no result of this volume uses them.

2.4 The snap-and-chat construction

A subscripted Π names a protocol. A *longest-chain protocol* is a ledger protocol whose nodes extend, and confirm blocks at a fixed depth in, the longest chain they know.

Construction 2.9 (Snap-and-chat)**.** The protocol Π_{sac} composes a longest-chain protocol Π_{lc} (in the paper, the permissioned form of Sleepy, after Pass and Shi) with a BFT protocol (in the paper, Streamlet, after Chan and Shi, "Streamlet: Textbook streamlined blockchains", AFT 2020), its *BFT sub-protocol* (Neu, Tas and Tse, Section III-B and Figure 5).

(i) At node i and time t the protocol Π_{lc} outputs a chain of confirmed blocks, whose sequence of transactions is the ledger $\mathsf{LOG}^t_{\text{lc},i}$.

(ii) A proposal of the BFT sub-protocol carries a reference to a block of Π_{lc}; the chain that this block tips is the proposal's *snapshot*. The BFT sub-protocol outputs at node i and time t the chain Ch^t_i of BFT blocks finalised in that node's view.

(iii) The sequence $\mathsf{flatten}(\mathsf{Ch}^t_i)$ is the sequence of transactions of the snapshot chains of the blocks of Ch^t_i, concatenated in chain order (the first reading of Remark 2.17, item (3), which in the paper's setting coincides with the second). The function sanitize maps a sequence of transactions to its subsequence that keeps the first valid occurrence of each transaction and removes duplicates and transactions invalid at their position.

(iv) The ledgers are

$$\mathsf{LOG}^t_{\text{fin},i} := \mathsf{sanitize}\big(\mathsf{flatten}(\mathsf{Ch}^t_i)\big), \qquad \mathsf{LOG}^t_{\text{da},i} := \mathsf{sanitize}\big(\mathsf{LOG}^t_{\text{fin},i} \,\|\, \mathsf{LOG}^t_{\text{lc},i}\big),$$

where $\|$ is concatenation.

(v) *Boycott rule*, the only change to the BFT sub-protocol: an honest node votes for a proposed BFT block only if it views the block's snapshot as confirmed in its own longest chain (Algorithm 1).

The paper's ledgers are formally sequences of blocks of Π_{lc}, from which the transaction ledgers are obtained by sanitisation (its footnote 6). The Prefix property of Definition 2.7 holds by construction, since $\mathsf{LOG}^t_{\text{da},i}$ begins with $\mathsf{LOG}^t_{\text{fin},i}$.

Theorem 2.10 (Snap-and-chat security)**.** *The protocol* Π_{sac} *with permissioned Sleepy as* Π_{lc} *and Streamlet as its BFT sub-protocol is a* $(1/3, 1/2)$*-secure ebb-and-flow protocol, in the model of Definitions 2.1, 2.2 and 2.3 with the thresholds read strictly as in Remark 2.4.*

Status: proved (ebb-and-flow paper).

The theorem is Neu, Tas and Tse, Section III-C, Theorem 1, with its proof in Appendix B; the paper is peer-reviewed (IEEE Symposium on Security and Privacy 2021). Its assumptions are those of the paper's model: a permissioned node set, static corruption, a single GST and a single GAT, a public-key infrastructure and a random oracle; and a lottery probability of Π_{lc} below the bound of Theorem 2.11, which the proof uses for P1 and which implies the bound of Sleepy that it uses for P2. The paper's statement leaves this hypothesis implicit. It is not a theorem about Crosslink 2 or about proof of work.

Theorem 2.11 (Security of permissioned Sleepy)**.** *For the static version of permissioned Sleepy, with a lottery probability per node and slot below the explicit bound of the paper's Theorem 2, a function of the delay bound Δ, the number of nodes and the corrupted fraction, and for every* GST *and* GAT *of the environment $(\mathcal{A}_1^*, \mathcal{Z}_1)$, the protocol Π_{lc} is secure after a constant multiple of* $\max\{\mathsf{GST}, \mathsf{GAT}\} + \lambda$*, with confirmation time λ, except with probability $e^{-\Omega(\sqrt{\lambda})}$ (Definition 2.5;* Math Guide*, §"Asymptotic notation").*

Status: proved (ebb-and-flow paper).

The theorem is Neu, Tas and Tse, Appendix B, Theorem 2, with its proof in Appendix C, where the security parameter is written σ; the constant depends on the lottery probability, the number of nodes, the corrupted fraction and Δ. It concerns Sleepy's permissioned lottery in the model of Definition 2.1, not proof of work. The TFL book uses it only in the sense that it "essentially says that under certain conditions Prefix Agreement holds except with probability $e^{-\Omega(\sqrt{\sigma})}$" (chapter "The Crosslink 2 Construction", section "Model for best-chain protocols"), and the volume uses it in no other sense.

Lemma 2.12 (Safety of the finalised ledger)**.** *In Π_{sac} with permissioned Sleepy as Π_{lc}, its lottery probability below the bound of Theorem 2.11, and Streamlet as its BFT sub-protocol, under $(\mathcal{A}_2^*, \mathcal{Z}_2)$ the ledger $\mathsf{LOG}_{\mathrm{fin}}$ is a safe prefix of the output of Π_{lc} in the view of every honest node at all times, except with probability $e^{-\Omega(\sqrt{\lambda})}$.*

Status: proved (ebb-and-flow paper).

The lemma is Neu, Tas and Tse, Appendix B, Lemma 5: under $(\mathcal{A}_2^*, \mathcal{Z}_2)$ fewer than half of the awake nodes are adversarial, so finalising a BFT block needs at least one honest vote, and by the boycott rule an honest node votes only for a snapshot confirmed in its own view. The proof derives from the security of Π_{lc} under $(\mathcal{A}_2^*, \mathcal{Z}_2)$ that any two snapshots confirmed in honest views, at any times, are prefix-comparable, except with that probability; with the honest vote and the boycott rule, the same then holds for the snapshots of finalised BFT blocks even if those blocks conflict. Section III-C states the step as "since Π_{lc} is safe, the fact that one honest node sees that snapshot as confirmed implies that every honest node sees the same snapshot as confirmed". As for Theorem 2.10, the paper leaves the lottery bound implicit.

Proposition 2.13 (Case analysis of snap-and-chat)**.** *In Π_{sac} with permissioned Sleepy and Streamlet:*

(i) if no two honest views contain conflicting final BFT blocks, then $\mathsf{LOG}_{\mathrm{fin}}$ is safe, whether or not Π_{lc} is safe;

(ii) under $(\mathcal{A}_2^*, \mathcal{Z}_2)$, with the lottery probability of Π_{lc} below the bound of Theorem 2.11, whether or not the BFT sub-protocol is safe, $\mathsf{LOG}_{\mathrm{fin}}$ is a prefix of the output of Π_{lc} in every honest view, except with probability $e^{-\Omega(\sqrt{\lambda})}$.

Status: proved (ebb-and-flow paper).

The first case follows from the safety of the BFT sub-protocol in the proof of Theorem 1 (Neu, Tas and Tse, Appendix B, Lemma 1 and the ledger extraction); the second is Lemma 2.12; Sec-

tion III-C and Figure 9 illustrate both. In the first case the safe BFT sub-protocol orders conflicting snapshots linearly and sanitisation removes the transactions that the conflicts invalidate. In the second, fewer than half of the awake nodes are adversarial, so Π_{lc}, with that lottery bound, is secure except with that probability and finalising a snapshot needs an honest vote, as in Lemma 2.12. The TFL book states the aim of Crosslink 2 beyond the second case: safety of the finalised ledger even when the BFT protocol is entirely subverted, provided the adversary has under half of the best-chain protocol's hash rate (chapter "Notes On Snap-and-Chat", section "Comment on security assumptions").

Remark 2.14 (Other BFT protocols and the scope of the theorems). The paper replaces Streamlet by HotStuff and by PBFT (Section III-D). The HotStuff variant is proved only in an appendix of the arXiv version (Appendix D), which the proceedings version omits; status: proved (ebb-and-flow paper), in the arXiv version only. The PBFT variant is argued, "intuitively", in prose; status: argued (ebb-and-flow paper). Within the paper's model its theorems are therefore settled for snap-and-chat with Sleepy and Streamlet, and with HotStuff in the arXiv appendix. They do not extend to the paper's informal statement for proof of work, nor to PBFT.

2.5 The proof-of-work setting

In the paper's proof-of-work setting nodes are of two kinds: *miners*, measured by hash rate, extend the proof-of-work longest chain Π_{lc}, and *validators*, with unique cryptographic identities, run the BFT sub-protocol (Neu, Tas and Tse, Section V-B).

> **Proposition 2.15** (Permissioned finality for proof of work)**.** *Let snap-and-chat use the proof-of-work longest chain as* Π_{lc} *and Streamlet, HotStuff or PBFT, adapted as in the paper's Sections III-B and III-D, as its BFT sub-protocol, in a network in which communication is asynchronous until* GST *and synchronous after it, validators are always awake, and miners may sleep and wake at any time.*
>
> *(i)* P1, finality. *The ledger* $\mathsf{LOG}_{\mathrm{fin}}$ *is safe at all times and live after* GST *if fewer than one third of the validators are adversarial, under half of the awake hash rate is adversarial, and the awake hash rate is bounded away from zero.*
>
> *(ii)* P2, dynamic availability. *If* $\mathsf{GST} = 0$*, the ledger* $\mathsf{LOG}_{\mathrm{da}}$ *is safe and live at all times if fewer than two thirds of the validators are adversarial, under half of the awake hash rate is adversarial, and the awake hash rate is bounded away from zero.*
>
> *Status: argued (ebb-and-flow paper).*

The arXiv version of the paper states the proposition as "Theorem (Informal, Permissioned Finality for Permissionless PoW)" in Section V-B, with the validator bounds written as 33 and 66 per cent; its only support is the sentence "Security is proved analogously to Section III-C", and no proof is given. The proceedings version contains only a prose summary.

Remark 2.16 (The open question of the paper). The hypotheses of Proposition 2.15 beyond the permissioned case are, under P1, that under half of the awake hash rate is adversarial and that

the awake hash rate is bounded away from zero, without which Π_{lc} may not be live and $\mathsf{LOG}_{\text{fin}}$ has nothing to finalise; and, under P2, that fewer than two thirds of the validators are adversarial, without which adversarial validators finalise blocks that are not on the longest chain. The paper leaves open "whether these additional requirements are fundamental or a limitation of the snap-and-chat construction" (Section V-B).

Status: open problem.

2.6 From snap-and-chat to Crosslink 2

Remark 2.17 (Deficiencies of snap-and-chat). Among the problems of snap-and-chat for Zcash that the TFL book records (chapter "Notes On Snap-and-Chat"), four determine the construction of Crosslink 2.

(1) *Scope of the one-honest-vote argument.* The argument of Lemma 2.12 needs an honest vote in every finalising quorum, hence fewer than two thirds of the votes adversarial, and the security of Π_{lc}, which in the paper's model needs fewer than half of the awake nodes adversarial; the book notes that it "only applies when the number of malicious nodes f is such that $n/3 < f < n/2$", the range in which the BFT sub-protocol may be unsafe. It does not reach the target of Crosslink 2: safety of the finalised ledger with every validator subverted, provided the adversary has under half of the hash rate (section "Comment on security assumptions").

(2) *An assumption on the best chain.* The safety of Π_{lc}, from which the paper derives that a snapshot confirmed in one honest view is confirmed in all (Section III-C), is to be supplemented by evidence that any party can check: a proposal carries the headers of a fixed number of best-chain blocks above its snapshot, whose proof of work confirms it (section "Finalization Availability"; chapter "The Crosslink 2 Construction", section "Structural additions").

(3) *Sanitisation.* Sanitisation admits two readings: concatenate the transactions of all snapshots and keep each transaction that is valid at its position; or concatenate the blocks of all snapshots, remove duplicate blocks, and only then apply the same filter. The two coincide if double spends and missing inputs are the only reasons for a transaction to be invalid at a position, since a transaction omitted for either reason cannot later be reinstated. Zcash anchors violate the condition: the anchor rule admits the final treestate of any earlier block (*Ironwood Guide*, §"Anchors"), and the same treestate can be produced by unrelated transactions, so a transaction invalid for a missing anchor can become valid at a later position; expiry, by contrast, is never undone. The ledgers are therefore to be plain block chains, and sanitisation disappears (section "Subtlety in the definition of sanitization"; chapter "The Crosslink 2 Construction", section "Locally finalized chain").

(4) *Spending finalised outputs.* Snap-and-chat cannot let a transaction spend a finalised output that comes from another fork's snapshot, because the best chain does not determine the finalised ledger, and validity of a block may depend only on the block and its ancestors. Each best-chain block is therefore to commit to a BFT block from which the finalised prefix is computable, and that commitment is not to move backwards along a chain (sections

"Spending finalized outputs" and "Finalization Availability", the latter stating the rule "Extension").

Status: argued (TFL book).

Remark 2.18 (The model of Crosslink 2). The TFL book takes neither synchrony nor partial synchrony as an implicit assumption of the communication model of Crosslink 2: unless a sub-protocol states otherwise, messages may be delayed arbitrarily or dropped, and each sub-protocol imports its timing assumptions explicitly. This choice is designed but unspecified. Its reason is argued (TFL book): the single-GST argument of partial synchrony is sound for the safety of a non-terminating protocol, but cannot correctly be applied to its liveness, and properties that hold after a first transition to synchrony need not hold after later ones (chapter "The Crosslink 2 Construction", admonition "Discussion of incorrect applications of the GST formalization of partial synchrony to continuously operating protocols."). No (β_1, β_2)-security theorem in the sense of Definition 2.7 exists for Crosslink 2. Its safety statements are implications between properties of single executions, without probability, and are not comparable with that definition.

3 The model and the two sub-protocols

This section defines Crosslink 2 as a transformation of two black-box protocols, fixes its communication and execution model and the notation for chains, and states the interface that each black box must satisfy: the best-chain protocol with Prefix Consistency and Prefix Agreement at a depth k, and the BFT protocol with voting units, notarisation, the one-third bound, its last final block and Final Agreement. The Zcash instances of the score and of header-checkable work are specified facts; the prefix properties of Zcash proof of work are an open problem, stated once, at the end of the best-chain interface.

3.1 The hybrid construction

Definition 3.1 (Crosslink 2)**.** *Crosslink 2* is the hybrid construction that keeps a best-chain protocol Π_{origbc} and a BFT protocol Π_{origbft} as black boxes and transforms them into protocols Π_{bc} and Π_{bft} by structural additions, changed validity rules and changed rules of honest behaviour, so that best-chain blocks and BFT blocks commit to each other; these mutual commitments are the *cross-links*. Every node of Crosslink 2 takes part in both Π_{bc} and Π_{bft}, keeping a view of each; the roles of bft-proposer, bft-validator and bc-block producer are optional, and every party acting in one of them is a node of Crosslink 2.

The definition follows the TFL book, chapter "The Crosslink 2 Construction", section "Subprotocols". Here Π names a protocol; the *Crypto Guide* uses the letter for a scheme. The lineage of the construction is the ebb-and-flow model and snap-and-chat (§2); its additions answer the deficiencies of Remark 2.17. In the classes of Definition 1.1, Crosslink 2 is designed but unspecified.

3.2 Executions, chains and prefix order

Definition 3.2 (Communication model)**.** Unless a sub-protocol states a stronger model, messages between participants may be arbitrarily delayed or dropped, each message is received at most once, and messages are non-malleably authenticated as coming from their sender wherever a sub-protocol needs it. A sub-protocol that needs a stronger model imports it explicitly.

Definition 3.3 (Execution and view)**.** An *execution* of a protocol is the set of all events, sends and receipts of messages and decisions of participants, of one run from its initiation up to a given time; a prefix of an execution is an execution. Times are totally ordered with least element 0, and every execution starts at time 0. Up to each time an execution has finitely many events, and the view and the best chain (Definition 3.11) of a node change only at its events, so that its best chain changes at finitely many times in every interval $[0, t]$. A node i is *honest at time* t in an execution iff it has followed the protocol up to and including t and the adversary has not, up to t, cut it off from the rest of the network, for example by an eclipse; a node so cut off counts as not honest even when it follows the protocol (TFL book, admonition "Discussion of [GKL2020]'s communication model and network partition."). A *time series* S assigns a value S^t to each time t. A *view* V assigns to each node i and time t a value V_i^t, here a chain.

The total order of times is a modelling convenience; a happens-before order on events would serve as well (TFL book, chapter "The Crosslink 2 Construction", section "Conventions").

Definition 3.4 (Chain)**.** Let x range over the two chain kinds bc and bft. An *x-chain* is a non-empty sequence of x-blocks that starts at the *genesis block* $\mathcal{O}_x$, written $\mathcal{O}_{\mathrm{bc}}$ and $\mathcal{O}_{\mathrm{bft}}$, each later block committing by a collision-resistant hash to its predecessor, its *parent*. The last block is the chain's *tip*, and a block is identified with the chain it tips. The *height* $\mathsf{height}(B)$ of a block B is its number of predecessors, so that the genesis block has height 0 (*Ironwood Guide*, Definition "Block chain, transaction, consensus rules", in §"Scope and status").

The kind bc comprises the blocks of Π_{origbc} and of Π_{bc}, the kind bft those of Π_{origbft} and of Π_{bft}; a property relative to one protocol names it, as in "valid in Π_{origbft}". The TFL book writes a star for a metavariable over protocol names and as a wildcard within them; the variable x over the two kinds replaces it. The identification of a block with its chain is sound under collision resistance of the parent hash: exactly one chain has a given tip. The book uses heights without defining them; Definition 3.4 supplies the definition.

Definition 3.5 (Truncation and depth)**.** For an x-chain C and $k \in \mathbb{N}$, the chain $C\lceil_x^k$, written $C\lceil^k$ where the kind is clear, is the prefix of C whose tip has height $\max\{0, \mathsf{height}(C) - k\}$: the chain C without its last k blocks, or the genesis chain if C has at most k blocks. The block at *depth* d of C is the tip of $C\lceil^d$; the tip of C is at depth 0. For an object P that names a parent x-block B without being an x-block itself, and for $k \geq 1$, the truncation is extended by $P\lceil_x^k := B\lceil_x^{k-1}$.

The block at depth d of C is the last one that no rollback of d blocks can remove; the genesis chain cannot roll back. For $d \leq \mathsf{height}(C)$ it has height $\mathsf{height}(C) - d$, the depth convention of the *Ironwood Guide*, §"Anchors".

Remark 3.6 (Depth and confirmations). The *Consensus Guide*, Definition "Confirmations" (§"Work, and the best chain"), counts confirmations inclusively: a transaction in a block of the best chain has as many confirmations as there are blocks from that block to the tip, both included. A transaction in the block at depth $d \le \mathsf{height}(C)$ of the best chain C therefore has $d + 1$ confirmations, and for $k \le \mathsf{height}(C)$ the chain $C\lceil^k$ holds exactly the blocks of C whose transactions have at least $k + 1$ confirmations. For $k > \mathsf{height}(C)$ the chain $C\lceil^k$ is the genesis chain, whose block has fewer than $k + 1$ confirmations. The ebb-and-flow paper uses "depth" for what this volume calls height.

Definition 3.7 (Prefix order and agreement)**.** For x-chains B and C, the relation $B \preceq_x C$ holds iff B is an initial segment of C, equality included, and $B \prec_x C$ iff $B \preceq_x C$ and $B \neq C$; the subscript is omitted where the kind is clear. The chains B and C *agree*, written $B \between C$, iff $B \preceq_x C$ or $C \preceq_x B$; they *conflict* iff they do not agree. A time series S of x-chains, or a sequence of them indexed by an ordered set, is *x-linear* iff $t \le u$ implies $S^t \preceq_x S^u$.

Agreement is reflexive and symmetric but not transitive: the genesis block agrees with every block, including two blocks that conflict. It is therefore never written as an equivalence.

Definition 3.8 (Last common ancestor)**.** For x-chains A and B, the *last common ancestor* $\mathsf{lca}(A, B)$ is the longest x-chain C with $C \preceq_x A$ and $C \preceq_x B$.

The chain $\mathsf{lca}(A, B)$ exists, because $\mathcal{O}_x$ is a prefix of both A and B and the common prefixes are finitely many; and it is unique, because the prefixes of A are its initial segments, one of each length, so two common prefixes of the same length coincide. The book uses the function without defining it. If $A \preceq_x B$, then $\mathsf{lca}(A, B) = A$.

Definition 3.9 (Agreement on a view)**.** An execution has *Agreement* on a view V iff for all times t and u and all nodes i and j, $i = j$ permitted, such that i is honest at t and j is honest at u, the chains V_i^t and V_j^u agree.

Lemma 3.10 (Linear prefix)**.** *If $A \preceq_x C$ and $B \preceq_x C$, then $A \between B$.*
Status: proved (TFL book).

The TFL book states and proves the lemma in chapter "The Crosslink 2 Construction", section "Conventions"; the proof below expands its one-line argument.

Proof. The chains A and B are initial segments of C, and the initial segments of a sequence are totally ordered by length: the shorter of A and B is an initial segment of the longer. □

3.3 The best-chain protocol

The interface of this subsection is the TFL book's model for best-chain protocols (chapter "The Crosslink 2 Construction", section "Model for best-chain protocols"). It is stated for Π_{origbc} and holds for Π_{bc} with the validity rules that the construction adds.

Definition 3.11 (Best-chain protocol)**.** A *best-chain protocol* Π_{origbc} has the following interface.

(i) *Views and validity.* The view of a node is a set of bc-chains rooted at $\mathcal{O}_{\text{bc}}$. A validity rule on bc-blocks, the consensus rules, depends only on the content of the block and its ancestors, and a non-genesis block is valid only if its parent is valid; a *valid chain* is a chain of valid blocks.

(ii) *Extension.* Informally, it is hard to extend a valid chain unless one is selected by a random process that chooses block producers in proportion to a resource, with an approximately known and consistent time distribution, under an assumption on the share of the resource that honest nodes hold.

(iii) *Score and best chain.* A function score maps valid chains into a set with a strict total order $<$, and $\mathsf{score}(H\lceil^1) < \mathsf{score}(H)$ for every valid non-genesis chain H. The *best chain* ch_i^t of node i at time t is a valid chain of highest score in its view, any rule breaking ties, and $\mathsf{ch}_i^0 := \mathcal{O}_{\text{bc}}$; the best chain of an honest node is valid.

(iv) *Transactions.* A valid block commits to a collection of transactions. The consensus rules on transactions are unchanged by Crosslink 2, including the rules that depend on the height of the block or on earlier blocks: transactions are neither reordered nor sanitised, and a block enters a chain of Crosslink 2 with its entire parent chain, so heights are preserved.

(v) *Headers.* Each bc-block has a *bc-header* that commits to it. Header validity is necessary, but not sufficient, for block validity. For proof of work, the proof of work and the difficulty of a block are checkable from its header, given a valid parent chain.

Clause (ii) is informal and enters no proof. The book asks of a proof-of-work protocol that the score be its accumulated work.

Remark 3.12 (The Zcash instance). For Zcash the score of clause (iii) and the header check of clause (v) are specified facts. The *work* $\mathsf{work}(B)$ of a block B is $\lfloor 2^{256} / (\mathsf{ToTarget}(\mathsf{nBits}) + 1) \rfloor$ (*Consensus Guide*, Definition "Work of a block", in §"Work, and the best chain"; protocol specification, §"Definition of Work"), where ToTarget converts the header field nBits to its target threshold, and the best valid block chain is the valid chain of greatest total work, ties between leaf blocks going to the block received first (*Consensus Guide*, Definition "Best valid block chain"; protocol specification, §"The Block Chain"). The score of a chain C is therefore

$$\mathsf{score}(C) = \sum_{B \preceq_{\text{bc}} C} \mathsf{work}(B).$$

For a valid block at height h, the field nBits equals $\mathsf{ToCompact}(\mathsf{Threshold}(h))$ (protocol specification, §"Block Header Encoding and Consensus"), where $\mathsf{Threshold}(h) \le \mathsf{PoWLimit} < 2^{256}$ (§"Difficulty adjustment"), except that on Testnet from height 299,188 a minimum-difficulty block, whose header timestamp nTime exceeds that of its predecessor by more than a threshold,

has nBits = ToCompact(PoWLimit) (ZIP 205, "Change to difficulty adjustment on Testnet"; ZIP 208, "Minimum difficulty blocks on Testnet"); since ToCompact rounds down (§"nBits conversion"), ToTarget(nBits) ≤ PoWLimit in both cases, so work(B) ≥ 1 for every valid block, and score(H) = score($H\lceil^1$) + work(H) > score($H\lceil^1$) for every valid non-genesis chain H; the tie rule is one that the interface allows. Header-checkable proof of work is the condition of the *Fly-Client Guide*, Definition "Block header" (§"Header-only verification"): a header satisfies proof of work, and its field nBits equals the value that the difficulty adjustment assigns from the preceding headers, or, for a minimum-difficulty block on Testnet, the encoding of PoWLimit, so the check needs the headers of the ancestors (*Consensus Guide*, §"Difficulty adjustment"; protocol specification, §"Difficulty adjustment"). ZIP 218 (Draft) changes the averaging window of that adjustment from NU7 (ZIP 218, "Averaging window"); the volume states no window length.

Definition 3.13 (Coinbase-only block)**.** A *coinbase transaction*, in the TFL book's abstraction, is a bc-transaction that only distributes newly issued funds and has no inputs. The predicate is-coinbase-only-block(B) on bc-blocks holds iff B has exactly one transaction and that transaction is a coinbase transaction; the block is then *coinbase-only*.

The abstraction is narrower than the deployed coinbase transaction, which also collects the fees of the block's other transactions and has consensus rules of its own (*Ironwood Guide*, Definition "Coinbase transaction", in §"The transaction format"; protocol specification, §"Coinbase Transactions").

Remark 3.14 (Switching depth). The book asks informally of Π_{origbc} that, unless an adversary can censor the node's knowledge of other chains, it be hard to make a node switch to a best chain whose last common ancestor with its previous best chain lies more than σ blocks below that previous chain's tip; the volume writes a generic depth k. The requirement enters no proof. Its precise form is Prefix Consistency (Definition 3.15, below), whose instance $i = j$ bounds the depth of a node's own switches; Prefix Agreement (Definition 3.16) gives no such bound.

Definition 3.15 (Prefix Consistency)**.** For $k \in \mathbb{N}$, an execution of a best-chain protocol has *Prefix Consistency* at depth k iff for all times $t \le u$, every node i honest at t and every node j honest at u, $i = j$ permitted,

$$\mathsf{ch}_i^t\lceil_{\text{bc}}^k \preceq_{\text{bc}} \mathsf{ch}_j^u.$$

The property is the consistency of Pass, Seeman and shelat, "Analysis of the Blockchain Protocol in Asynchronous Networks" (EUROCRYPT 2017), Section 3.3, which, unlike the common-prefix property of Garay, Kiayias and Leonardos, "The Bitcoin backbone protocol: analysis and applications" (EUROCRYPT 2015), also compares a node with its own later chains. The book assumes it of executions of Π_{origbc}; later sections state it of Π_{bc}.

Definition 3.16 (Prefix Agreement)**.** For $k \in \mathbb{N}$, an execution of a best-chain protocol has *Prefix Agreement* at depth k iff it has Agreement on the view $(i, t) \mapsto \mathsf{ch}_i^t\lceil_{\text{bc}}^k$ (Definition 3.9): for all times t and u and all nodes i honest at t and j honest at u, $i = j$ permitted, $\mathsf{ch}_i^t\lceil_{\text{bc}}^k \preceq\succeq \mathsf{ch}_j^u\lceil_{\text{bc}}^k$.

Prefix Consistency compares a truncated chain with an untruncated one, Prefix Agreement two truncated chains. Both are stated at a generic depth k, so that one definition serves every depth at which later sections instantiate them. Prefix Consistency at depth k implies Prefix Agreement at depth k: for $t \le u$, both $\mathsf{ch}_i^t\lceil^k$ (by Prefix Consistency) and $\mathsf{ch}_j^u\lceil^k$ (Definition 3.5) are prefixes of ch_j^u, so they agree by Lemma 3.10; the case $u < t$ is symmetric. The converse fails: a node whose best chain moves from $\mathcal{O}_{\mathrm{bc}}a_1a_2$ to $\mathcal{O}_{\mathrm{bc}}a_1'$ keeps Prefix Agreement at depth 1 and breaks Prefix Consistency.

Remark 3.17 (Properties of executions). Prefix Consistency and Prefix Agreement, and the properties of the BFT protocol and of the ledgers stated later in terms of executions, are unconditional properties of single executions. The probability that an execution has them is a separate question and is not part of the definitions: a statement "every execution with property $\mathcal{P}$ has property $\mathcal{Q}$" bounds the probability of failure of $\mathcal{Q}$ by that of $\mathcal{P}$ under any distribution of executions (TFL book, section "Model for best-chain protocols"). Assuming Prefix Consistency at depth k for proof of work implicitly excludes network partitions longer than about k block times: after the best chain on one side of a full partition has grown by more than k blocks, its truncation contains blocks that the other side cannot know. A BFT protocol, by contrast, can remain safe under asynchrony. A node that the adversary cuts off from the rest of the network, for example by an eclipse, is counted as not honest (Definition 3.3), even though it follows the protocol, so that the properties make no statement about it. This modelling matches the protocol specification, §"The Block Chain": "the vast majority of well-connected nodes should eventually agree on their best valid block chain up to that height".

Remark 3.18 (Status of the prefix properties for Zcash). For Zcash proof of work, Prefix Consistency and Prefix Agreement at any depth, at the 75-second block spacing before NU7 or the 25-second spacing of ZIP 218 (Draft), "Block target spacing", are assumptions, not results. No lower volume proves a prefix property: the analysis of the *Consensus Guide*, §"Confirmation requirements", rests on its Assumption "Constant difficulty" (§"The catch-up probability") and its first-seen tie model, in which a private branch displaces the public one only once it carries strictly more work, and the *FlyClient Guide*'s Assumption "Persistence and liveness" (§"The variable-difficulty backbone model") is unproved in its model. No source proves either property for Zcash, and no source computes an adequate depth under the 25-second spacing. Every result of this volume whose premise is a prefix property is therefore conditional for Zcash.

Status: open problem.

3.4 The BFT protocol

The interface of this subsection is the TFL book's model for BFT protocols (chapter "The Crosslink 2 Construction", section "Model for BFT protocols"). It is stated for Π_{origbft} and holds for Π_{bft} with the validity rules that the construction adds.

Definition 3.19 (BFT protocol)**.** A *BFT protocol* Π_{origbft} has the following interface.

(i) *Views and validity.* The view of a node is a set of bft-chains rooted at $\mathcal{O}_{\mathrm{bft}}$. A validity rule on bft-blocks depends only on the content of the block and its ancestors, and a non-

genesis block is valid only if its parent is valid. A *bft-valid block* is a bft-block that satisfies the rule, and a *bft-valid chain* a chain of such blocks.

(ii) *BFT epochs and proposals.* An execution proceeds in a sequence of *BFT epochs*. In each BFT epoch an honest proposer for that BFT epoch may make a *bft-proposal*, which names its parent bft-block and its BFT epoch and is signed by its proposer; the signature is part of the proposal.

(iii) *Proposal validity.* A validity rule on proposals depends only on the content of the proposal, its parent block and the validity of the proposer's signature; a *valid proposal* satisfies it.

A proposal names a parent bft-block without being one, so Definition 3.5 gives $P\lceil^{k}_{\text{bft}} = B\lceil^{k-1}_{\text{bft}}$ for a proposal P with parent B and $k \geq 1$.

Assumption 3.20 (Strong unforgeability of proposer signatures)**.** The signature scheme of proposers, used for the signature on a proposal (the *inner* signature) and for the signature on the block formed from it (the *outer* signature), is strongly unforgeable under chosen-message attack in the sense of the *Crypto Guide*, Definition "Strong unforgeability" (§"Syntax and security goal").

Status: assumption.

No lower volume proves strong unforgeability for a concrete scheme, and no source fixes a signature scheme for proposers, so the property is a hypothesis.

Definition 3.21 (Voting units and notarisation)**.** Each BFT epoch has a fixed number $n \in \mathbb{N}^{+}$ of *voting units*, distributed among the validators of the BFT protocol, which use them to vote for proposals of that BFT epoch. The holders of a BFT epoch's units form its *finaliser set*, and each holder is a *finaliser*. A unit is *cast* for a proposal P at a time iff P is a valid proposal and a ballot for P authenticated by the unit's holder exists at that time. A *notarisation proof* proof_P for P is obtainable from, and only from, knowledge of ballots cast for P that together satisfy a *notarisation rule*; the rule requires at least two thirds of the units of P's BFT epoch to be cast for P, and it may require more. *Soundness* of notarisation proofs is the clause "only from": a valid proof for P exists only if ballots meeting the rule have been cast.

The term "finaliser" is Shielded Labs'; the volume uses it for the holders of voting units throughout. In this volume n denotes only the number of voting units of a BFT epoch.

Definition 3.22 (Non-honest casting)**.** A voting unit is *cast non-honestly* for a proposal of its BFT epoch iff it is cast by a party other than its holder, including after compromise of the holder's key; or it is *double-cast*, that is, ballots cast it for two distinct proposals of the BFT epoch; or its holder, following the conditions for honest voting that the protocol prescribes, would not have cast it in its view. For Π_{bft} those conditions are the criteria of Construction 4.14. Several ballots casting one unit for one proposal are not non-honest, provided the unit is not counted twice towards the notarisation rule.

Assumption 3.23 (One-third bound)**.** In the execution considered, for every BFT epoch, strictly fewer than one third of its voting units are ever cast non-honestly.

Status: assumption.

The assumption is the book's "one-third bound on non-honest voting", a property of executions of Π_{bft}. The word "ever" ranges over the whole execution: a key compromised long after its BFT epoch can still cast that BFT epoch's units. The keys of honest validators must therefore remain secret indefinitely, and a rotated key must be securely deleted (TFL book, chapter "The Crosslink 2 Construction", section "Model for BFT protocols", admonition "Security caveat").

Assumption 3.24 (Unforgeable ballots)**.** In every execution considered, no voting unit is cast by a party other than its holder: ballot authentication is unforgeable, and no holder's key is compromised.

Status: assumption.

The volume adds this hypothesis. The book states only that a ballot is authenticated by the holder of the unit and fixes no scheme. Shielded Labs' overview signs ballots with Ed25519 (§10.7), an instance of the *Crypto Guide*'s Construction "EdDSA, schematically" (§"RedDSA: re-randomisable Schnorr for Orchard"); Brendel, Cremers, Jackson and Zhao, "The Provable Security of Ed25519: Theory and Practice" (IEEE Symposium on Security and Privacy 2021), analyse the unforgeability of each variant of Ed25519 separately, and which variant the design uses is unverified. No specification fixes a ballot scheme. Assumption 3.23 does not imply it: a forged cast counts as non-honest under Definition 3.22, and the bound limits how many units are cast non-honestly without excluding any one such cast. Of the results of this volume, only the withholding power of a coalition of finalisers rests on it.

Definition 3.25 (Integer threshold)**.** For a BFT epoch with n voting units, the notarisation rule requires at least

$$q := \lceil 2n/3 \rceil$$

units cast for the proposal, and Assumption 3.23 reads $m \le \lceil n/3 \rceil - 1$, where m is the number of units of the BFT epoch ever cast non-honestly.

The two forms are exact, because unit counts are integers: an integer is at least $2n/3$ iff it is at least $\lceil 2n/3 \rceil$, and below $n/3$ iff it is at most $\lceil n/3 \rceil - 1$. The book leaves the integer form implicit. A count of $2f + 1$ units with $f = \lfloor (n-1)/3 \rfloor$, the usual form for protocols that tolerate f faults, does not meet the rule unless $n = 3f + 1$: writing $n = 3f + 1 + s$ with $s \in \{0, 1, 2\}$, the inequality $2f + 1 \ge 2n/3 = 2f + (2 + 2s)/3$ holds iff $s = 0$.

Definition 3.26 (bft-block)**.** A *bft-block* is a pair (P, proof_P) of a proposal P and a notarisation proof for it, signed again by the proposer of P (the outer signature). It is valid in Π_{origbft} iff P is a valid proposal, proof_P is a valid proof that ballots cast for P satisfy the notarisation rule, and the outer signature on (P, proof_P) is valid. The parent reference of a proposal hashes the whole parent block: proposal, proof and outer signature. Neither proof_P nor the outer signature is

unique for a given P. A bft-block is *in honest view* iff some party observes it at a time at which that party is honest.

Since each block commits to its proposal and each proposal hashes its whole parent block with a collision-resistant hash, every bft-block determines its chain of ancestors, although one proposal may underlie several valid blocks. By Assumption 3.20, the outer signature is not malleable by a third party.

Lemma 3.27 (Per-BFT-epoch uniqueness)**.** *In an execution that satisfies Assumption 3.23 and in which notarisation proofs are sound, all bft-valid blocks of one BFT epoch in honest view carry the same proposal.*

Status: proved (this volume), of a designed-but-unspecified construction.

Proof. Let B and B' be bft-valid blocks of a BFT epoch with n voting units, carrying proposals P and P', and suppose $P \neq P'$. By Definition 3.26 both blocks carry valid notarisation proofs, so by soundness and Definition 3.25 there are sets S and S' of at least $q = \lceil 2n/3 \rceil$ units cast for P and for P' respectively. Then

$$|S \cap S'| \geq |S| + |S'| - n \geq 2q - n \geq \lceil n/3 \rceil.$$

The last inequality holds for every n: writing $n = 3a + r$ with $r \in \{0, 1, 2\}$, one has $q = 2a + r$, so $2q - n = a + r$, while $\lceil n/3 \rceil$ is a for $r = 0$ and $a + 1$ otherwise. Every unit in $S \cap S'$ is cast for the two distinct proposals P and P' of one BFT epoch, so it is double-cast, hence cast non-honestly (Definition 3.22). The number m of units of the BFT epoch ever cast non-honestly is thus at least $\lceil n/3 \rceil$, contradicting $m \leq \lceil n/3 \rceil - 1$ (Definition 3.25). Hence $P = P'$. □

The proof does not use that the blocks are in honest view. The book's version, the theorem "On bft-valid blocks for a given epoch in honest view" (chapter "Security Analysis of Crosslink 2"), is adapted from Lemma 1 of Chan and Shi's Streamlet paper; it lies below the chapter's marker that the text after it is not updated for Crosslink 2, but it uses no ledger notation, and it argues with the real bound $2n/3$. The proof above gives the integer form, which the quorum script also checks for every $n \leq 1000$.

Remark 3.28 (Honestly submitted blocks). An *honest proposal* is a proposal made for a BFT epoch by its proposer at a time at which that proposer is honest, and a bft-block is *honestly submitted* iff its proposer submits it at a time at which that proposer is honest. Honesty at a time covers all earlier conduct (Definition 3.3), and the protocol lets a proposer make at most one proposal and at most one outer signature per BFT epoch, so, when each BFT epoch has a single proposer, at most one honestly submitted bft-block exists per BFT epoch; a dishonest proposer can produce several valid blocks for one proposal, with different proofs or outer signatures. The book states that the property matters for liveness and marks its use there as unchecked ("TODO check that we are correctly using this in the liveness analysis").

Status: open problem.

Definition 3.29 (Last final block)**.** The BFT protocol has an efficiently computable function

$$\text{bft-last-final}\colon \{\text{bft-blocks}\} \to \{\text{bft-blocks}\} \cup \{\bot\}$$

that satisfies the following requirements.

(i) The value bft-last-final(C) is $\bot$ iff C is not bft-valid.

(ii) If C is bft-valid, then bft-last-final(C) $\preceq_{\text{bft}} C$, so that bft-last-final(C) is bft-valid.

(iii) If C is bft-valid, then for every bft-valid D with $C \preceq_{\text{bft}} D$, bft-last-final(C) $\preceq_{\text{bft}}$ bft-last-final(D).

(iv) The genesis block satisfies bft-last-final($\mathcal{O}_{\text{bft}}$) = $\mathcal{O}_{\text{bft}}$.

For Π_{origbft} the function is written origbft-last-final.

Definition 3.30 (Finality in context)**.** A bft-block B is *final in the context of* a bft-block C iff $B \preceq_{\text{bft}}$ bft-last-final(C). No bft-block is final other than in a context.

Each bft-valid block C determines its last final block bft-last-final(C), but no bft-block is objectively final. The notion concerns bft-blocks only; the volume defines finality of a best-chain block separately, in the client contract.

Definition 3.31 (Final Agreement)**.** An execution of a BFT protocol has *Final Agreement* iff for all bft-valid blocks C in honest view at a time t and C' in honest view at a time t', bft-last-final(C) $\not\preceq$ bft-last-final(C').

Final Agreement can hold in an asynchronous communication model. If Assumption 3.23 is broken at any time of an execution, Final Agreement may fail from that time on (TFL book, section "Model for BFT protocols").

Remark 3.32 (Adapting a concrete protocol). A concrete BFT protocol may need protocol-specific adaptation to fit the interface before Crosslink 2 transforms it. Proposal validity is to be the strongest property of a proposal that is objectively and feasibly checkable from the proposal and its parent block when the proposal is made, and it includes the check of the proposer's signature. Block validity is to be the strongest property of a block that is objectively and feasibly checkable from the block and its ancestors when the block is added to a chain, and it includes the checks of the notarisation proof and of the outer signature. For a node that has observed a valid block C, a rollback of its view past origbft-last-final(C) is to be infeasible, and its view up to that block is to agree with that of every other honest node; Final Agreement formalises the latter.

Example 3.33 (Adapted Streamlet). In Streamlet (Chan and Shi, "Streamlet: Textbook streamlined blockchains", AFT 2020) a block is notarised once at least two thirds of the players have voted for it, and in a notarised chain the prefix up to the second of three adjacent blocks with consecutive BFT epoch numbers is final. The TFL book adapts it to the interface as follows (section "Model for BFT protocols", admonition "Adapting the Streamlet BFT protocol."). Once a proposal has votes from two thirds of the units, its proposer aggregates them into a notarisation proof,

signs the resulting pair and submits the bft-block; validity in Π_{origbft} then coincides with notarisation and is objectively checkable. For the liveness analysis to carry over, the combined latency of voting, aggregation and submission must be such that every node receives the notarised block within two BFT epochs. For a valid C, the block origbft-last-final(C) is the last valid $B \preceq_{\text{bft}} C$ that is $\mathcal{O}_{\text{bft}}$ or the second of three adjacent blocks with consecutive BFT epoch numbers. Streamlet's liveness theorem guarantees progress after GST whenever five consecutive BFT epochs have honest leaders; the book refers to this condition, and to Streamlet's liveness proof, in the admonition "Why does the Linearity rule allow keeping the same snapshot as the parent?" of its construction chapter.

4 The Crosslink 2 construction

This section constructs Π_{bft} and Π_{bc} from the black boxes Π_{origbft} and Π_{origbc} of Definition 3.1: the parameters, the two cross-link fields and the functions derived from them, the validity rules and the honest behaviour on each side, and Stalled Mode. Rule names are those of the TFL book, chapter "The Crosslink 2 Construction", and each rule that answers an item of Remark 2.17 is tied to that item; the Finality depth rule serves bounded availability (§8). In the classes of Definition 1.1, every field, function and rule of this section is designed but unspecified: the book defines it and no ZIP specifies it (§1.1). Components that no source fixes are marked as open problems where they arise, and each result carries its status. Where no ambiguity arises, a bc-header is identified with the bc-block it commits to (Definition 3.11).

4.1 Parameters

Definition 4.1 (Parameters of Crosslink 2)**.** Crosslink 2 has three parameters, fixed by the protocol:

(i) the *bc-confirmation depth* $\sigma \in \mathbb{N}^+$, the same for every node;

(ii) the *finalisation gap bound* $L \in \mathbb{N}^+$;

(iii) a predicate is-stalled-block on bc-blocks; a bc-block H is *stalled* iff is-stalled-block(H) holds.

Every node uses the one value of σ, both in the rules below and in computing its finalised chain. The book requires L to be "significantly greater than σ" (section "Parameters") and "in practice" at least 2σ (section "Stalled Mode"). This volume does not assume $\sigma \leq L$: Proposition 8.5 shows that, for some best-chain protocol of the interface and every $L < \sigma$, a best chain can carry a non-stalled tip more than L blocks above the finalised chain, so the constraint is derived where it is needed.

The values of σ and L are an open problem. No value is chosen, no source computes an adequate σ at 75-second or at 25-second spacing (Remark 3.18), and no source gives a rule for choosing L; the book's liveness argument leaves its "dependence on L" to a more detailed argument that it does not give (chapter "Security Analysis of Crosslink 2"). No value of σ or L appears in

this volume (§1.1). The predicate is-stalled-block is designed but unspecified: the book proposes coinbase-only stalled blocks for Zcash and never fixes the predicate.

4.2 Structural additions

Definition 4.2 (Cross-link fields)**.** (i) Each bc-header carries, besides the fields of a Π_{origbc} header, a field context_bft that commits to a bft-block, the *context* of the block; for a bc-block H it is written H.context_bft.

(ii) Each proposal and each non-genesis bft-block B carries, besides the fields of Π_{origbft}, a field B.headers_bc: a sequence of exactly σ bc-headers, indexed from 0, deepest first. The genesis bft-block has $\mathcal{O}_{\text{bft}}$.headers_bc = null.

For a bc-chain T of at least $\sigma + 1$ blocks, the *σ-block tail* of T is the sequence of the headers of $T\lceil^{\sigma-1}, \ldots, T\lceil^{1}, T$; its entry j is the header of $T\lceil^{\sigma-1-j}$. Every entry is the header of a non-genesis block, so the deepest has a parent (the book's admonition "Why $\sigma + 1$?").

The field context_bft makes every best-chain block commit to a BFT block from which its finalised prefix is computed, which answers item (4) of Remark 2.17. The field headers_bc is the evidence that item (2) asks for: the headers of σ best-chain blocks above the snapshot, which any party can check.

Definition 4.3 (Snapshot)**.** For a proposal or bft-block B,

$$\mathsf{snapshot}(B) := \begin{cases} \mathcal{O}_{\text{bc}}, & \text{if } B.\text{headers_bc} = \text{null}, \\ B.\text{headers_bc}[0]\lfloor^{1}_{\text{bc}}, & \text{otherwise}, \end{cases}$$

the parent of the block whose header is B.headers_bc[0].

If B.headers_bc is the σ-block tail of a bc-chain T, then $\mathsf{snapshot}(B) = T\lceil^{\sigma}$: entry 0 is the header of $T\lceil^{\sigma-1}$, and truncations compose, since by Definition 3.5 both $(T\lceil^{\sigma-1})\lceil^{1}$ and $T\lceil^{\sigma}$ are the prefix of T whose tip has height $\max\{0, \mathsf{height}(T) - \sigma\}$. The snapshot then lies σ blocks below the block of the newest carried header, and $\mathsf{height}(\mathsf{snapshot}(B)) = \mathsf{height}(T) - \sigma$.

Definition 4.4 (Last final context)**.** For a bc-block H,

$$\mathsf{LF}(H) := \mathsf{bft\text{-}last\text{-}final}(H.\text{context_bft}),$$

the last block final in the context of H.context_bft (Definitions 3.29 and 3.30), and $\mathsf{snapshot}(\mathsf{LF}(H))$ is the *final snapshot* of H. For a bft-block C, $\mathsf{snapshot}(\mathsf{bft\text{-}last\text{-}final}(C))$ is the final snapshot of C.

In the book's prose the final snapshot of a node's best tip is also called its *finalisation point*; Definition 4.5 lowers it to a σ-confirmed prefix of the tip.

Definition 4.5 (Candidate finalisation point)**.** For a bc-block H,

$$\mathsf{candidate}(H) := \mathsf{lca}\big(\mathsf{snapshot}(\mathsf{LF}(H)),\ H\lceil^{\sigma}\big).$$

For the tip H of a node's best chain, $\mathsf{candidate}(H)$ is the node's *candidate finalisation point*.

When the final snapshot of H is a prefix of H, as a rule of Π_{bc} below requires, the last common ancestor reduces to the lower of its two arguments; the reduction is proved with that rule.

Definition 4.6 (Finality depth)**.** For a bc-block H,

$$\mathsf{finality\text{-}depth}(H) := \mathsf{height}(H) - \mathsf{height}(\mathsf{snapshot}(\mathsf{LF}(H))).$$

The finality depth is objective, a function of H through its context and not of any node's view, as a consensus rule must be. It counts the blocks of H's chain above the final snapshot when that snapshot is a prefix of H, which the Last Final Snapshot rule below requires; without that rule a difference of heights would measure no distance along H's chain. This form supersedes that of the book's earlier Crosslink 1, which measures from $\mathsf{lca}(\mathsf{snapshot}(\mathsf{LF}(H)), H)$; when the final snapshot is a prefix of H the last common ancestor is the final snapshot itself, and the two forms coincide.

Remark 4.7 (Purpose of the additions). The book gives four reasons for the additions (section "Structural additions" and its admonitions).

(a) The field headers_bc of a proposal or bft-block shows any party that sees it that σ bc-blocks of valid proof of work existed above its snapshot when the proposal was made; snap-and-chat's snapshot reference carries no such evidence (item (2) of Remark 2.17).

(b) Full headers, not hashes, let a validator check proof of work and difficulty from headers alone before it downloads any block, which limits denial of service: the proof of work of a Zcash block is checkable from its header (*FlyClient Guide*, Definition "Block header", §"Header-only verification"), and its target is a function of the headers of its ancestors and, on Testnet, of its own nTime (*Consensus Guide*, §"Difficulty adjustment"; Remark 3.12).

(c) The genesis bft-block carries no headers, since there are not σ bc-headers to carry at genesis; $\mathsf{snapshot}(\mathcal{O}_{\mathrm{bft}})$ is defined to be $\mathcal{O}_{\mathrm{bc}}$ instead of making $\mathcal{O}_{\mathrm{bft}}$ refer to $\mathcal{O}_{\mathrm{bc}}$, which would close a hash cycle with $\mathcal{O}_{\mathrm{bc}}$.context_bft = $\mathcal{O}_{\mathrm{bft}}$.

(d) One field context_bft suffices, with no separate field for the last final block, because a context determines its last final ancestor through the efficiently computable function bft-last-final (Definition 3.29).

Status: argued (TFL book).

Remark 4.8 (Encoding). No source fixes the encoding of context_bft and headers_bc or the place of the cross-link data in a Zcash block. The specified place for a commitment to new block data is the terminator of the block commitment hash (*FlyClient Guide*, Remark "Extension of the

block commitment hash", §"The block commitment hash"), which no upgrade through NU7 uses. Shielded Labs' *Zcash Crosslink Design Overview* (draft 10, 14 September 2026), section "3. The proof-of-work side: the fat pointer and where it lives", weighs five placements of its own cross-link data, from a commitment through that terminator alone to a change of the header format, judges the commit-only option "probably insufficient, since some data must actually be stored", and lists the choice among the five as open (Part VI, "Open questions", "Header and encoding").

Status: open problem.

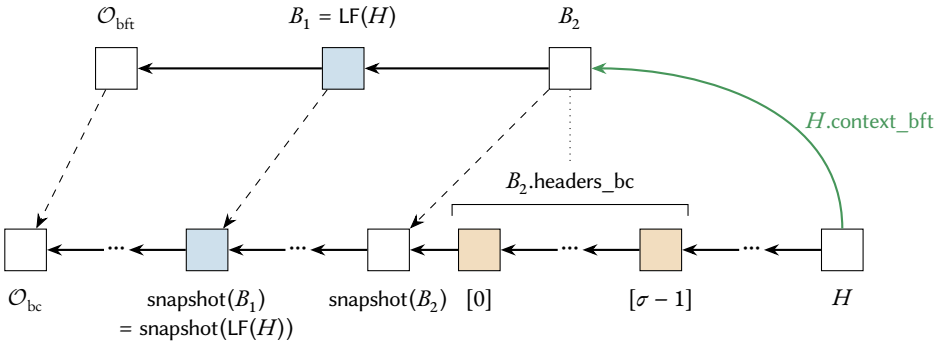


Figure 1. The cross-link structure. Solid black arrows point from child to parent on the bc-chain (below) and the bft-chain $\mathcal{O}_{\text{bft}}$, B_1, B_2 (above); dashed arrows point from a bft-block to its snapshot. The bracket marks the σ headers B_2.headers_bc[0] to B_2.headers_bc[$\sigma - 1$] (orange) that B_2 carries, and snapshot(B_2) is the parent of B_2.headers_bc[0]. The green arrow is H.context_bft = B_2; in the case drawn the last block final in the context of B_2 is B_1 (blue), so LF(H) = B_1 and the final snapshot of H is snapshot(B_1) (blue). The Linearity rule appears as snapshot(B_1) lying at or below snapshot(B_2) on one bc-chain.

4.3 Validity rules of the BFT protocol

Definition 4.9 (Validity in Π_{bft})**.** *Genesis bft-block rule*: the genesis bft-block $\mathcal{O}_{\text{bft}}$ is bft-block-valid, and $\mathcal{O}_{\text{bft}}$.headers_bc = null. A proposal B, respectively a non-genesis bft-block B, is *bft-proposal-valid*, respectively *bft-block-valid*, iff the following hold.

(i) *Inherited origbft rules*: B satisfies the proposal-validity, respectively block-validity, rules of Π_{origbft} (Definitions 3.19 and 3.26). These rules are assumed to include the *Parent rule*, that the parent of B is bft-block-valid; for a block they include a valid notarisation proof (Definition 3.21) and a valid outer signature.

(ii) *Linearity rule*: $\text{snapshot}(B\lceil^1_{\text{bft}}) \preceq_{\text{bc}} \text{snapshot}(B)$.

(iii) *Tail Confirmation rule*: B.headers_bc is the σ-block tail of a bc-valid chain.

Proposal validity and block validity may differ, as they may in Π_{origbft}. The function bft-last-final of Π_{bft} is the function origbft-last-final of Π_{origbft}, read with bft-block-validity and $\mathcal{O}_{\text{bft}}$: finality in context (Definition 3.30) is unchanged.

For Π_{bft}, the bft-valid blocks of Definition 3.19, on which Definitions 3.29 and 3.31 and Lemma 3.27 are stated, are the bft-block-valid blocks.

For a proposal P with parent block B', $P\lceil^1_{\text{bft}} = B'$ (Definition 3.5), so the Linearity rule compares a proposal with its parent block, and a block with its parent. The rule is relative to the parent's snapshot even when the parent is not final in the context of B, because it must hold if the parent is later final in the context of a descendant; equality is permitted, so a block may repeat its parent's snapshot. The Tail Confirmation rule makes the blocks behind the carried headers, not only the headers, part of bft-validity: bc-validity of the chain needs the blocks themselves, and honest validators download and check them before voting.

Along a valid bft-chain the Linearity rule makes the snapshots bc-linear: for bft-blocks C and C',

$$C \preceq_{\text{bft}} C' \text{ and } C' \text{ bft-block-valid} \implies \mathsf{snapshot}(C) \preceq_{\text{bc}} \mathsf{snapshot}(C'). \tag{1}$$

The proof is by induction on the number of blocks of C' not in C. If there are none, $C = C'$. Otherwise C' is not $\mathcal{O}_{\text{bft}}$, its parent $C'\lceil^1_{\text{bft}}$ is bft-block-valid by the Parent rule and has C as a prefix, so the induction hypothesis gives $\mathsf{snapshot}(C) \preceq_{\text{bc}} \mathsf{snapshot}(C'\lceil^1_{\text{bft}})$, and the Linearity rule for C' gives $\mathsf{snapshot}(C'\lceil^1_{\text{bft}}) \preceq_{\text{bc}} \mathsf{snapshot}(C')$.

Remark 4.10 (Simultaneous definition of validity). Definition 4.9 refers to bc-validity through the Tail Confirmation rule, and Definition 4.17 refers to bft-validity through the Valid context rule; the two are one simultaneous definition of bft-validity and bc-validity. Each rule of either definition refers only to blocks that the block under test commits to by hash, directly or through a chain of commitments: its ancestors, the blocks behind its carried headers, its context, and their ancestors. Validity is defined by recursion along this commitment relation, restricted to its well-founded part: a block from which some chain of commitments is infinite, in particular a block that commits through a cycle to its own hash, is neither bft-block-valid nor bc-block-valid. The genesis case serves the same end: $\mathcal{O}_{\text{bc}}$ commits to $\mathcal{O}_{\text{bft}}$ through $\mathcal{O}_{\text{bc}}$.context_bft, so $\mathcal{O}_{\text{bft}}$ carries no bc-header.

Remark 4.11 (Objectivity of validity). Validity in Π_{bft} depends only on a proposal or block and on the blocks it commits to by hash, never on an observer's best chain, and it is checked independently of the ballots for a proposal. A proposal that fails a rule of Definition 4.9 remains invalid however many ballots name it: no voting unit counts as cast for it (Definition 3.21), and no honest node accepts a block formed from it as bft-block-valid, whatever notarisation proof the block carries.

Remark 4.12 (Purpose of the Linearity rule). Besides the route to Assured Finality under Final Agreement (Theorem 6.5; Remark 6.7), the book gives the following reasons for the rule (admonitions "What does the Linearity rule do?" and "Why does the Linearity rule allow keeping the same snapshot as the parent?").

(a) It replaces the Increasing Score rule of Crosslink 1, which required each snapshot of a valid bft-chain to equal its parent's or to have higher score, and it implies that rule, since score strictly increases along a bc-chain (Definition 3.11).

(b) It excludes proposals of a bc-valid chain that forks from a much earlier block, where the difficulty could have been much lower.

(c) It limits the disruption that an adversary that has subverted Π_{bft} can cause to the best chain: the finalised chain is a Π_{bc} chain, so its safety is no worse than that of Π_{bc} alone for a rollback of any depth.

(d) Either the snapshot advances on that of the parent bft-block, or it needs no further validation, having been validated already.

(e) It removes sanitisation: by (1) the snapshots of a valid bft-chain extend one another, so the finalised history is one bc-chain whose transactions keep their consensus rules, which answers item (3) of Remark 2.17.

(f) It permits equality so that an honest proposer can always make a valid proposal, by repeating its parent's snapshot; liveness of Π_{origbft} may require honest proposals at a minimum rate. The book prefers this to null proposals, which would need rules of their own in Π_{origbft}.

Status: argued (TFL book).

4.4 Honest proposal and honest voting

Construction 4.13 (Honest proposal)**.** An honest proposer i that makes a proposal P at time t proceeds as follows.

(i) It makes no proposal before its best chain ch_i^t has at least $\sigma + 1$ blocks.

(ii) It sets P.headers_bc to the σ-block tail of ch_i^t if P then satisfies the Linearity rule, and otherwise to the field headers_bc of the parent bft-block of P.

When ch_i^t has exactly $\sigma + 1$ blocks the first branch gives the snapshot $\mathcal{O}_{\text{bc}}$, which satisfies the Linearity rule against $\mathcal{O}_{\text{bft}}$ because $\mathsf{snapshot}(\mathcal{O}_{\text{bft}}) = \mathcal{O}_{\text{bc}}$. When the parent is $\mathcal{O}_{\text{bft}}$ the first branch always applies, since $\mathcal{O}_{\text{bc}}$ is a prefix of every bc-chain, so the second branch never copies the null field of $\mathcal{O}_{\text{bft}}$. The first branch satisfies the Tail Confirmation rule because an honest node's best chain is bc-valid (Definition 3.11); the second satisfies the Linearity rule with equality and the Tail Confirmation rule because the parent does. The first branch can fail the Linearity rule after a reorganisation of the proposer's best chain, since the block σ deep on the new chain need not descend from the parent's snapshot.

Construction 4.14 (Honest voting)**.** An honest validator v that considers a proposal P at time t proceeds as follows.

(i) It updates its views of both sub-protocols with P.headers_bc: it downloads the bc-blocks of those headers that it lacks and checks their bc-validity, and for each such block it validates the bft-chain that the block's context_bft references, if it has not seen it, and recursively the bc-chains carried by that bft-chain's blocks. Non-recursive checks come first: the notarisation proof of a bft-block before its ancestry, and the proof of work of carried headers, with their connection to an already validated bc-chain, before any block is downloaded.

(ii) It then votes for P only if both criteria hold:

- *Valid proposal criterion*: P is bft-proposal-valid;
- *Confirmed best-chain criterion*: $\mathsf{snapshot}(P) \preceq_{\mathrm{bc}} \mathsf{ch}_v^t\lceil^{\sigma}$.

The Confirmed best-chain criterion places $\mathsf{snapshot}(P)$ at depth at least σ in the validator's best chain after the update; the block of the newest carried header, $P.\mathsf{headers_bc}[\sigma - 1]$, need not lie on that chain, which may fork above the snapshot. Validity of the carried blocks is checked in step (i) and not inferred from the criterion, because the criterion concerns the validator's best chain while validity is objective (Remark 4.11). Snap-and-chat's voting condition asked only that the validator view the snapshot as confirmed, and its proposals carried no headers (item (2) of Remark 2.17). Steps (i) and (ii) are the honest-voting conditions of Π_{bft} to which Definition 3.22 refers.

Remark 4.15 (Refusal to vote). Honest voting is an "only if" condition, not "if and only if": a validator may refuse to vote for a proposal that meets both criteria, on any good ground, including information obtained outside the protocol, at a possible cost to the liveness of Π_{bft}. The book declines to make refusal on long rollbacks a voting rule only because such a rule would be stateful (chapter "Questions About Crosslink"). The book's argument in section "Complementarity" of the chapter "The Arguments for Bounded Availability and Finality Overrides", that validators asked to stop validating cause a finalisation stall which Stalled Mode then contains, depends on this allowance.

Remark 4.16 (Validation cost). Honest voting validates recursively: a referenced bft-chain in full, the bc-chain carried by each of its blocks, the bft-chains those bc-blocks reference, and so on. The book bounds the cost only by the validation order and by the Linearity rule (admonition "Wait what, how much validation is that?"). Non-recursive checks come first; the proof of work of carried headers, and their connection to a validated bc-chain, is checked before blocks are downloaded, so bc-block validation proceeds no faster than the network finds proof of work; and by (1) the snapshots of one valid bft-chain lie on one bc-chain, so one bc-chain needs validation per bft-chain, which is, if Π_{bft} is safe and the adversary has no overwhelming advantage in proof of work, effectively one bc-chain overall with short side branches. No bound on the validation cost is proved.

Status: argued (TFL book).

4.5 Validity rules of the best-chain protocol

Definition 4.17 (Validity in Π_{bc})**.** This definition completes the simultaneous definition of Remark 4.10. *Genesis bc-block rule*: $\mathcal{O}_{\mathrm{bc}}.\mathsf{context_bft} = \mathcal{O}_{\mathrm{bft}}$; hence $\mathsf{LF}(\mathcal{O}_{\mathrm{bc}}) = \mathcal{O}_{\mathrm{bft}}$ (Definition 3.29), and the final snapshot of $\mathcal{O}_{\mathrm{bc}}$ is $\mathsf{snapshot}(\mathcal{O}_{\mathrm{bft}}) = \mathcal{O}_{\mathrm{bc}}$. A non-genesis bc-block H is *bc-block-valid* iff the following hold.

(i) *Inherited origbc rules*: H satisfies the block-validity rules of Π_{origbc} (Definition 3.11), which include validity of its parent.

(ii) *Valid context rule*: H.context_bft is bft-block-valid.

(iii) *Extension rule*: $\mathsf{LF}(H\lceil^1_{\mathrm{bc}}) \preceq_{\mathrm{bft}} \mathsf{LF}(H)$.

(iv) *Last Final Snapshot rule*: $\mathsf{snapshot}(\mathsf{LF}(H)) \preceq_{\mathrm{bc}} H$.

(v) *Finality depth rule*: $\mathsf{finality\text{-}depth}(H) \leq L$ or $\mathsf{is\text{-}stalled\text{-}block}(H)$.

A bc-block is *bc-valid* iff it is $\mathcal{O}_{\mathrm{bc}}$ or bc-block-valid, and a *bc-valid chain* is a chain of bc-valid blocks; by the Inherited origbc rules a bc-valid block tips a bc-valid chain. The book classes every change as one to non-contextual validity; contextual validity, the validity of a block's transactions against the state of its chain, is that of Π_{origbc}.

The Valid context rule makes $\mathsf{LF}(H)$ a bft-block-valid block, not $\perp$ (Definition 3.29). The Extension rule makes the commitment of item (4) of Remark 2.17 monotone along a chain: the last final context of a block extends that of its parent. The Last Final Snapshot rule places the final snapshot on the block's own chain, so that its transactions can spend outputs of the finalised prefix, and makes the finality depth a distance along that chain. The Finality depth rule forces the predicate is-stalled-block, not coinbase-only blocks; coinbase-only blocks are the policy that the book proposes for Zcash.

Lemma 4.18 (Candidate under the Last Final Snapshot rule)**.** *Let H be a bc-block with* $\mathsf{snapshot}(\mathsf{LF}(H)) \preceq_{\mathrm{bc}} H$*, as the Last Final Snapshot rule requires. Then* $\mathsf{candidate}(H)$ *is the shorter of* $\mathsf{snapshot}(\mathsf{LF}(H))$ *and* $H\lceil^{\sigma}$*.*

Status: proved (this volume), of a designed-but-unspecified construction.

Proof. Both arguments of the last common ancestor are prefixes of H: the final snapshot by hypothesis, and $H\lceil^{\sigma}$ by Definition 3.5. By Lemma 3.10 they agree; let A be the shorter and B the other, so $A \preceq_{\mathrm{bc}} B$. Then A is a common prefix of both, and every common prefix of both is a prefix of A and so no longer than A; hence $\mathsf{lca}(A, B) = A$ (Definition 3.8). □

A script checks the lemma on instances.

Lemma 4.19 (Prefix Lemma)**.** *If H_1 and H_2 are bc-valid blocks with $H_1 \preceq_{\mathrm{bc}} H_2$, then* $\mathsf{LF}(H_1) \preceq_{\mathrm{bft}} \mathsf{LF}(H_2)$.

Status: proved (TFL book), of a designed-but-unspecified construction.

Proof. The book proves the lemma by induction on the distance between H_1 and H_2 with the Extension rule (chapter "Security Analysis of Crosslink 2"). If $H_1 = H_2$ there is nothing to prove. Otherwise H_2 is not genesis and $H_1 \preceq_{\mathrm{bc}} H_2\lceil^1_{\mathrm{bc}}$, which is bc-valid by the Inherited origbc rules and lies one block closer to H_1; the induction hypothesis gives $\mathsf{LF}(H_1) \preceq_{\mathrm{bft}} \mathsf{LF}(H_2\lceil^1_{\mathrm{bc}})$, the Extension rule for H_2 gives $\mathsf{LF}(H_2\lceil^1_{\mathrm{bc}}) \preceq_{\mathrm{bft}} \mathsf{LF}(H_2)$, and the prefix order is transitive. □

With (1), the lemma makes final snapshots monotone along a valid chain: if $H_1 \preceq_{\mathrm{bc}} H_2$ are bc-valid, then $\mathsf{snapshot}(\mathsf{LF}(H_1)) \preceq_{\mathrm{bc}} \mathsf{snapshot}(\mathsf{LF}(H_2))$, since $\mathsf{LF}(H_2)$ is bft-block-valid by the Valid context rule and Definition 3.29.

4.6 Honest block production and fork choice

Construction 4.20 (Honest block production)**.** An honest producer of a bc-block H follows the rules of Definition 4.17; in particular it produces a stalled block when the Finality depth rule requires one. To choose H.context_bft it considers the bft-block-valid blocks T of its view whose last final block bft-last-final(T) extends the last final context of the parent,

$$\{\, T : T \text{ bft-block-valid and } \mathsf{LF}(H\lceil^{1}_{\mathrm{bc}}) \preceq_{\mathrm{bft}} \text{bft-last-final}(T) \,\},$$

the *candidate contexts* of H. Among the candidate contexts T for which H with H.context_bft = T satisfies the rules of Definition 4.17, it chooses one of the longest, C, breaking ties by the greatest score(snapshot(bft-last-final(C))) and then by the smallest hash, and sets H.context_bft := C.

The restriction to choices that satisfy the rules is this volume's reading; the book's rule chooses among all candidate contexts, a set defined without the Last Final Snapshot and Finality depth rules, of which a longest member can make H invalid. The book's reasons for the rule are argued (TFL book): choosing the longest chain follows Streamlet, which builds on the longest notarised chain rather than the longest finalised one; and the tie-break by the score of the final snapshot inhibits an adversary from finalising a chain of its own with lower score, while an adversary chain of greater score has already won the race of proof of work.

Proposition 4.21 (Existence of a valid context)**.** *Let G be a bc-valid block, and let H be a bc-block with parent G that satisfies the Inherited origbc rules.*

(i) The context G.context_bft *is a candidate context of H. With* H.context_bft = G.context_bft*, block H satisfies the Valid context, Extension and Last Final Snapshot rules,* finality-depth(H) = finality-depth(G) + 1*, and H is bc-valid if it is stalled whenever this depth exceeds L.*

(ii) Every candidate context satisfies the Valid context and Extension rules as context of H. A candidate context T whose final snapshot conflicts with G violates the Last Final Snapshot rule, and the rules of Π_{bft} *do not exclude that a longest candidate context is such a T; so the literal longest choice need not give a valid block.*

Status: proved (this volume), of a designed-but-unspecified construction.

Proof. (i) If $G = \mathcal{O}_{\mathrm{bc}}$, then G.context_bft = $\mathcal{O}_{\mathrm{bft}}$, which is valid by the Genesis bft-block rule; otherwise G.context_bft is valid by the Valid context rule for G. In both cases bft-last-final(G.context_bft) = $\mathsf{LF}(G)$, so the context is a candidate. With it, $\mathsf{LF}(H) = \mathsf{LF}(G)$, so the Extension rule holds with equality; the final snapshot of H is that of G, which is a prefix of G (the Last Final Snapshot rule for G, or snapshot($\mathcal{O}_{\mathrm{bft}}$) = $\mathcal{O}_{\mathrm{bc}}$ for $G = \mathcal{O}_{\mathrm{bc}}$) and hence of H; and finality-depth(H) = height(G) + 1 − height(snapshot($\mathsf{LF}(G)$)) = finality-depth(G) + 1. The Finality depth rule then holds if H is stalled whenever that depth exceeds L.

(ii) A candidate context T is bft-block-valid, and as context of H it gives $\mathsf{LF}(H)$ = bft-last-final(T) $\succeq_{\mathrm{bft}}$ $\mathsf{LF}(G)$. Let S be its final snapshot and suppose S conflicts with G. If

$S \preceq_{\mathrm{bc}} H$, then either $S = H$, so $G \preceq_{\mathrm{bc}} S$, or $S \preceq_{\mathrm{bc}} G$; both contradict the conflict. Hence $S \not\preceq_{\mathrm{bc}} H$ and the Last Final Snapshot rule fails.

For a witness, take an instance of Π_{origbft} in which every bft-block-valid block is final in its own context, $\mathsf{bft\text{-}last\text{-}final}(C) = C$, which meets Definition 3.29, and a block G whose context $B_1 := G.\mathsf{context_bft} = \mathsf{LF}(G)$ has $\mathsf{snapshot}(B_1) \prec_{\mathrm{bc}} G$. Let a bc-valid branch fork from G's chain at a block D with $\mathsf{snapshot}(B_1) \preceq_{\mathrm{bc}} D \prec_{\mathrm{bc}} G$ and carry $\sigma + 1$ blocks $Y_1, \ldots, Y_{\sigma+1}$ above D, each citing B_1 and stalled where the Finality depth rule requires: the Valid context rule holds since B_1 is valid, the Extension rule holds for Y_1 since $\mathsf{LF}(D) \preceq_{\mathrm{bft}} \mathsf{LF}(G) = B_1$ (Lemma 4.19) and for the others with equality, and the Last Final Snapshot rule holds since $\mathsf{snapshot}(B_1) \preceq_{\mathrm{bc}} D$. Let B_2, a child of B_1 valid in Π_{origbft}, carry the σ-block tail of $Y_{\sigma+1}$. Then $\mathsf{snapshot}(B_2) = Y_1$, which conflicts with G; the block B_2 satisfies the Linearity rule, since $\mathsf{snapshot}(B_1) \preceq_{\mathrm{bc}} D \prec_{\mathrm{bc}} Y_1$, and the Tail Confirmation rule, so it is bft-block-valid, and $\mathsf{bft\text{-}last\text{-}final}(B_2) = B_2 \succeq_{\mathrm{bft}} B_1 = \mathsf{LF}(G)$ makes it a candidate context of H with final snapshot Y_1. In a view whose bft-blocks are those of the chain of B_2 it is the only longest candidate context, so the literal choice selects it. □

The construction is accordingly read as a choice among the candidate contexts that satisfy the consensus rules, which by (i) is non-empty for a block H that is stalled whenever $\mathsf{finality\text{-}depth}(G) + 1 > L$; such a block exists under the producibility constraint of Remark 4.26. The volume does not claim that the literal choice is valid whenever Π_{bft} is not subverted: Assumption 3.23 limits non-honest casting, not the honesty of the nodes whose best chains confirmed a snapshot, and no claim is made about the executions in which a longest candidate context has a final snapshot off the producer's chain.

> **Definition 4.22** (Unchanged fork choice)**.** Crosslink 2 has *unchanged fork choice*: no honest node, and in particular no honest bc-block producer, uses information from Π_{bft} other than the rules of Definition 4.17 to decide which bc-valid chain to follow. An honest node's best chain ch_i^t is a bc-valid chain of highest score in its view, as in Π_{origbc} (Definition 3.11), with validity that of Definition 4.17.

The definition states, under a neutral name, the book's admonition titled "Attention" (section "Π_{bc} honest block production"). The book's reason is that the rules depending on Π_{bft} were constructed to preserve the safety of Π_{bc} relative to Π_{origbc}, and that any further constraint could let an adversary that subverts Π_{bft} steer the best chain in ways the safety argument does not consider. A chain whose context lags far behind the newest BFT blocks can therefore still become the best chain (chapter "Questions About Crosslink"). The book's liveness argument rests on this definition.

Remark 4.23 (Bidirectional dependence)*.* Crosslink 2 makes each sub-protocol depend on the other: the Valid context, Extension, Last Final Snapshot and Finality depth rules make bc-validity depend on Π_{bft}, and the Tail Confirmation rule and honest voting make Π_{bft} depend on Π_{bc}. The ebb-and-flow paper of Neu, Tas and Tse, in its Appendix E, "Bouncing Attack on Casper FFG", warns that such a bidirectional interaction is "intricate to reason about and a gateway for liveness attacks"; in Casper FFG it arises because finalisation constrains the fork choice. The book's mitigation is Definition 4.22: the fork choice of Π_{bc} is not modified (chapter "Security Analysis of Crosslink 2"). Asked whether the fork choice should be changed to follow the last final snapshot,

the book answers "Probably not. I don't know how to repair the safety and liveness arguments" (chapter "Questions About Crosslink").

Status: argued (TFL book).

4.7 Stalled Mode

Definition 4.24 (Stalled Mode)**.** A bc-block H is *in Stalled Mode* iff $\mathsf{finality\text{-}depth}(H) > L$; stalled blocks are those of Definition 4.1. By the Finality depth rule, every bc-valid block in Stalled Mode is stalled. Entry into Stalled Mode is per block and objective: whether H is in Stalled Mode is a function of H, and no network-wide flag exists. Exit is automatic: along a chain, Stalled Mode ends at the first block that cites a context whose final snapshot lies within L blocks below it.

The book states exit in the same terms: Stalled Mode ends "as soon as the finalization point catches up to within L blocks (if it does without an intentional rollback)" (chapter "Notes On Snap-and-Chat").

Remark 4.25 (Proposed Zcash policy). For Zcash the book proposes that stalled blocks be coinbase-only, $\mathsf{is\text{-}stalled\text{-}block} = \mathsf{is\text{-}coinbase\text{-}only\text{-}block}$ (Definition 3.13), and never fixes the predicate (chapter "The Arguments for Bounded Availability and Finality Overrides", section "How to block hazards"). A coinbase-only block spends no funds: its one transaction collects the block subsidy and, there being no other transaction, no fees (*Ironwood Guide*, Definition "Coinbase transaction", §"The transaction format"). Mining pools therefore cannot distribute rewards in stalled blocks. Whether shielded coinbase outputs are also disabled in Stalled Mode is an open problem: the book says only that disabling them "might" be wanted.

Remark 4.26 (Producibility of stalled blocks). The only formal constraint on $\mathsf{is\text{-}stalled\text{-}block}$, imposed by the book's liveness argument (chapter "Security Analysis of Crosslink 2", section "Liveness argument"), is that the set of bc-blocks H with $\mathsf{is\text{-}coinbase\text{-}only\text{-}block}(H)$ and $\mathsf{is\text{-}stalled\text{-}block}(H)$ be producible at all times: a bc-block producer can always produce a block in it. Proposition 4.21(i) supplies the context such a block needs. Under the proposed policy the set is the set of coinbase-only blocks.

Remark 4.27 (Independence of Stalled Mode). Stalled Mode is technically independent of the rest of Crosslink 2: omitting the Finality depth rule removes it and leaves every other rule in place. The book argues that the protocol then keeps its security advantages over snap-and-chat, stays simpler and still lets transactions spend finalised outputs (item (4) of Remark 2.17); the incentive for producers to advance the finalisation point is weaker, and honest producers still advance it (chapter "Security Analysis of Crosslink 2", section "Liveness argument").

Status: argued (TFL book).

Lemma 4.28 (Growth of finality depth)**.** *Let H be a non-genesis bc-valid block. Then*

$$\mathsf{finality\text{-}depth}(H) \le \mathsf{finality\text{-}depth}(H\lceil^{1}_{\mathrm{bc}}) + 1 \quad \textit{and} \quad \mathsf{candidate}(H\lceil^{1}_{\mathrm{bc}}) \preceq_{\mathrm{bc}} \mathsf{candidate}(H).$$

Status: proved (this volume), of a designed-but-unspecified construction.

Proof. Write $G = H\lceil^{1}_{\mathrm{bc}}$, which is bc-valid, and let S_G and S_H be the final snapshots of G and H. By Lemma 4.19, $\mathsf{LF}(G) \preceq_{\mathrm{bft}} \mathsf{LF}(H)$, and $\mathsf{LF}(H)$ is bft-block-valid by the Valid context rule and Definition 3.29; so (1) gives $S_G \preceq_{\mathrm{bc}} S_H$ and hence $\mathsf{height}(S_G) \le \mathsf{height}(S_H)$. Since $\mathsf{height}(H) = \mathsf{height}(G) + 1$,

$$\begin{aligned}\mathsf{finality\text{-}depth}(H) &= \mathsf{height}(G) + 1 - \mathsf{height}(S_H)\\ &\le \mathsf{height}(G) + 1 - \mathsf{height}(S_G) = \mathsf{finality\text{-}depth}(G) + 1.\end{aligned}$$

For the second claim, $\mathsf{candidate}(G) \preceq_{\mathrm{bc}} S_G \preceq_{\mathrm{bc}} S_H$, and $\mathsf{candidate}(G) \preceq_{\mathrm{bc}} G\lceil^{\sigma} = H\lceil^{\sigma+1} \preceq_{\mathrm{bc}} H\lceil^{\sigma}$. So $\mathsf{candidate}(G)$ is a common prefix of S_H and $H\lceil^{\sigma}$, and so is $\mathsf{candidate}(H)$, the longest common prefix. Both are prefixes of $H\lceil^{\sigma}$, so they agree (Lemma 3.10), and the longer is $\mathsf{candidate}(H)$; hence $\mathsf{candidate}(G) \preceq_{\mathrm{bc}} \mathsf{candidate}(H)$. □

Entry into Stalled Mode is therefore gradual. Along a bc-valid chain the finality depth starts at $\mathsf{finality\text{-}depth}(\mathcal{O}_{\mathrm{bc}}) = 0$ and grows by at most one per block, so the first block in Stalled Mode, the first block that the Finality depth rule forces to be stalled, has finality depth exactly $L + 1$. A script checks the lemma and this consequence on random chains.

Proposition 4.29 (Branch dependence of exit)**.** *Let H be a bc-valid block.*

(i) If H is not in Stalled Mode, its final snapshot lies on its own chain at most L blocks below it: $\mathsf{snapshot}(\mathsf{LF}(H)) \preceq_{\mathrm{bc}} H$ *and* $\mathsf{height}(H) - \mathsf{height}(\mathsf{snapshot}(\mathsf{LF}(H))) \le L$*. Stalled Mode can therefore end on a branch only at a block whose final snapshot lies on that branch within L of the block.*

(ii) Let X and Y be conflicting bc-chains with fork point $D = \mathsf{lca}(X, Y)$, and suppose that Π_{bft} has Final Agreement (Definition 3.31) and that some bft-block-valid block C^* in honest view has final snapshot S^* with $D \prec_{\mathrm{bc}} S^* \preceq_{\mathrm{bc}} X$. If $H \preceq_{\mathrm{bc}} Y$, $\mathsf{height}(H) > \mathsf{height}(D) + L$ *and an honest node has validated H, then H is stalled.*

Status: proved (this volume), of a designed-but-unspecified construction.

Proof. (i) The Last Final Snapshot rule gives the prefix relation, and a block not in Stalled Mode has finality depth at most L.

(ii) To validate H the honest node obtained H.context_bft and checked it (the Valid context rule), so H.context_bft is a bft-block-valid block in honest view (Definition 3.26). Final Agreement makes $A := \mathsf{bft\text{-}last\text{-}final}(C^*)$ and $B := \mathsf{LF}(H)$ agree. First, S^* is not a prefix of Y: otherwise it would be a common prefix of X and Y, hence a prefix of D, against $D \prec_{\mathrm{bc}} S^*$. If $A \preceq_{\mathrm{bft}} B$, then (1), with B valid, gives $S^* \preceq_{\mathrm{bc}} \mathsf{snapshot}(B)$; the Last Final Snapshot rule gives $\mathsf{snapshot}(B) \preceq_{\mathrm{bc}} H \preceq_{\mathrm{bc}} Y$, so $S^* \preceq_{\mathrm{bc}} Y$, which is excluded; this case does not occur. Hence $B \preceq_{\mathrm{bft}} A$, and (1), with A valid as a prefix of the valid C^*, gives $\mathsf{snapshot}(B) \preceq_{\mathrm{bc}} S^* \preceq_{\mathrm{bc}} X$. With $\mathsf{snapshot}(B) \preceq_{\mathrm{bc}} H \preceq_{\mathrm{bc}} Y$, the final snapshot of H is a common prefix of X and Y, so $\mathsf{height}(\mathsf{snapshot}(B)) \le \mathsf{height}(D)$ and $\mathsf{finality\text{-}depth}(H) \ge \mathsf{height}(H) - \mathsf{height}(D) > L$. The Finality depth rule makes H stalled. □

Under the hypotheses of (ii), Stalled Mode persists on Y above $\mathsf{height}(D) + L$, whatever the score of Y. Unchanged fork choice (Definition 4.22) keeps an honest node on Y while Y has the

highest score in its view, and every block that node produces on Y above that height is stalled. The proposition makes no claim about which assumptions exclude its hypothesis, a final snapshot beyond the fork point on the branch that honest nodes do not follow.

Proposition 4.30 (Forced entry into Stalled Mode)**.** *(i) Let $G \preceq_{\mathrm{bc}} H$ be bc-valid blocks with $G \neq \mathcal{O}_{\mathrm{bc}}$, such that every block H' with $G \prec_{\mathrm{bc}} H' \preceq_{\mathrm{bc}} H$ has the final snapshot of G, as when each of those blocks repeats its parent's* context_bft. *Then* $\mathsf{finality\text{-}depth}(G) \geq 1$ *and*

$$\mathsf{finality\text{-}depth}(H) = \mathsf{finality\text{-}depth}(G) + \mathsf{height}(H) - \mathsf{height}(G),$$

so H is stalled whenever $\mathsf{height}(H) - \mathsf{height}(G) \geq L$.

(ii) A bc-valid block H' with $\mathsf{finality\text{-}depth}(H') \leq L$, *as when it cites a context whose final snapshot lies on its chain at most L blocks below it, is not in Stalled Mode, whatever the finality depth of its parent, and no block of a bc-valid chain within $L - \mathsf{finality\text{-}depth}(H')$ blocks above H' is in Stalled Mode.*

(iii) If more than one third of the voting units of a BFT epoch are cast for no proposal of that BFT epoch, then no proposal of the BFT epoch is notarised, and no bft-block of the BFT epoch is bft-block-valid.

Status: proved (this volume), of a designed-but-unspecified construction.

Proof. (i) Let S be the final snapshot of G; it is a prefix of G by the Last Final Snapshot rule. It is not G itself: $\mathsf{snapshot}(\mathcal{O}_{\mathrm{bft}}) = \mathcal{O}_{\mathrm{bc}} \neq G$, so $S = G$ would make $\mathsf{LF}(G)$ carry the header of a child of G, and G would commit to its own hash through G.context_bft, its ancestor $\mathsf{LF}(G)$ and that child, a cycle excluded as in Remark 4.10. Hence $\mathsf{height}(S) < \mathsf{height}(G)$ and $\mathsf{finality\text{-}depth}(G) \geq 1$. The block H has final snapshot S, so $\mathsf{finality\text{-}depth}(H) = \mathsf{height}(H) - \mathsf{height}(S) = \mathsf{finality\text{-}depth}(G) + \mathsf{height}(H) - \mathsf{height}(G)$, which exceeds L when $\mathsf{height}(H) - \mathsf{height}(G) \geq L$; the Finality depth rule makes H stalled.

(ii) The first claim is Definition 4.24. For a block H'' that lies k blocks above H' on a bc-valid chain, k applications of Lemma 4.28 give $\mathsf{finality\text{-}depth}(H'') \leq \mathsf{finality\text{-}depth}(H') + k$, which is at most L when $k \leq L - \mathsf{finality\text{-}depth}(H')$.

(iii) Let n be the number of voting units of the BFT epoch and $q = \lceil 2n/3 \rceil$ the threshold of Definition 3.25. For any proposal P of the BFT epoch, the units cast for P are among those cast for some proposal of the BFT epoch, fewer than $n - n/3 = 2n/3 \leq q$; so fewer than q units are cast for P. A notarisation proof for P is obtainable only from ballots that meet the notarisation rule (Definition 3.21), which requires q units cast for P, so none exists, and a bft-block (P, proof_P) needs a valid notarisation proof (Definition 3.26). □

By (i) and (ii), forcing Stalled Mode against producers that cite newer contexts requires producing every block of the best chain above the last block H' that cites a newer context, $L - \mathsf{finality\text{-}depth}(H') + 1$ consecutive blocks, at most L since $\mathsf{finality\text{-}depth}(H') \geq 1$ by (i); a party with more than half of the hash rate can do so, since it can reject every other producer's blocks (*Consensus Guide*, Remark "Majority is a different problem", in §"Reward-normalised exposure"), and a smaller party only by chance, with a probability that no result of this volume

bounds; by (iii), holders of more than one third of a BFT epoch's units who withhold their ballots, and whose units no one else casts, prevent every block of that BFT epoch, so finalisation cannot advance through a block of it. The book's sentence that Stalled Mode "can only be entered by consensus of a larger validator set, or if there is an availability failure of the finalization protocol" contrasts the design with an earlier proposal to trigger Stalled Mode by a key (chapter "The Arguments for Bounded Availability and Finality Overrides", section "Complementarity"); by (i), block producers can force entry while Π_{bft} is live, so the sentence is not a property of the construction.

Remark 4.31 (Escape from Stalled Mode). A bc-block producer is constrained to produce stalled blocks exactly while no candidate context in its view has a final snapshot on the new block's chain at most L blocks below the new block; the book states this only "roughly speaking", as holding while its view of the finalisation point is not advancing. The book cautions that an adversary that has subverted Π_{bft} without stopping the finalisation point from advancing can always avoid the constraint (section "Stalled Mode"): a block is in Stalled Mode only when its final snapshot lies more than L blocks below it, and such an adversary can keep citing contexts whose final snapshots lie within L of its blocks.

Status: argued (TFL book).

5 The finalised and bounded-available chains

This section defines the state that a Crosslink 2 node keeps besides its views of Π_{bc} and Π_{bft}: the locally finalised chain fin with its no-rollback update and hazard record, the locally bounded-available chain ba_μ, the syncing gate that withholds both from clients, and the computation that a node performs on each change of its best chain, with the lag of fin behind the best tip. The lemmas proved here are those that the safety proofs use. Every definition and construction of the section is designed but unspecified in the classes of Definition 1.1; it comes from the sections "Parameters", "Locally finalized chain", "Locally bounded-available chain" and "Syncing and checkpoints" of the TFL book's chapter "The Crosslink 2 Construction", and Construction 5.13 orders the definitions of §4 and of this section as a node applies them. Open problems are marked where they arise. Throughout, i is a node and ch_i^t its best chain at time t (Definition 3.11).

5.1 The locally finalised chain

Definition 5.1 (Locally finalised chain)**.** Each node i keeps a bc-chain fin_i^t, its *locally finalised chain* at time t, as follows.

(i) The chain starts at $\mathsf{fin}_i^0 = \mathcal{O}_{\text{bc}}$.

(ii) *Update rule* update-fin: at each time t at which the best chain of node i changes, with s the time of the previous change, or 0 if there is none,

$$\mathsf{fin}_i^t := \begin{cases} \mathsf{candidate}(\mathsf{ch}_i^t), & \text{if } \mathsf{fin}_i^s \preceq_{\text{bc}} \mathsf{candidate}(\mathsf{ch}_i^t), \\ \mathsf{fin}_i^s, & \text{otherwise.} \end{cases}$$

If moreover $\mathsf{candidate}(\mathsf{ch}_i^t)$ conflicts with fin_i^s, node i records a *finalisation safety hazard* that holds ch_i^t and the history of its updates of fin_i including and since the last one whose value was a prefix of $\mathsf{candidate}(\mathsf{ch}_i^t)$.

(iii) Between changes of its best chain, fin_i is constant.

An update never replaces fin_i by a chain that does not extend it. A candidate that is a strict prefix of fin_i^s, as after a short reorganisation, leaves fin_i unchanged and is no hazard; only a conflicting candidate is. The book states that a hazard can occur only if global safety assumptions are violated.

Lemma 5.2 (Local finalization linearity)**.** *If node i follows the update rule of Definition 5.1 up to time u, then $\mathsf{fin}_i^t \preceq_{\mathrm{bc}} \mathsf{fin}_i^u$ for every $t \le u$: the series fin_i is bc-linear up to u (Definition 3.7).*
Status: proved (this volume), of a designed-but-unspecified construction.

Proof. The book names the property and asserts that the update guarantees it by construction (section "Locally finalized chain"); the proof is the following. Between changes of ch_i the value of fin_i is constant, and at each change it is replaced by an extension of itself: by $\mathsf{candidate}(\mathsf{ch}_i^t)$ under the case condition $\mathsf{fin}_i^s \preceq_{\mathrm{bc}} \mathsf{candidate}(\mathsf{ch}_i^t)$, or by itself. Induction over the changes in $(t, u]$, with transitivity of $\preceq_{\mathrm{bc}}$, gives $\mathsf{fin}_i^t \preceq_{\mathrm{bc}} \mathsf{fin}_i^u$. □

Lemma 5.3 (Local fin-depth)**.** *In every execution of Crosslink 2, for every node i honest at time t there is a time $r \le t$ with $\mathsf{fin}_i^t \preceq_{\mathrm{bc}} \mathsf{ch}_i^r\lceil^\sigma$.*
Status: proved (TFL book), of a designed-but-unspecified construction.

Proof. The proof is the book's (section "Locally finalized chain"). For every bc-valid block H, $\mathsf{candidate}(H) \preceq_{\mathrm{bc}} H\lceil^\sigma$, since a last common ancestor is a prefix of each argument (Definition 3.8). Node i, honest at t, has followed the update rule up to t (Definition 3.3), and its best chains are bc-valid (Definition 3.11). If fin_i has not changed up to t, then $\mathsf{fin}_i^t = \mathcal{O}_{\mathrm{bc}}$; take $r = 0$, where $\mathsf{ch}_i^0 = \mathcal{O}_{\mathrm{bc}}$ and $\mathcal{O}_{\mathrm{bc}}\lceil^\sigma = \mathcal{O}_{\mathrm{bc}}$ (Definitions 3.11 and 3.5). Otherwise let $r \le t$ be the time of its last change up to t. A change occurs only in the first case of the update rule, so $\mathsf{fin}_i^t = \mathsf{fin}_i^r = \mathsf{candidate}(\mathsf{ch}_i^r) \preceq_{\mathrm{bc}} \mathsf{ch}_i^r\lceil^\sigma$. □

Remark 5.4 (Purpose of the no-rollback rule). When the best chain of a node reorganises to another fork, even by fewer than σ blocks, $\mathsf{candidate}(\mathsf{ch}_i^t)$ can move back. If Π_{bft} has Final Agreement (Definition 3.31), the new candidate should then be an ancestor of the old one and, as the best chain grows, move forward again along the same path. Clients of the node must not see that temporary regression, so fin_i is node state that never rolls back (the book's admonition "Why does fin_i need to be maintained using local state?").

Status: argued (TFL book).

Remark 5.5 (Candidate or final snapshot). The update uses $\mathsf{candidate}(H)$ rather than the final snapshot $\mathsf{snapshot}(\mathsf{LF}(H))$ so that the finalised prefix is σ-confirmed: the Last Final Snapshot rule places the final snapshot on H's chain without guaranteeing it a depth of σ or more, while the book's safety argument from Prefix Agreement needs $\mathsf{candidate}(H) \preceq_{\mathrm{bc}} H\lceil^\sigma$, the property behind Lemma 5.3. The book records the alternative of strengthening the Last Final Snapshot rule to

$\mathsf{snapshot}(\mathsf{LF}(H)) \preceq_{\mathrm{bc}} H\lceil^{\sigma}$ and leaves the choice open: "Choose between these options based on what works well for the security proofs and finalization latency" (admonition "Why do we need to use $\mathsf{candidate}(H)$ rather than $\mathsf{snapshot}(\mathsf{LF}(H))$?"). It expects the two to differ rarely in practice.

Status: open problem.

5.2 The locally bounded-available chain

Definition 5.6 (Bounded-available depth)**.** Each node i chooses a *bounded-available depth* $\mu \in \mathbb{N}$ with $0 < \mu \leq \sigma$, possibly different from the choices of other nodes; the default is $\mu = \sigma$.

The book restricts μ to at most σ because a larger value has no use. A node that chooses $\mu < \sigma$ does so at its own risk: the smaller depth may increase the risk of rollback against its bounded-available chain, and it does not affect fin_i (section "Parameters").

Definition 5.7 (Locally bounded-available chain)**.** The *locally bounded-available chain* of node i at time t is

$$(\mathsf{ba}_{\mu})_i^t := \begin{cases} \mathsf{ch}_i^t\lceil^{\mu}, & \text{if } \mathsf{fin}_i^t \preceq_{\mathrm{bc}} \mathsf{ch}_i^t\lceil^{\mu}, \\ \mathsf{fin}_i^t, & \text{otherwise.} \end{cases}$$

Proposition 5.8 (Ledger prefix property)**.** *For every node i honest at time t and every bounded-available depth μ, $\mathsf{fin}_i^t \preceq_{\mathrm{bc}} (\mathsf{ba}_{\mu})_i^t$.*

Status: proved (TFL book), of a designed-but-unspecified construction.

Proof. In the first case of Definition 5.7 the relation is the case condition; in the second the two chains are equal. The book states the property as a theorem and proves it by construction of ba_{μ} (section "Locally bounded-available chain"). □

Lemma 5.9 (Local ba-depth)**.** *In every execution of Crosslink 2, for every bounded-available depth $\mu \leq \sigma$ and every node i honest at time t, there is a time $r \leq t$ with $(\mathsf{ba}_{\mu})_i^t \preceq_{\mathrm{bc}} \mathsf{ch}_i^r\lceil^{\mu}$.*

Status: proved (TFL book), of a designed-but-unspecified construction.

Proof. The proof is the book's (section "Locally bounded-available chain"). In the first case of Definition 5.7, $(\mathsf{ba}_{\mu})_i^t = \mathsf{ch}_i^t\lceil^{\mu}$; take $r = t$. Otherwise $(\mathsf{ba}_{\mu})_i^t = \mathsf{fin}_i^t$, and Lemma 5.3 gives $r \leq t$ with $\mathsf{fin}_i^t \preceq_{\mathrm{bc}} \mathsf{ch}_i^r\lceil^{\sigma}$. Since $\mu \leq \sigma$, $\mathsf{ch}_i^r\lceil^{\sigma} = (\mathsf{ch}_i^r\lceil^{\mu})\lceil^{\sigma-\mu} \preceq_{\mathrm{bc}} \mathsf{ch}_i^r\lceil^{\mu}$ (Definition 3.5). □

Remark 5.10 (The fallback arm). The second case of Definition 5.7 is needed even when every safety assumption holds. The book argues it for proof of work (admonition "Why is the 'otherwise' case in the definition of $(\mathsf{ba}_{\mu})_i^t$ necessary?"). After the best chain switches to a fork on which the difficulty is higher than on the chain of the previous snapshot, the new chain can reach a higher score than the old one with fewer than σ blocks above the fork point, so $\mathsf{ch}_i^t\lceil^{\mu}$ need not descend from fin_i^t, more often so when $\mu = \sigma$. Using fin_i^t in that case is safe, the book argues, because although fin_i^t is not a prefix of $\mathsf{ch}_i^t\lceil^{\mu}$, so that its tip has at most μ confirmations on ch_i^t (Remark 3.6; the book's "μ confirmations" means depth μ), it carries at least the required work above it: score is accumulated work (Remark 3.12; *Consensus Guide*, §"Work, and the best chain"),

and Zcash's difficulty adjustment damps and clamps the effect of timestamps, so an adversary gains little by manipulating it (*Consensus Guide*, §"Difficulty adjustment"). The argument rests on that damping, not on the length of the averaging window, which ZIP 218, "Averaging window", changes under NU7; no window length is used here. The *Consensus Guide*'s Remark "What per-block retargeting changes" records that window, damping and clamp do not prove that two forked branches keep close targets, so the safety of the second case remains an argument. The book's wording, a switch of fin_i "to a different fork", is loose: by Lemma 5.2, fin_i never moves to a chain that conflicts with it; the best chain switches. A script exhibits, for every $2 \le \sigma \le 10$ with $\mu = \sigma$, a switch of the best chain to a fork of $\sigma - 1$ blocks above fin_i^t, with Prefix Consistency at depth σ (Definition 3.15) holding, after which $\mathsf{ch}_i^t\lceil^{\mu}$ is a strict prefix of fin_i^t and $(\mathsf{ba}_{\mu})_i^t = \mathsf{fin}_i^t$ through the second case only.

Status: argued (TFL book).

Remark 5.11 (Ledgers as chains). The ledgers of Crosslink 2 are the bc-chains fin_i^t and $(\mathsf{ba}_{\mu})_i^t$. Since no snapshot is sanitised, they need not be expressed as transaction logs (section "Parameters"). Where the ledger names of the paper and of the TFL book are used, $\mathsf{LOG}_{\mathrm{fin},i}^t$ denotes the transaction sequence of fin_i^t, the transactions of its blocks from $\mathcal{O}_{\mathrm{bc}}$ to its tip in chain order and, within a block, in block order, and $\mathsf{LOG}_{\mathrm{ba},i}^t$ that of $(\mathsf{ba}_{\mu})_i^t$ for node i's choice of μ. The book's security analysis below its marker "Not updated for Crosslink 2" and its chapter "Potential Changes to Crosslink" use two other ledger forms, from Crosslink 1, whose finalised ledger is read from the snapshots of $\mathsf{LF}(\mathsf{ch}_i^t\lceil^{\sigma})$; they are not the ledgers of Crosslink 2 and are not used in this volume.

5.3 Syncing and checkpoints

Remark 5.12 (Syncing gate). The book recommends, as a rule for nodes and not as a consensus rule, that a node embed a checkpoint bft-block B_{ckpt}, fixed in each release, and that node i expose fin_i^t and $(\mathsf{ba}_{\mu})_i^t$ to its clients only once it is *synced*, that is, when

(i) $B_{\mathrm{ckpt}} \preceq_{\mathrm{bft}} \mathsf{LF}(\mathsf{ch}_i^t)$,

(ii) $\mathsf{snapshot}(B_{\mathrm{ckpt}}) \preceq_{\mathrm{bc}} \mathsf{fin}_i^t$, and

(iii) the timestamp of the tip of fin_i^t is within some threshold of the current time

(section "Syncing and checkpoints"). The gate carries the book's withholding clauses for the two chains, which Definitions 5.1 and 5.7 leave out: neither chain is exposed to clients before the node is synced (sections "Locally finalized chain" and "Locally bounded-available chain"). By condition (i), an observation of this volume, a node is not synced while its best tip cites a context that does not descend from B_{ckpt}, such as one on a bft-chain forged with the keys of BFT epochs before the checkpoint; the book does not state this. The checkpoint and the threshold are an open problem: neither the book nor Shielded Labs' *Zcash Crosslink Design Overview* (draft 10, 14 September 2026) fixes them.

5.4 Computation of the finalised prefix

Construction 5.13 (Node update)**.** On each change of its best chain, at time t with new tip $H = \mathsf{ch}_i^t$, node i performs the following steps.

(1) It resolves H.context_bft to a bft-block-valid block, with its ancestry, the σ-block tail that each ancestor carries and, recursively, the contexts of those bc-blocks, in the validation order of Construction 4.14; validating H under the Valid context rule has done so.

(2) It computes $\mathsf{LF}(H) = \mathsf{bft\text{-}last\text{-}final}(H.\mathsf{context_bft})$.

(3) It computes the final snapshot $\mathsf{snapshot}(\mathsf{LF}(H))$ of H.

(4) It computes $\mathsf{candidate}(H)$, the lower of the final snapshot and $H\lceil^{\sigma}$ (Lemma 4.18, since H is bc-valid).

(5) It applies the update rule of Definition 5.1.

(6) It computes $(\mathsf{ba}_{\mu})_i^t$ (Definition 5.7).

(7) It exposes fin_i^t and $(\mathsf{ba}_{\mu})_i^t$ only through the syncing gate of Remark 5.12.

A decision of Π_{bft} by itself changes nothing at a node: fin_i changes only at changes of ch_i, and its new value depends on Π_{bft} only through $\mathsf{LF}(\mathsf{ch}_i^t)$. By Lemma 4.19, $\mathsf{LF}(\mathsf{ch}_i^t)$ extends the last final context of every block of ch_i^t, so a decided bft-block moves fin_i only when a bc-block whose last final context extends it is, or is an ancestor of, the best tip.

Proposition 5.14 (Finalisation lag)**.** *Let H be a bc-valid block.*

(i) The candidate satisfies $\mathsf{candidate}(H) \preceq_{\mathrm{bc}} H\lceil^{\sigma}$.

(ii) If $\mathsf{LF}(H) \neq \mathcal{O}_{\mathrm{bft}}$ *and the headers that* $\mathsf{LF}(H)$ *carries are headers of ancestors of H, then* $\mathsf{candidate}(H) = \mathsf{snapshot}(\mathsf{LF}(H))$ *and*

$$\mathsf{height}(H) - \mathsf{height}(\mathsf{candidate}(H)) = \mathsf{finality\text{-}depth}(H) \geq \sigma + 1.$$

Hence at every update of a node with best tip H that sets fin *to* $\mathsf{candidate}(H)$ *under these hypotheses,* fin *trails the tip by at least $\sigma + 1$ blocks, and the distance from the tip to* fin *equals the finality depth.*

(iii) The bound is attained: if $T = H\lceil^{1}_{\mathrm{bc}}$ has at least $\sigma + 1$ blocks and $\mathsf{LF}(H)$ *carries the σ-block tail of T, then* $\mathsf{height}(H) - \mathsf{height}(\mathsf{candidate}(H)) = \sigma + 1$. *This is the case when an honest proposer whose best tip is T carries the σ-block tail of T (Construction 4.13) and the bft-block formed from its proposal is the last block final in the context that H cites (Definition 3.30).*

Status: proved (this volume), of a designed-but-unspecified construction.

Proof. (i) A last common ancestor is a prefix of each argument (Definition 3.8).

(ii) The block $\mathsf{LF}(H)$ is bft-block-valid (the Valid context rule and Definition 3.29) and is not $\mathcal{O}_{\mathrm{bft}}$, so by the Tail Confirmation rule its field headers_bc is the σ-block tail of a bc-valid chain T' of at least $\sigma + 1$ blocks, T' being the block of the newest carried header. By the paragraph after Definition 4.3, $\mathsf{snapshot}(\mathsf{LF}(H)) = T'\lceil^{\sigma}$, of height $\mathsf{height}(T') - \sigma$. By hypothesis $T' \preceq_{\mathrm{bc}} H$, and $T' \neq H$: block H commits by hash to H.context_bft, which has $\mathsf{LF}(H)$ as a prefix and commits to it through parent links, and $\mathsf{LF}(H)$ commits to the header of T', so $T' = H$ would be a cycle of commitments, excluded as in Remark 4.10. Hence $\mathsf{height}(T') \leq \mathsf{height}(H) - 1$ and

$$\mathsf{height}(\mathsf{snapshot}(\mathsf{LF}(H))) \leq \mathsf{height}(H) - \sigma - 1.$$

Both $\mathsf{snapshot}(\mathsf{LF}(H)) \preceq_{\mathrm{bc}} T' \preceq_{\mathrm{bc}} H$ and $H\lceil^{\sigma} \preceq_{\mathrm{bc}} H$ are prefixes of H, and $\mathsf{height}(H\lceil^{\sigma}) = \mathsf{height}(H) - \sigma$ because $\mathsf{height}(H) \geq \mathsf{height}(T') + 1 \geq \sigma + 1$; so the final snapshot is a strict prefix of $H\lceil^{\sigma}$ (Lemma 3.10), and Lemma 4.18 gives $\mathsf{candidate}(H) = \mathsf{snapshot}(\mathsf{LF}(H))$. Then $\mathsf{height}(H) - \mathsf{height}(\mathsf{candidate}(H)) = \mathsf{finality\text{-}depth}(H) \geq \sigma + 1$. At an update that sets $\mathsf{fin}_t^t := \mathsf{candidate}(\mathsf{ch}_t^t)$ with $\mathsf{ch}_t^t = H$, the distance from the tip to fin_t^t is this number.

(iii) Here the newest carried header is that of T, so by the computation of (ii), with $T' = T$, $\mathsf{height}(\mathsf{snapshot}(\mathsf{LF}(H))) = \mathsf{height}(T) - \sigma = \mathsf{height}(H) - \sigma - 1$ and $\mathsf{candidate}(H) = \mathsf{snapshot}(\mathsf{LF}(H))$; the distance is $\sigma + 1$. In the case described, the proposal carries the σ-block tail of T and its block is $\mathsf{LF}(H)$. □

The hypothesis $\mathsf{LF}(H) \neq \mathcal{O}_{\mathrm{bft}}$ cannot be dropped: if $\mathsf{LF}(H) = \mathcal{O}_{\mathrm{bft}}$, then $\mathsf{candidate}(H) = \mathsf{lca}(\mathcal{O}_{\mathrm{bc}}, H\lceil^{\sigma}) = \mathcal{O}_{\mathrm{bc}}$, at distance $\mathsf{height}(H)$ from H, which is below $\sigma + 1$ whenever $\mathsf{height}(H) \leq \sigma$. A script checks (i) for final snapshots on and off H's chain, (ii) and (iii) for every $\sigma \leq 20$, and the excluded case. The latency of Π_{bft} and contexts older than the newest decision add to the lag. The proposition counts blocks, not confirmations (Remark 3.6). It corrects the book's remark that the finality depth and the distance from a node's tip to fin differ only just after a reorganisation (admonition "Explain the definition of finality-depth."): by (ii) they can differ only at an update that keeps fin, or when the headers that $\mathsf{LF}(H)$ carries are not all ancestors of H, and the second case needs no reorganisation, as in the construction of Proposition 8.5.

Remark 5.15 (Latency estimate of the book). The book estimates that the finalised view lies "about $\mu + 1 + \sigma$ blocks back", against σ blocks for snap-and-chat at the same confirmation depth, a relative increase of "at most slightly more than a factor of 2" for $\mu = \sigma$ (chapter "Security Analysis of Crosslink 2", section "Finalization latency"). Both figures concern a ledger form of the book's Crosslink 1 analysis, which reads the BFT context from the block at depth μ of the best chain, and are stale for Crosslink 2. In Crosslink 2, at an update that sets fin to $\mathsf{candidate}(H)$, fin lies at least $\min\{\sigma, \mathsf{height}(H)\}$ blocks below H (Proposition 5.14(i)). Under the hypotheses of (ii) it lies at least $\sigma + 1$ blocks below, attained when a decision is final in context at once and cited by the next block, so the cost over snap-and-chat's σ is about one block plus the latency of Π_{bft}. No source gives a validated latency bound for Crosslink 2.

Figure 2 draws the two chains at an update in Stalled Mode. Every block $B \preceq_{\mathrm{bc}} H$ with $\mathsf{height}(B) > \mathsf{height}(\mathsf{snapshot}(\mathsf{LF}(H))) + L$ is stalled: B is bc-valid as a prefix of H, and by Lemma 4.19 and (1) its final snapshot is a prefix of that of H, so $\mathsf{finality\text{-}depth}(B) > L$ and the Finality depth rule applies.

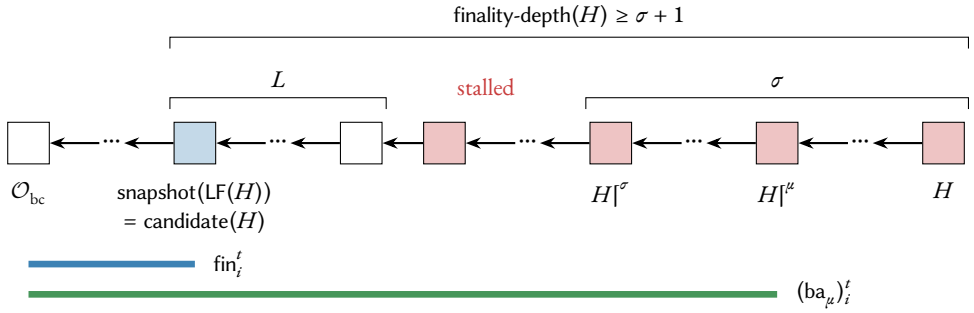


Figure 2. The two chains of node i at an update with best tip $H = \mathsf{ch}_i^t$ in Stalled Mode, when the headers that $\mathsf{LF}(H)$ carries are ancestors of H and the update sets fin_i^t to $\mathsf{candidate}(H)$. Arrows point from child to parent. The final snapshot $\mathsf{snapshot}(\mathsf{LF}(H))$ is the lower of itself and $H\lceil^{\sigma}$, hence $\mathsf{candidate}(H)$; fin_i^t (blue) ends there, $\mathsf{finality\text{-}depth}(H) \geq \sigma + 1$ blocks below H. The chain $(\mathsf{ba}_{\mu})_i^t$ (green) is $H\lceil^{\mu}$, and fin_i^t is a prefix of it. The blocks more than L above the final snapshot, up to H, are stalled (red). Positions are symbolic and represent no value of σ, μ or L.

6 Safety

This section states the safety goal of Crosslink 2 and proves it from either of two premises: Prefix Agreement of Π_{bc} at depth σ, which needs no property of Π_{bft}, and Final Agreement of Π_{bft}, which needs the Linearity rule. It records which rules the two proofs use, the attack that defeats both premises, and what is known of the safety of the bounded-available chain. Every object of the section belongs to the construction of Definition 3.1, which is designed but unspecified in the classes of Definition 1.1; the theorems are implications proved of that construction, and no rule they use is specified.

6.1 Assured Finality

Definition 6.1 (Assured Finality)**.** Assured Finality holds in an execution of Crosslink 2 iff for all times t, u and all nodes i honest at t and j honest at u, $i = j$ permitted, the chains fin_i^t and fin_j^u agree; that is, iff Agreement on the view $(i, t) \mapsto \mathsf{fin}_i^t$ holds (Definition 3.9).

Assured Finality is the main safety goal of the TFL book for Crosslink 2, which intends it to hold under assumptions on either sub-protocol (chapter "The Crosslink 2 Construction", section "Locally finalized chain"). For a single node the goal concerns one time series of bc-chains.

Proposition 6.2 (The single-node case)**.** *Let S be a time series of bc-chains. If S is bc-linear (Definition 3.7), then S^t and S^u agree for all times t and u. The converse fails: some series whose values agree pairwise is not bc-linear. Hence Lemma 5.2 implies the case $i = j$ of Assured Finality, and that case does not imply bc-linearity of a general series.*

Status: proved (this volume).

Proof. If $t \leq u$ then $S^t \preceq_{\mathrm{bc}} S^u$, and if $u \leq t$ then $S^u \preceq_{\mathrm{bc}} S^t$; either way the two agree. For the converse let A be a bc-chain other than $\mathcal{O}_{\mathrm{bc}}$, and let $S^0 = A$ and $S^t = A\lceil^1$ for $t > 0$. Any two

values agree, since $A\lceil^1 \preceq_{\mathrm{bc}} A$, and the series is not bc-linear, since $A \not\preceq_{\mathrm{bc}} A\lceil^1$. If node i is honest at t and at u, it has followed the update rule up to $\max\{t, u\}$ (Definition 3.3), so Lemma 5.2 makes fin_i bc-linear up to that time, and the first part makes fin_i^t and fin_i^u agree. □

The TFL book states that the restriction of Assured Finality to the case $i = j$ "is equivalent to" Local finalization linearity (chapter "The Crosslink 2 Construction", section "Locally finalized chain"). By Proposition 6.2 only one direction holds for a general series, as a script check of the two-element series confirms. For fin both hold, because update-fin makes each series fin_i bc-linear by construction; the book's inference from Assured Finality to Local finalization linearity at every honest node uses the false direction, and its conclusion holds for fin by Lemma 5.2 alone.

6.2 The Prefix Agreement route

Theorem 6.3 (Prefix Agreement implies Assured Finality)**.** *In every execution of Crosslink 2 in which* Π_{bc} *has Prefix Agreement at depth* σ *(Definition 3.16), Assured Finality holds.*

Status: proved (TFL book), of a designed-but-unspecified construction.

The TFL book states and proves the theorem in chapter "Security Analysis of Crosslink 2", section "Safety argument"; its proof follows.

Proof. Let node i be honest at t and node j honest at u. By Lemma 5.3 there are times $r \le t$ and $s \le u$ with

$$\mathsf{fin}_i^t \preceq_{\mathrm{bc}} \mathsf{ch}_i^r\lceil^\sigma \quad \text{and} \quad \mathsf{fin}_j^u \preceq_{\mathrm{bc}} \mathsf{ch}_j^s\lceil^\sigma,$$

and node i is honest at r and node j at s (Definition 3.3). Prefix Agreement at σ, applied at (i, r) and (j, s), makes $\mathsf{ch}_i^r\lceil^\sigma$ and $\mathsf{ch}_j^s\lceil^\sigma$ agree; by symmetry let the first be a prefix of the second. Then $\mathsf{fin}_i^t \preceq_{\mathrm{bc}} \mathsf{ch}_j^s\lceil^\sigma$ by transitivity, and $\mathsf{fin}_j^u \preceq_{\mathrm{bc}} \mathsf{ch}_j^s\lceil^\sigma$, so fin_i^t and fin_j^u agree by Lemma 3.10. □

The proof uses Prefix Agreement of Π_{bc} at σ and update-fin, through Lemma 5.3, and nothing about Π_{bft}. For Zcash the premise is an assumption, not a result (Remark 3.18); the nearest statement in the lower volumes, the persistence clause of Assumption "Persistence and liveness" (*FlyClient Guide*, §"The variable-difficulty backbone model"), is a prefix property of the backbone model and is itself adopted there as an assumption.

Corollary 6.4 (Safety under subversion of the BFT protocol)**.** *In every execution in which* Π_{bc} *has Prefix Agreement at* σ*, Assured Finality holds whatever the holders of voting units do, including executions in which every voting unit is cast non-honestly (Definition 3.22) and signatures are forged.*

Status: proved (this volume), of a designed-but-unspecified construction.

Proof. The proof of Theorem 6.3 uses Prefix Agreement of Π_{bc}, Lemma 5.3 and Lemma 3.10. The proof of Lemma 5.3 uses update-fin, $\mathsf{ch}_i^0 = \mathcal{O}_{\mathrm{bc}}$, bc-validity of honest best chains (Definition 3.11), whose Valid context rule makes $\mathsf{LF}(H)$ a bft-block-valid block, and the relation $\mathsf{candidate}(H) \preceq_{\mathrm{bc}} H\lceil^\sigma$, which holds by Definitions 4.5 and 3.8 whatever block $\mathsf{snapshot}(\mathsf{LF}(H))$ is. None of these refers to the behaviour of Π_{bft}: validity in Π_{bft} is objective (Remark 4.11), not a matter of the conduct of the holders of voting units; the proof uses neither Final Agreement

(Definition 3.31), nor Assumption 3.23, nor Assumption 3.24, nor Assumption 3.20. The theorem therefore holds in every execution with the premise, whatever votes are cast and whatever signatures exist. □

The corollary makes precise the consequence that the TFL book argues from the fact recorded in Remark 4.11: the finalised chain stays safe under a complete break of Π_{bft}, its signature scheme included, while Π_{bc} is safe (chapter "The Crosslink 2 Construction", section "Π_{bft} changes from Π_{origbft}"). It meets item (1) of Remark 2.17, with Prefix Agreement of Π_{bc} at σ in place of the condition that the adversary holds under half of the hash rate. Whether Π_{bc} keeps Prefix Agreement while Π_{bft} is subverted is a separate question, which the corollary does not address: validity in Π_{bc} depends on Π_{bft} (Remark 4.23), and transfer of the prefix properties from Π_{origbc} to Π_{bc} is an open problem, recorded with the liveness of the sub-protocols.

6.3 The Final Agreement route

Theorem 6.5 (Final Agreement implies Assured Finality)**.** *In every execution of Crosslink 2 in which* Π_{bft} *has Final Agreement (Definition 3.31), Assured Finality holds.*
Status: proved (this volume), of a designed-but-unspecified construction.

Proof. Let node i be honest at t and node j honest at u. The chain $\mathcal{O}_{\text{bc}}$ is a prefix of every bc-chain, so the claim holds if $\mathsf{fin}_i^t = \mathcal{O}_{\text{bc}}$ or $\mathsf{fin}_j^u = \mathcal{O}_{\text{bc}}$; let both differ from $\mathcal{O}_{\text{bc}}$. Since $\mathsf{fin}_i^0 = \mathcal{O}_{\text{bc}}$, update-fin (Definition 5.1) has set fin_i to a candidate at some time at most t; let $r \le t$ be the last such time, so that $\mathsf{fin}_i^t = \mathsf{candidate}(\mathsf{ch}_i^r)$. By Definitions 4.5 and 3.8,

$$\mathsf{fin}_i^t \preceq_{\text{bc}} \mathsf{snapshot}(\mathsf{LF}(\mathsf{ch}_i^r)).$$

Node i is honest at r, so ch_i^r is valid in Π_{bc} (Definition 3.11). It is not $\mathcal{O}_{\text{bc}}$, whose candidate is $\mathcal{O}_{\text{bc}}$ by the Genesis bc-block rule; so the Valid context rule (Definition 4.17) makes $C_i = \mathsf{ch}_i^r.\mathsf{context_bft}$ a valid bft-block. Node i observed it, while honest, in validating ch_i^r, so the block C_i is in honest view (Definition 3.26), and $\mathsf{LF}(\mathsf{ch}_i^r) = \text{bft-last-final}(C_i)$ (Definition 4.4). In the same way there are a time $s \le u$ and a valid bft-block C_j in honest view with

$$\mathsf{fin}_j^u = \mathsf{candidate}(\mathsf{ch}_j^s) \preceq_{\text{bc}} \mathsf{snapshot}(\mathsf{LF}(\mathsf{ch}_j^s)) \quad \text{and} \quad \mathsf{LF}(\mathsf{ch}_j^s) = \text{bft-last-final}(C_j).$$

Final Agreement makes $\mathsf{LF}(\mathsf{ch}_i^r)$ and $\mathsf{LF}(\mathsf{ch}_j^s)$ agree; by symmetry let $\mathsf{LF}(\mathsf{ch}_i^r) \preceq_{\text{bft}} \mathsf{LF}(\mathsf{ch}_j^s)$. The block $\mathsf{LF}(\mathsf{ch}_j^s)$ is valid (Definition 3.29), so by the Parent rule every bft-block D with $\mathsf{LF}(\mathsf{ch}_i^r) \prec_{\text{bft}} D \preceq_{\text{bft}} \mathsf{LF}(\mathsf{ch}_j^s)$ is valid, and the Linearity rule (Definition 4.9) gives $\mathsf{snapshot}(D\lceil^1) \preceq_{\text{bc}} \mathsf{snapshot}(D)$ for each. By transitivity along this path,

$$\mathsf{snapshot}(\mathsf{LF}(\mathsf{ch}_i^r)) \preceq_{\text{bc}} \mathsf{snapshot}(\mathsf{LF}(\mathsf{ch}_j^s)).$$

Both fin_i^t and fin_j^u are therefore prefixes of $\mathsf{snapshot}(\mathsf{LF}(\mathsf{ch}_j^s))$, and Lemma 3.10 makes them agree. □

The proof uses Final Agreement of Π_{bft}, the Genesis bc-block, Valid context, Parent and Linearity rules and update-fin. It uses neither σ nor Prefix Agreement, nor the Extension, Last Final

Snapshot or Finality depth rules. The TFL book's printed proof of this theorem (chapter "Security Analysis of Crosslink 2", section "Safety argument") is a verbatim copy of its proof of Theorem 6.3; the admonition "What does the Linearity rule do?" of its construction chapter sketches the route taken above.

A randomised check by script agrees with the theorem: in 1500 executions with one bft-chain obeying the Linearity rule and every block final in its own context, over 4500 node runs, 1949 of them with forks deeper than σ, so that Prefix Agreement at σ fails, all values of fin agree. The Linearity rule cannot be dropped from the theorem.

Proposition 6.6 (Necessity of the Linearity rule)**.** *Let the Linearity rule be removed from Definition 4.9. For every σ there is an execution in which Π_{bft} has Final Agreement, the best chains of two honest nodes i and j satisfy the Valid context, Extension and Last Final Snapshot rules, the voters of every bft-block they cite meet the Confirmed best-chain criterion (Construction 4.14), and fin_i and fin_j conflict.*

Status: proved (this volume), of a designed-but-unspecified construction.

Proof. Let Π_{origbft} be a protocol in which every valid bft-block is final in its own context, $\mathsf{bft\text{-}last\text{-}final}(B) = B$, as in a protocol that decides each block individually; the identity on valid blocks meets Definition 3.29. Let X and Y be two bc-branches that fork at $\mathcal{O}_{\text{bc}}$, with blocks $X_1, \dots, X_{\sigma+2}$ and $Y_1, \dots, Y_{\sigma+2}$, the index being the height (Figure 3).

BFT side. Let $B_0 = \mathcal{O}_{\text{bft}}$; let B_1 be a bft-block with parent B_0 whose field headers_bc is the σ-block tail of the chain $X_{\sigma+1}$, the headers of $X_2, \dots, X_{\sigma+1}$; and let B_2 be a bft-block with parent B_1 whose field headers_bc is the σ-block tail of $Y_{\sigma+1}$. Then $\mathsf{snapshot}(B_1) = X_1$ and $\mathsf{snapshot}(B_2) = Y_1$ (Definition 4.3). Both blocks satisfy the Tail Confirmation rule. The block B_1 also satisfies the Linearity rule, since $\mathsf{snapshot}(B_0) = \mathcal{O}_{\text{bc}} \preceq_{\text{bc}} X_1$; the block B_2 violates it, since X_1 and Y_1 conflict, and is valid once that rule is removed. The only bft-blocks are B_0, B_1 and B_2, on one chain, so any two values of bft-last-final agree and Π_{bft} has Final Agreement.

Voting. At least two thirds of the units of B_1's BFT epoch are cast by validators whose best chain is $X_{\sigma+1}$, and at least two thirds of those of B_2's by validators whose best chain is $Y_{\sigma+1}$. Each such validator meets the Confirmed best-chain criterion, since $X_1 = X_{\sigma+1}\lceil^{\sigma}$ and $Y_1 = Y_{\sigma+1}\lceil^{\sigma}$. Let B_2 be of a later BFT epoch than B_1, and let $\mathsf{score}(X_{\sigma+1}) < \mathsf{score}(Y_{\sigma+1})$, which Definition 3.11 permits. A network partition that hides Y from the validators of B_1 and from node i permits this. The validators of B_2 and node j validate B_1, and so obtain $X_1, \dots, X_{\sigma+1}$, but keep Y as their best chain. It breaks Prefix Consistency (Definition 3.15), since $X_{\sigma+1}\lceil^{\sigma}$ is not a prefix of the later best chain $Y_{\sigma+1}$ of an honest validator; no unit is cast non-honestly, so Assumption 3.23 holds.

Best chains. Node i is honest, and its best chain grows from $\mathcal{O}_{\text{bc}}$ through $X_1, \dots, X_{\sigma+2}$; the blocks $X_1, \dots, X_{\sigma+1}$ cite $\mathcal{O}_{\text{bft}}$, and the tip $X_{\sigma+2}$ cites B_1. Node j follows Y in the same way, with $Y_{\sigma+2}$ citing B_2. On X the Valid context rule holds, $\mathcal{O}_{\text{bft}}$ and B_1 being valid; the Extension rule holds, since $\mathsf{LF}(X_k) = \mathcal{O}_{\text{bft}}$ for $k \le \sigma + 1$ and $\mathcal{O}_{\text{bft}} \preceq_{\text{bft}} B_1 = \mathsf{LF}(X_{\sigma+2})$; the Last Final Snapshot rule holds, since $\mathsf{snapshot}(\mathcal{O}_{\text{bft}}) = \mathcal{O}_{\text{bc}}$ and $X_1 \preceq_{\text{bc}} X_{\sigma+2}$; and the headers carried by B_1 are those of strict ancestors of $X_{\sigma+2}$, so no block commits to its own hash. The same holds on Y. The Finality depth rule is met by letting each block whose finality depth exceeds L be a stalled block (Remark 4.26), which changes no value of fin.

Finalised chains. For $k \le \sigma + 1$, $\mathsf{candidate}(X_k) = \mathsf{lca}(\mathcal{O}_{\mathrm{bc}}, X_k\lceil^{\sigma}) = \mathcal{O}_{\mathrm{bc}}$, so fin_i stays $\mathcal{O}_{\mathrm{bc}}$; at the tip,

$$\mathsf{candidate}(X_{\sigma+2}) = \mathsf{lca}(X_1, X_2) = X_1,$$

of which $\mathcal{O}_{\mathrm{bc}}$ is a prefix, so update-fin sets $\mathsf{fin}_i = X_1$. In the same way $\mathsf{fin}_j = Y_1$. The blocks X_1 and Y_1 conflict. □

A script checks the construction for every $\sigma \le 20$: the two values of fin conflict at height 1, with the tips at height $\sigma + 2$. In the execution Prefix Agreement at σ fails, as Theorem 6.3 requires, since $X_{\sigma+2}\lceil^{\sigma} = X_2$ and $Y_{\sigma+2}\lceil^{\sigma} = Y_2$ conflict. The proposition is the only result in any source that shows a rule of Crosslink 2 necessary for Assured Finality.

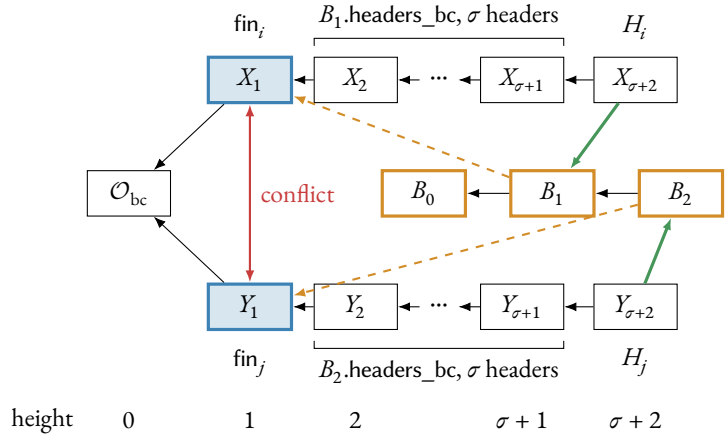


Figure 3. The execution of Proposition 6.6, symbolic in σ. The bft-chain $B_0 = \mathcal{O}_{\mathrm{bft}}, B_1, B_2$ (orange outline) has Final Agreement. The best-chain branches X and Y fork at $\mathcal{O}_{\mathrm{bc}}$; B_1 carries the headers of $X_2, \ldots, X_{\sigma+1}$ and B_2 those of $Y_2, \ldots, Y_{\sigma+1}$, so the dashed arrows give $\mathsf{snapshot}(B_1) = X_1$ and $\mathsf{snapshot}(B_2) = Y_1$, and B_2 violates the Linearity rule. The green arrows are the fields context_bft of the tips $H_i = X_{\sigma+2}$ and $H_j = Y_{\sigma+2}$; every other best-chain block cites $\mathcal{O}_{\mathrm{bft}}$. The finalised chains $\mathsf{fin}_i = X_1$ and $\mathsf{fin}_j = Y_1$ (blue) conflict (red). Solid black arrows point to parents.

Remark 6.7 (Linearity in the book). The construction chapter of the TFL book calls the Linearity rule "key to combining simplicity with strong security properties in Crosslink 2" and argues that under Final Agreement it implies Assured Finality "without needing to assume any safety property of Π_{bc}". A sentence marked "Edit" in the security analysis, in its section "Every rule in Crosslink 2 is individually necessary", written of the Increasing Score rule of Crosslink 1 and renamed for Crosslink 2, states that "some rules, e.g. the Linearity rule, only contribute heuristically to security in the analysis so far". It holds of the analysis as printed (see the discussion after Theorem 6.5) and conflicts with the construction chapter; Theorem 6.5 with Proposition 6.6 shows the rule's contribution proved, not heuristic. The chapter "Questions About Crosslink" opens by asking why there is no bc-block-validity rule $\mathsf{snapshot}(\mathsf{LF}(H)) \preceq_{\mathrm{bc}} H$, which is the Last Final Snapshot rule that Crosslink 2 then adopted; that chapter predates the construction and is not read against it.

6.4 Consequences of the two routes

Corollary 6.8 (Assured Finality under either premise)**.** *Assured Finality holds in every execution in which* Π_{bc} *has Prefix Agreement at* σ *or* Π_{bft} *has Final Agreement.*
Status: proved (this volume), of a designed-but-unspecified construction.

Proof. Theorem 6.3 gives the first case and Theorem 6.5 the second. □

The finalised chains of two honest nodes can therefore conflict only in an execution in which both premises fail.

Remark 6.9 (Role of the best-chain rules). Neither safety proof uses the Extension, Last Final Snapshot or Finality depth rule. The proof of Theorem 6.3 needs only update-fin, the Valid context rule, which makes candidate(H) defined, and candidate(H) $\preceq_{\text{bc}} H\lceil^{\sigma}$; the proof of Theorem 6.5 needs the Genesis bc-block, Valid context, Parent and Linearity rules and update-fin. The three rules serve other ends. The Extension and Last Final Snapshot rules, with the Linearity rule, make the final snapshot of each best-chain block lie on its chain and advance along it (Lemma 4.19 and (1)), and the candidate advance with it (Lemmas 4.18 and 4.28), which serves the spending of finalised outputs (item (4) of Remark 2.17); the Finality depth rule serves bounded availability, through Stalled Mode (Definition 4.24).

Remark 6.10 (Residual attack). Let the network be partitioned, and let the adversary hold more than one third of the voting units of a BFT epoch. Voting with the honest validators on each side, it can make each side's last final bft-block consistent with the fork that side follows, and the finalised chains of honest nodes on the two sides then conflict. Both premises of Corollary 6.8 fail in such an execution: the partition breaks Prefix Agreement, and Final Agreement fails, the two last final bft-blocks being in conflict; the units cast on both sides violate Assumption 3.23. The TFL book calls the attack unavoidable, since a best-chain protocol has no mechanism that maintains finality under partition, and contrasts it with snap-and-chat, which is unsafe without any partition once Π_{bft} is subverted (chapter "Security Analysis of Crosslink 2", section "Safety argument").
Status: argued (TFL book).

Remark 6.11 (Necessity of the other rules). Beyond Proposition 6.6, no result shows a rule of Crosslink 2 necessary for Assured Finality or for bounded availability. The section "Every rule in Crosslink 2 is individually necessary" of the TFL book's security analysis contains only its statement of intent and the edit quoted in Remark 6.7. Witness runs of a model in Z. Manian's Quint suite for Crosslink (18 May to 4 June 2026) that is aligned with the book restate the Last Final Snapshot and Extension rules as invariants and show that dropping either rule violates its own restatement; the model is only witness-tested (Remark 12.3), and that is not a necessity result.
Status: open problem.

6.5 Safety of the bounded-available chain

Theorem 6.12 (Prefix Consistency implies Prefix Consistency of ba)**.** *Let $\mu \le \sigma$ (Definition 5.6). In every execution in which Π_{bc} has Prefix Consistency at depth μ (Definition 3.15), $(\mathsf{ba}_\mu)_i^t \preceq_{\mathrm{bc}} \mathsf{ch}_j^u$ for all times $t \le u$, every node i honest at t and every node j honest at u.*

Status: proved (TFL book), of a designed-but-unspecified construction.

Proof. By Lemma 5.9 there is a time $r \le t$ with $(\mathsf{ba}_\mu)_i^t \preceq_{\mathrm{bc}} \mathsf{ch}_i^r\lceil^\mu$; node i is honest at r, and $r \le u$. Prefix Consistency at μ, applied at (i, r) and (j, u), gives $\mathsf{ch}_i^r\lceil^\mu \preceq_{\mathrm{bc}} \mathsf{ch}_j^u$, and transitivity of $\preceq_{\mathrm{bc}}$ ends the proof. □

The hypothesis is Prefix Consistency at the depth μ at which the node reads ba. Since $C\lceil^{k'} \preceq_{\mathrm{bc}} C\lceil^k$ for $k \le k'$, Prefix Consistency at a depth implies it at every greater depth, so for $\mu < \sigma$ the hypothesis is stronger than Prefix Consistency at σ. The TFL book states the theorem in chapter "Security Analysis of Crosslink 2" below its marker "Not updated for Crosslink 2 below this point", as it does the two theorems on Assured Finality; all three are written in Crosslink 2 notation.

Proposition 6.13 (Retention of the finalised prefix in ba)**.** *For times $t \le u$ and a node i honest at u, $\mathsf{fin}_i^t \preceq_{\mathrm{bc}} (\mathsf{ba}_\mu)_i^u$. In every execution with Assured Finality, for a node i honest at t and a node j honest at u, the shorter of fin_i^t and fin_j^u is a prefix of both $(\mathsf{ba}_\mu)_i^t$ and $(\mathsf{ba}_\mu)_j^u$.*

Status: proved (this volume), of a designed-but-unspecified construction.

Proof. Lemma 5.2 gives $\mathsf{fin}_i^t \preceq_{\mathrm{bc}} \mathsf{fin}_i^u$, and Proposition 5.8 gives $\mathsf{fin}_i^u \preceq_{\mathrm{bc}} (\mathsf{ba}_\mu)_i^u$; transitivity gives the first claim. Under Assured Finality the chains fin_i^t and fin_j^u agree, so the shorter of them is a prefix of both, hence of $(\mathsf{ba}_\mu)_i^t$ and of $(\mathsf{ba}_\mu)_j^u$ by Proposition 5.8 and transitivity. □

The proof uses facts about each node separately, so it holds with a different depth μ at each node. The proposition does not claim that $(\mathsf{ba}_\mu)_i^t$ and $(\mathsf{ba}_\mu)_j^u$ agree: both extend the shorter finalised chain, and Lemma 3.10 orders two prefixes of one chain, not two extensions of one prefix. Nor does it bound how far ba_μ can roll back above fin; the TFL book notes that such rollbacks "may be of unbounded length". The book's version, "Theorem: $\mathsf{LOG}_{\mathsf{ba}}$ does not roll back past the agreed $\mathsf{LOG}_{\mathsf{fin}}$" (section "Proof of safety for $\mathsf{LOG}_{\mathsf{ba}}$"), is written in Crosslink 1 notation, over transaction logs, and transfers directly to the chains of Crosslink 2.

Definition 6.14 (Agreement on ba)**.** Agreement on ba holds in an execution iff Agreement on the view $(i, t) \mapsto (\mathsf{ba}_\mu)_i^t$ holds (Definition 3.9).

The TFL book states Agreement on ba as its security goal for ba_μ (chapter "The Crosslink 2 Construction", section "Locally bounded-available chain") and does not prove it; Corollary 6.15 derives it from Prefix Agreement at μ.

Corollary 6.15 (Agreement on ba from Prefix Agreement)**.** *In every execution in which Π_{bc} has Prefix Agreement at depth μ, and in particular in every execution in which it has Prefix Consistency at μ, Agreement on* ba *holds; with depths differing between nodes, Prefix Agreement at the smallest suffices.*

Status: proved (this volume), of a designed-but-unspecified construction.

Proof. Let node i be honest at t and node j at u, with depths μ_i and μ_j at least μ. Lemma 5.9 gives times $r \le t$ and $s \le u$, at which the nodes are honest, with $(\mathsf{ba}_{\mu_i})_i^t \preceq_{\mathrm{bc}} \mathsf{ch}_i^r\lceil^{\mu_i} \preceq_{\mathrm{bc}} \mathsf{ch}_i^r\lceil^{\mu}$ and $(\mathsf{ba}_{\mu_j})_j^u \preceq_{\mathrm{bc}} \mathsf{ch}_j^s\lceil^{\mu}$ (Definition 3.5). Prefix Agreement at μ makes the two truncated chains agree; by symmetry let the first be a prefix of the second. Both values of ba are then prefixes of $\mathsf{ch}_j^s\lceil^{\mu}$, and Lemma 3.10 makes them agree. Prefix Consistency at μ implies Prefix Agreement at μ (§3.3). □

Remark 6.16 (Missing results for ba). Agreement on ba holds under Prefix Agreement at μ (Corollary 6.15). It does not follow from Final Agreement, which constrains only Π_{bft}: with $\mathcal{O}_{\mathrm{bft}}$ the only bft-block, two best chains that fork at $\mathcal{O}_{\mathrm{bc}}$, each with more than μ blocks above $\mathcal{O}_{\mathrm{bc}}$, give $\mathsf{fin} = \mathcal{O}_{\mathrm{bc}}$ at both nodes and conflicting values of ba. Nor does it follow from Prefix Agreement at σ when $\mu < \sigma$: a fork more than μ and at most σ blocks below both tips gives conflicting values of ba. The safety of the transaction sequence of ba_{μ} (Remark 5.11) is therefore conditional on Prefix Agreement at μ, an assumption for Zcash (Remark 3.18). The TFL book's two results on snapshots in the finalised and bounded-available ledgers, under Assumption 3.23 and Prefix Consistency, are written in Crosslink 1 notation, and its argument for the bounded-available ledger breaks off ("What we want to show is that, under some conditions on executions, ..."); no Crosslink 2 form exists. No analogue for Crosslink 2 of the dynamic-availability property P2 of Definition 2.7 is proved, and no bound on the rollback of ba_{μ}.

Status: open problem.

7 Liveness

This section states each liveness claim of the TFL book with its status. The claims are those of chapter "Security Analysis of Crosslink 2", section "Liveness argument", with the transfer argument of its section "Proof of safety for $\mathsf{LOG}_{\mathrm{fin}}$". Four are argued in prose and none is proved, and the section proves one necessary condition, $L \ge \sigma + 1$; the transfer of the prefix properties from Π_{origbc}, the advance of finalisation and the choice of L are open problems. The book defines no liveness property of either sub-protocol: "live" below has the meaning of whatever liveness property is assumed of Π_{origbc} or Π_{origbft}, and for Π_{bc} the property argued is growth of the best chain (Proposition 7.1). The book argues that the dependence of each sub-protocol's validity on the other (Remark 4.23) does not obstruct liveness because the fork choice of Π_{bc} is unchanged (Definition 4.22), unlike that of a protocol whose fork choice follows its finality layer.

7.1 Liveness of the sub-protocols

Proposition 7.1 (Liveness of Π_{bc})**.** *Under the conditions under which the best chain of* Π_{origbc} *grows, the best chain of* Π_{bc} *grows, given that coinbase-only stalled blocks are always producible.*

Status: argued (TFL book).

The book's argument is the following. Let the set of bc-blocks that are both coinbase-only (Definition 3.13) and stalled (Definition 4.1) be such that a producer can always produce a block in it (Remark 4.26). No rule of Definition 4.17 then prevents producing such a block on the current best chain: the Genesis bc-block rule concerns only the genesis block; the Valid context, Extension

and Last Final Snapshot rules hold when the new block cites the context of its parent; and the Finality depth rule always admits a stalled block. An honest producer may produce such a block (Construction 4.20), so the best chain of Π_{bc} grows under the conditions under which that of Π_{origbc} does.

The argument is prose. It also lists a change to contextual validity that does not affect coinbase transactions; that change belongs to Crosslink 1, and Crosslink 2 changes no contextual rule (Definition 4.17), so the item is vacuous. What the argument shows is that a valid block can always be produced, so the property it transfers is the growth of the best chain. Inclusion of user transactions is not preserved: the book notes that Crosslink 2 intentionally gives up availability during a long finalisation stall, whose blocks may all be coinbase-only. For Zcash the liveness of Π_{origbc} is itself an assumption: the liveness clause of Assumption "Persistence and liveness" (*FlyClient Guide*, §"The variable-difficulty backbone model") is adopted there without proof.

Proposition 7.2 (Liveness of Π_{bft})**.** *The protocol* Π_{bft} *is live whenever* Π_{origbft} *is.*
Status: argued (TFL book).

The book argues as follows. Since Π_{bc} is live, a fresh header chain that satisfies the Linearity and Tail Confirmation rules (Definition 4.9) eventually exists for the field headers_bc of a proposal. When none is available, an honest proposer repeats the field of the parent bft-block (Construction 4.13), which satisfies both rules, so an honest proposer can always make a valid proposal, as the liveness of a protocol such as adapted Streamlet (Example 3.33) requires. The changes to voting only require a vote to be for a proposal that "could have been honestly proposed", and finality in context is unchanged from Π_{origbft}.

The argument rests on Proposition 7.1, itself argued. It does not address the Confirmed best-chain criterion of Construction 4.14, which each validator checks against its own best chain: whether enough voting units can be cast for an honest proposal depends on the validators' best chains, and the construction chapter relies there on enough validators being synced that Prefix Consistency holds for them.

Proposition 7.3 (Transfer by erasure)**.** *Every safety property of* Π_{origbft}*, Final Agreement included, holds for* Π_{bft}*.*
Status: argued (TFL book).

The book argues that the changes from Π_{origbft} to Π_{bft} only add the field headers_bc (Definition 4.2) and tighten validity (Definition 4.9). Erasing the added field therefore maps every execution of Π_{bft} to an execution of Π_{origbft} with the same nodes honest at every time. A safety property asserts only that certain executions do not occur and does not mention the added field, so each safety property of Π_{origbft}, Final Agreement included, holds for Π_{bft}; liveness is the subject of Proposition 7.2. The argument is sound in outline, but two of its steps are unjustified. Erasure does not by itself give an execution of Π_{origbft}: the proposer's signatures, the ballots, the notarisation proofs and the parent hashes commit to headers_bc (Definitions 3.19, 3.21 and 3.26), so the field must instead be read as part of the content of a Π_{origbft} proposal, which the interface leaves open. And the book asserts without argument that the same nodes are honest: honest voting in Π_{bft} is "only if" (Remark 4.15), so the image may withhold votes that honest voting in Π_{origbft} would cast.

Remark 7.4 (Transfer from Π_{origbc}). That Prefix Consistency and Prefix Agreement transfer from Π_{origbc} to Π_{bc} is unproved. Of Prefix Agreement the TFL book holds that the changes to Π_{bc} "seem unlikely to have any significant effect on this property, given that we proved above that they do not affect liveness", the liveness argument being prose (Proposition 7.1), and marks the sentence "TODO: that is a handwave; we should be able to do better, as we do for Π_{bft} below" (section "Proof of safety for $\mathsf{LOG}_{\text{fin}}$"). Erasure does not settle the question: an honest node of Π_{bc} rejects blocks valid in Π_{origbc} that fail a rule of Definition 4.17, so its best chain need not be that of an honest node of Π_{origbc} with the same view, and the rules it adds depend on Π_{bft}. This is the open problem to which the caveat of Corollary 6.4 refers. For Zcash the prefix properties of Π_{origbc} are themselves assumptions (Remark 3.18).

Status: open problem.

7.2 Progress of finalisation

Remark 7.5 (Advance of finalisation). That finalisation advances, that is, that a later snapshot eventually enters fin when both sub-protocols are live and enough participants are honest, is unproved. The TFL book argues that liveness of Π_{bft} and the Linearity rule imply that, "provided there are sufficient honest bft-proposers/validators", a bft-block with a higher-scoring snapshot is eventually final in the context of the longest valid bft-chain; that by the Extension rule it descends from the previous last final block in the context that best-chain producers see; and that the new finalised chain therefore extends the old. It marks the first step "TODO make that more precise". Neither the number of honest participants nor any time bound is stated.

Status: open problem.

Proposition 7.6 (Stalled Mode only on failure of the liveness assumptions)**.** *While there are sufficient honest best-chain producers and sufficient honest bft-proposers and validators, Stalled Mode is not entered, provided that L is large enough relative to σ, the network delay, the honest hash rate and the details of Π_{bft}.*

Status: argued (TFL book).

The book argues informally that, while there are sufficient honest best-chain producers and sufficient honest bft-proposers and validators, the finalisation point implied by the field context_bft at the tip of each node's best chain advances fast enough for the bound L not to be reached. It states that this depends on L relative to σ, on the network delay, on the hash rate of honest producers, on the number of honest proposers and the proportion of voting units they hold, and on details of the BFT protocol, and it marks the argument "TODO more detailed argument needed, especially for the dependence on L". No hypothesis is quantified.

Two results show hypotheses of the statement to be necessary. By Proposition 4.30, producers that never advance context_bft over L consecutive heights force a stalled block whatever Π_{bft} does, so honest best-chain producers are needed as well as a live Π_{bft}. By the following proposition, Stalled Mode persists while Π_{bft} is live if $L \le \sigma$, so $L \ge \sigma + 1$ is needed.

Proposition 7.7 (Stall when $L \le \sigma$)**.** *If $L \le \sigma$, then in every execution in which the bc-blocks of all views lie on one bc-chain, every participant is honest and no voting unit is cast non-honestly (Def-*

> *inition 3.22), every bc-valid block of height greater than L in a view is in Stalled Mode, whether or not* Π_{bft} *is live.*
>
> *Status: proved (this volume), of a designed-but-unspecified construction.*

Proof. Let H be such a block. If $\mathsf{LF}(H) = \mathcal{O}_{\text{bft}}$, then $\mathsf{finality\text{-}depth}(H) = \mathsf{height}(H) > L$, since $\mathsf{snapshot}(\mathcal{O}_{\text{bft}}) = \mathcal{O}_{\text{bc}}$. Otherwise, by the Tail Confirmation rule, $\mathsf{LF}(H)$ carries the σ-block tail of a bc-valid chain T'. Its notarisation proof is valid, so at least one unit was cast for the proposal of $\mathsf{LF}(H)$ (Definition 3.21), honestly by hypothesis, and its holder downloaded the blocks of the carried headers (Construction 4.14(i)); so T' is in a view and lies with H on one bc-chain. It is neither H nor a descendant of H, since either would give a cycle of commitments through H.context_bft and $\mathsf{LF}(H)$ (Remark 4.10); so T' is an ancestor of H, the carried headers are headers of ancestors of H, and Proposition 5.14(ii) gives $\mathsf{finality\text{-}depth}(H) \geq \sigma + 1 > L$. □

Remark 7.8 (Choice of L). No source gives a rule for choosing L. The TFL book requires L "significantly greater than σ" and states that in practice it "should be *at least* 2σ" (chapter "The Crosslink 2 Construction", sections "Parameters" and "Stalled Mode"), without derivation, and its liveness argument marks the dependence on L as missing. The condition $L \geq \sigma + 1$, necessary for Proposition 7.6 by Proposition 7.7, is a lower bound only.

Status: open problem.

8 Bounded availability and finality overrides

This section states the TFL book's informal notion of bounded availability and the book's case for it, proves the precise statement for Crosslink 2 that the book leaves unproved, shows that the statement needs $\sigma \leq L$, and states the book's argument for finality overrides, for which the construction has no rule. The objects of the section are designed but unspecified (Definition 1.1), and each claim carries its status.

8.1 Bounded availability

Remark 8.1 (Bounded availability). Bounded availability is the TFL book's weakening of dynamic availability, the property P2 of Definition 2.7: the finalisation gap, the number of best-chain blocks above the finalisation point, may grow during a finalisation stall only up to a bound, beyond which ordinary spends stop. The book does not define it formally: "We do not define bounded availability formally in this document" (chapter "The Arguments for Bounded Availability and Finality Overrides", section "What is Bounded Availability?"). Its only formal property, Bounded hazard-freeness, is stated for snap-and-chat (chapter "Notes On Snap-and-Chat"): no node i at time t observes in $\mathsf{LOG}^t_{\text{da},i}$ a hazardous transaction, such as a spend, from a block H of ch^t_i with $\mathsf{height}(H) - \mathsf{height}(\mathsf{lca}(\mathsf{snapshot}(\mathsf{LF}(H)), H)) > L$, the Crosslink 1 form of the finality depth. The book gives it no form for Crosslink 2. Read with $\mathsf{LOG}^t_{\text{ba},i}$ for $\mathsf{LOG}^t_{\text{da},i}$ (Remark 5.11), it holds for Crosslink 2 under the hypotheses of Proposition 8.2. It bounds spends relative to the final snapshot of each block; the next subsection bounds them relative to fin^t_i.

Proposition 8.2 (Bounded hazard-freeness)**.** *Suppose that no stalled block carries a hazardous transaction, and let the best chain* ch_i^t *of node* i *be bc-valid. Then no block* $H \preceq_{\mathrm{bc}} \mathsf{ch}_i^t$ *with* $\mathsf{height}(H) - \mathsf{height}(\mathsf{lca}(\mathsf{snapshot}(\mathsf{LF}(H)), H)) > L$ *contributes a hazardous transaction to* $\mathsf{LOG}_{\mathrm{ba},i}^t$.

Status: proved (this volume), of a designed-but-unspecified construction.

Proof. For $H = \mathcal{O}_{\mathrm{bc}}$ the difference of heights is $0 < L$, since $\mathsf{snapshot}(\mathsf{LF}(\mathcal{O}_{\mathrm{bc}})) = \mathcal{O}_{\mathrm{bc}}$ (Definitions 4.17 and 4.1), so $H \neq \mathcal{O}_{\mathrm{bc}}$; as a block of the bc-valid chain ch_i^t, H is bc-block-valid (Definition 4.17). By the Last Final Snapshot rule $\mathsf{snapshot}(\mathsf{LF}(H)) \preceq_{\mathrm{bc}} H$, so the last common ancestor is $\mathsf{snapshot}(\mathsf{LF}(H))$ (Definition 3.8) and the difference is $\mathsf{finality\text{-}depth}(H) > L$ (Definition 4.6); the Finality depth rule makes H stalled, and a stalled block carries no hazardous transaction. □

Remark 8.3 (Rationale of bounded availability). The TFL book argues as follows (chapter "The Arguments for Bounded Availability and Finality Overrides"). A partition can stall any protocol with finality for as long as it lasts, a consequence of the CAP theorem that the book attributes, made precise, to Lewis-Pye and Roughgarden. Under strict dynamic availability the best chain meanwhile keeps advancing, so the finalisation gap grows without bound while spends continue, and the cost of rolling back spent funds grows with it. Bounded availability caps that exposure at L blocks and then stops ordinary spends. The best chain keeps producing stalled blocks rather than halting, because a halt gives an attacker unbounded time to build a fork of the recent tail, which the book calls a tail-thrashing attack; the book adds that honest hash rate is expected to fall once a stall is known, miners expecting their blocks to be rolled back, which raises the attacker's share.

Status: argued (TFL book).

The book's example of tail thrashing: an attacker with a tenth of the hash rate finds in expectation one block per 10 block times, hence, by linearity of expectation (*Math Guide*, §"Random variables and expectation"), 10 blocks, a 10-block fork, in 100 block times.

8.2 The bounded non-stalled run

Proposition 8.4 (Bounded non-stalled run)**.** *Let* $H = \mathsf{ch}_i^t$*, and let node* i *have followed the update rule* update-fin *(Definition 5.1) up to* t*. Suppose that the Extension, Last Final Snapshot and Finality depth rules hold along* H*, that the Parent and Linearity rules hold along the bft-chain that ends at* $\mathsf{LF}(H)$*, that* $\sigma \leq L$*, and that* $\mathsf{fin}_i^t \preceq_{\mathrm{bc}} H$*. Then every block of* H *at height greater than* $\mathsf{height}(\mathsf{fin}_i^t) + L$ *is stalled.*

Status: proved (this volume), of a designed-but-unspecified construction.

Proof. The Extension rule presupposes that every block of H has a last final context, so each context cited along H is a valid bft-block (Definition 3.29). Write $S = \mathsf{snapshot}(\mathsf{LF}(H))$, the final snapshot of H. By the Last Final Snapshot rule $S \preceq_{\mathrm{bc}} H$, and $\mathsf{fin}_i^t \preceq_{\mathrm{bc}} H$ by hypothesis, so S and fin_i^t agree (Lemma 3.10).

First, $\mathsf{snapshot}(\mathsf{LF}(H')) \preceq_{\mathrm{bc}} S$ for every $H' \preceq_{\mathrm{bc}} H$. Lemma 4.19 gives $\mathsf{LF}(H') \preceq_{\mathrm{bft}} \mathsf{LF}(H)$; by the Parent rule every bft-block D with $\mathsf{LF}(H') \prec_{\mathrm{bft}} D \preceq_{\mathrm{bft}} \mathsf{LF}(H)$ is valid, the Linearity rule gives $\mathsf{snapshot}(D\lceil^1) \preceq_{\mathrm{bc}} \mathsf{snapshot}(D)$ for each, and transitivity gives the claim.

Case $S \preceq_{\mathrm{bc}} \mathsf{fin}_i^t$. Let $H' \preceq_{\mathrm{bc}} H$ with $\mathsf{height}(H') > \mathsf{height}(\mathsf{fin}_i^t) + L$. Then $\mathsf{snapshot}(\mathsf{LF}(H')) \preceq_{\mathrm{bc}} S \preceq_{\mathrm{bc}} \mathsf{fin}_i^t$, so

$$\mathsf{finality\text{-}depth}(H') \geq \mathsf{height}(H') - \mathsf{height}(\mathsf{fin}_i^t) > L,$$

and the Finality depth rule makes H' stalled.

Case $\mathsf{fin}_i^t \prec_{\mathrm{bc}} S$. Then $H \neq \mathcal{O}_{\mathrm{bc}}$, so node i's best chain has changed to H, and by Lemma 4.18 the chain $\mathsf{candidate}(H)$ is the shorter of S and $H\lceil^{\sigma}$. At that change, node i following update-fin, the rule either set $\mathsf{fin}_i^t = \mathsf{candidate}(H)$, or kept a value with $\mathsf{fin}_i^t \not\preceq_{\mathrm{bc}} \mathsf{candidate}(H)$. In the first sub-case $\mathsf{candidate}(H) \prec_{\mathrm{bc}} S$, so $\mathsf{candidate}(H) = H\lceil^{\sigma}$ and $\mathsf{fin}_i^t = H\lceil^{\sigma}$. In the second, fin_i^t and $\mathsf{candidate}(H)$ are prefixes of H and agree, so $\mathsf{candidate}(H) \prec_{\mathrm{bc}} \mathsf{fin}_i^t \prec_{\mathrm{bc}} S$; again $\mathsf{candidate}(H) = H\lceil^{\sigma}$, now a strict prefix of fin_i^t. In both sub-cases

$$\mathsf{height}(H) - \mathsf{height}(\mathsf{fin}_i^t) \leq \sigma \leq L,$$

so no block of H lies at height greater than $\mathsf{height}(\mathsf{fin}_i^t) + L$. □

The proof uses $\sigma \leq L$ only in the second case. A randomised check by script, over 3000 trials of random block trees, bft-chains and best-chain switches, finds, in 270980 checks of 44776 distinct blocks, every block more than L above fin stalled. The proposition is the precise Crosslink 2 form of the TFL book's remark that "the number of non-Stalled Mode bc-blocks after the consensus finalization point is bounded by L, but we have also not proven that so far" (chapter "Security Analysis of Crosslink 2", section "Proof of safety for $\mathsf{LOG}_{\mathrm{ba}}$"). The bound needs $\sigma \leq L$. The book's requirement that L be "significantly greater than σ", and in practice at least 2σ, implies it, but the book ties the requirement to no bound; for some best-chain protocol of the interface the constraint cannot be dropped.

Proposition 8.5 (Necessity of the constraint $\sigma \leq L$)**.** *For all L and σ with $1 \leq L < \sigma$ there are a best-chain protocol Π_{origbc} with the interface of Definition 3.11 and an execution in which the bc-valid best chain ch_i^t of a node i has a tip that is not stalled and lies more than L blocks above fin_i^t.*

Status: proved (this volume), of a designed-but-unspecified construction.

Proof. Fix $h \geq \sigma$. Let A be a bc-chain with blocks $A_1, \ldots, A_h$, the index being the height, and let a branch fork from it at A_{h-1} with blocks $A'_h, \ldots, A'_{h+\sigma-1}$. Let B be a bft-block with parent $\mathcal{O}_{\mathrm{bft}}$ whose field headers_bc is the σ-block tail of the chain $A'_{h+\sigma-1}$, so that $\mathsf{snapshot}(B) = A_{h-1}$, and let C be a valid bft-block with $\mathsf{bft\text{-}last\text{-}final}(C) = B$ (in a protocol that decides each block individually, $C = B$). The blocks $A_1, \ldots, A_{h-1}$ and those of the branch cite $\mathcal{O}_{\mathrm{bft}}$, each block whose finality depth exceeds L being stalled (Remark 4.26); so the chain $A'_{h+\sigma-1}$ is bc-valid, and B satisfies the Tail Confirmation rule and, since $\mathsf{snapshot}(\mathcal{O}_{\mathrm{bft}}) = \mathcal{O}_{\mathrm{bc}}$, the Linearity rule.

Let the tip A_h cite C and not be stalled, as under the proposed Zcash policy (Remark 4.25) any block with a non-coinbase transaction. The tip satisfies the Valid context rule; the Extension rule, since $\mathsf{LF}(A_{h-1}) = \mathcal{O}_{\mathrm{bft}} \preceq_{\mathrm{bft}} B = \mathsf{LF}(A_h)$; the Last Final Snapshot rule, since $A_{h-1} \preceq_{\mathrm{bc}} A_h$; and the Finality depth rule, since $\mathsf{finality\text{-}depth}(A_h) = 1 \leq L$. Let node i's best chain grow from $\mathcal{O}_{\mathrm{bc}}$ through $A_1, \ldots, A_h$, node i receiving the branch with A_h, and let $\mathsf{score}(A_h) > \mathsf{score}(A'_{h+\sigma-1})$, which Definition 3.11 permits, since it requires the score to increase only along a chain. Every candidate

before A_b is $\mathcal{O}_{\mathrm{bc}}$. By Lemma 4.18, candidate(A_b) is the shorter of A_{b-1} and $A_b\lceil^{\sigma} = A_{b-\sigma}$, which is $A_{b-\sigma}$ since $\sigma > L \geq 1$; update-fin sets $\mathsf{fin}_i^t = A_{b-\sigma}$, and the tip lies $\sigma > L$ blocks above it. □

A script checks the construction for every $1 \leq L < \sigma \leq 20$. The context of the tip carries the headers of a sibling branch, as it must: by Proposition 5.14, a context whose last final block carries headers of ancestors of the tip gives the tip finality depth at least $\sigma + 1 > L$. Every hypothesis of Proposition 8.4 other than $\sigma \leq L$ holds in the construction. Under accumulated work on Mainnet (Remark 3.12) the instance is no best chain of a node that holds the branch: A_b and A'_b have the same ancestors, hence the same target and work, and the branch adds $\sigma - 1 \geq 1$ blocks; on Testnet a minimum-difficulty sibling can carry less work, and the argument fails. The proposition is shown for the interface of Definition 3.11, not for the Zcash instance; whether $\sigma \leq L$ is necessary for Zcash is open.

Remark 8.6 (Finalised chain off the best chain). Proposition 8.4 does not cover a node whose finalised chain is not a prefix of its best chain, $\mathsf{fin}_i^t \not\preceq_{\mathrm{bc}} \mathsf{ch}_i^t$. Under unchanged fork choice, for a node honest at t, that case arises only after a failure of Prefix Consistency at σ: Lemma 5.3 gives a time $r \leq t$ with $\mathsf{fin}_i^t \preceq_{\mathrm{bc}} \mathsf{ch}_i^r\lceil^{\sigma}$, and Prefix Consistency at σ, with $j = i$, gives $\mathsf{ch}_i^r\lceil^{\sigma} \preceq_{\mathrm{bc}} \mathsf{ch}_i^t$. A node that follows update-fin but is not honest, such as an eclipsed one (Definition 3.3), can reach the case while Prefix Consistency holds. The TFL book does not define bounded availability formally, and every theorem about it beyond Propositions 8.4, 8.5 and 8.2, this case included, is open.

Status: open problem.

8.3 Finality overrides

Remark 8.7 (Override argument). The TFL book argues that consensus protocols fail, through a design flaw of the finality layer that causes or permits a stall, a balance-violation or spend-authorisation flaw that is exploited or likely to be, or an implementation bug that makes many nodes diverge from consensus, and that overriding finality may then be the best response (chapter "The Arguments for Bounded Availability and Finality Overrides", sections "Finality Overrides" and "Complementarity"). The override procedure should be formalised, its governance process included, because that makes security analysis of the protocol, of the governance and of their interaction feasible, and settles in advance the procedure stakeholders agree to follow. Modelling an overridden protocol as a new protocol and chain is rejected as insufficient: it does not match how users regard an override and does not help its analysis. Overrides complement Stalled Mode. Validators asked to stop validating cause a finalisation stall; once the stall exceeds the availability bound, the best chain enters Stalled Mode, so that at most L blocks with spends lie above a node's finalised chain fin_i^t (Proposition 8.4, under $\sigma \leq L$ and $\mathsf{fin}_i^t \preceq_{\mathrm{bc}} \mathsf{ch}_i^t$, when stalled blocks carry no spends, as under the coinbase-only policy of Remark 4.25) and a long rollback stays realistic. A coalition of validators that ignores the request removes only the extra time for a governance decision. An intentional rollback should preserve the coinbase-only mining rewards, so that producers keep an incentive to produce blocks during the stall. The argument depends on the freedom of validators to refuse a vote (Remark 4.15).

Status: argued (TFL book).

Remark 8.8 (Overrides outside the construction). Crosslink 2 has no override rule, message or state: an override is a change of the consensus rules, a hard fork outside the construction. Assured Finality (Definition 6.1) binds executions of the protocol only. A reversal out of band, by users who fork the network with a new validity rule, remains possible, and for that reason the TFL book avoids the term "absolute finality" (chapter "Terminology"). The deployed chain has the same boundary: consensus imposes no maximum reorganisation depth, and the requirement that a validator's chain include the activation block of the most recent settled network upgrade is a social checkpoint, distinct from probabilistic finality (*Consensus Guide*, §"Rollback limits, coinbase maturity, and finality").

Remark 8.9 (Open parts of the override design). No source designs a governance process for finality overrides; the TFL book argues only that one should be formalised. Its out-of-date chapter "Potential Changes to Crosslink" recommends that honest best-chain producers record every proposed and notarised bft-block, not only the cited context, which would expose double proposals and double notarisations. Its chapter "Questions About Crosslink" proposes posting objective evidence of a violation of the safety of Π_{bft} or of its assumptions, two final bft-blocks with the same parent or more than one third of a BFT epoch's voting units cast for conflicting blocks, in a best-chain transaction, optionally latching the best chain into Stalled Mode until manual intervention. Neither is part of Crosslink 2.

Status: open problem.

9 The Shielded Labs design

This section states how Shielded Labs' *Zcash Crosslink Design Overview* (draft 10, 14 September 2026, last edited 5 October 2026), the *overview* below, departs from the construction of §4 and §5: its terms and the location of ledger state, the rules it keeps and drops, its certificates and cross-link references, its fork choice, its proposer behaviour, its finality vocabulary, and which results of §6 and §8 survive, each premise read through the design's own validity notion. The overview is cited by section title. Every component this section states is designed but unspecified (Definition 1.1) unless it is marked an open problem; no ZIP specifies any of them.

9.1 Terms and ledger state

The overview departs from the book chiefly in four respects; its further departures are stated in §9.2, §9.3 and Remark 9.10. It has no Stalled Mode, no finalisation gap bound L and no bounded-available chain. It replaces the unchanged fork choice by a sticky fork choice, which never follows a chain that does not contain fin. It adds an Inclusion Depth rule, a lower bound on how far a bc-block lies above the snapshot of the bft-block it cites. It punishes the stake behind finalisers by user-configured slash forks instead of by any in-protocol slashing. It is a living document, and the items it marks as to be confirmed are unsettled.

The following terms of the overview are used in this section and the next two. *Tenderlink* is the design's BFT protocol Π_{origbft}, derived from Tendermint (E. Buchman, J. Kwon and Z. Milosevic, "The latest gossip on BFT consensus", arXiv 1807.04938, 2018); it has no published specification. A *trailing finality certificate*, or *certificate*, is the design's bft-block: it carries the field

headers_bc of Definition 4.2 and has the snapshot of Definition 4.3, but it contains neither its notarisation signatures nor an outer signature. A *fat pointer* is the field of a bc-block that references a certificate by hash together with notarisation signatures; its rule is Definition 9.2. A *lockbox* is a finaliser's reward account. The staking objects are glossed here and defined in §10. A staker holds delegation bonds, each an amount of ZEC, its *stake*, assigned to one finaliser and reassignable to another by redelegation (Definition 10.11). The voting power of a finaliser is the stake delegated to it (Definition 10.4). The *BFT height* of a certificate is its height in the bft-chain, and the *roster* of a BFT height is the set of finalisers with voting powers derived from staking state on the bc-chain, of which the active roster's votes count at that height (Definition 10.7). A *slash fork* is a user-configured hard fork that burns the bonds delegated to named finalisers (Definition 10.27).

All ledger state of the design, the roster included, lives on the bc-chain, and the bft-chain holds none (overview, §"3. The proof-of-work side: the fat pointer and where it lives"). Every payout, burn and slash is computed from what the bc-chain records about the bft-chain, and the fat pointer is the only such record.

9.2 Retained and removed rules

The design keeps, on the certificate side, the Tail Confirmation rule and the Linearity rule of Definition 4.9; on the best-chain side, the Valid context, Extension and Last Final Snapshot rules of Definition 4.17; and update-fin of Definition 5.1 (overview, §"4. The BFT side: TFCs and σ", §"9. Sticky fork choice"). The Parent rule has no counterpart: the parent of a certificate carries no signatures, so judging a certificate does not judge its parent, which is judged only by a bc-block that cites it, if one exists. The Tail Confirmation rule is read as in the book: a validator downloads and validates the bc-blocks under the carried headers, since the rule requires the tail of a bc-valid chain and headers alone do not establish validity.

The design drops the Finality depth rule, Stalled Mode with its predicate is-stalled-block, the parameter L, the bounded-available chain ba_{μ} with its depth μ, and the outer proposer signature of Definition 3.26. The overview's bc-block rules are the Valid context, Extension, Last Final Snapshot and Inclusion Depth rules (§"4. The BFT side: TFCs and σ"), it states that the design has no Stalled Mode (§"9. Sticky fork choice"), and neither its certificate nor its fat pointer carries an outer signature (§"3. The proof-of-work side: the fat pointer and where it lives").

Remark 9.1 (Rationale of the Shielded Labs design). A blog post of J. McGee for Shielded Labs of 29 May 2026 allocates roles: wallets protect users; miners include transactions and produce blocks; finalisers enforce finality; stakers "select good finalizers"; users "slash stakers if they fail to do their job". The overview supplies the mechanisms, redelegation and the slash fork, and draws the consequence for stalls. Redelegation and slash forks are to resolve them, "finality is a ratchet rather than an override" (§"2. The one-paragraph version"), the finality layer's only job is to finalise the proof-of-work chain, and if that layer fails the network "decays gracefully back to plain PoW" (§"1. Purpose"); a stall "can never be proven permanent", so the judgement is left to users, and the threat of a slash fork is to move stake before any burn (§"16. Social slashing"). The book's Stalled Mode instead bounds the run of non-stalled blocks above fin (Proposition 8.4; Remark 8.3).

Status: argued (Shielded Labs).

9.3 Certificates and cross-link references

In the design Π_{origbft} is Tenderlink (overview, §"6. The BFT implementation"), where the book leaves it abstract and gives adapted Streamlet as an example (Example 3.33). Tenderlink's peers agree on each decision, whether a certificate is approved by two thirds of the stake-weighted voting power, but not on the set of signatures that made up that two thirds.

> **Definition 9.2** (Fat pointer)**.** In the Shielded Labs design each bc-block H other than $\mathcal{O}_{\text{bc}}$ carries a *fat pointer*, which is either nil or the hash of a certificate C other than $\mathcal{O}_{\text{bft}}$, which is the commitment H.context_bft of Definition 4.2, together with a set of notarisation signatures attesting C, which lie outside C; every bc-block of height at most h_2 (Definition 10.8) carries a nil fat pointer. The signatures are Ed25519 signatures, a digital signature scheme in the sense of the *Crypto Guide*, §"Syntax and security goal". The block *H cites C*. A block with a nil fat pointer has context $\mathcal{O}_{\text{bft}}$ and cites no certificate, and (i) and the Inclusion Depth rule do not apply to it. Under the design:
>
> (i) the Valid context rule of Definition 4.17 reads: the certificate that H cites satisfies the Linearity and Tail Confirmation rules of Definition 4.9 and Tenderlink's own rules for a proposal, without the Parent rule, and the signatures in the fat pointer of H are valid signatures on it by holders of at least two thirds of the stake of the active roster (Definition 10.6) of the roster at its BFT height;
>
> (ii) a certificate C is *validly cited in honest view* iff some party, at a time at which it is honest, observes a bc-block that cites C and whose fat pointer meets (i); this reading replaces "in honest view" of Definition 3.26 for certificates.
>
> A copy of a certificate is not evidence of canonicity, since the certificate carries no signature attesting it.

The book's bft-block (P, proof_p) under an outer signature is valid or invalid on its own (Remark 4.11). In the design "a certificate cannot carry its own signatures" (overview, §"3. The proof-of-work side: the fat pointer and where it lives"), so validity of a cited certificate is decided from the citing bc-block: two blocks may cite one certificate with different signature sets, and a certificate that no block cites is judged by no block. Where a block stores its fat pointer is an open problem: the overview weighs options from a commitment behind the block commitment hash (*FlyClient Guide*, §"The block commitment hash", Remark "Extension of the block commitment hash"), which it judges probably insufficient, to a change of the block header, and lists the choice among its open questions.

Remark 9.3 (Finality of each decision). Tenderlink decides each certificate individually, so bft-last-final is the identity on decided certificates and $\mathsf{LF}(H)$ is the certificate that H cites: the overview takes LF of a tip to be the certificate at which the tip's context_bft points (§"9. Sticky fork choice"). In the book's interface bft-last-final(C) may instead be a strict ancestor of C, as in adapted Streamlet (Example 3.33).

Remark 9.4 (Deferred validity). The design adds a third state, "not yet determinable", on both sides: a bc-block, proposal or certificate whose referenced data a node does not yet hold is de-

ferred, not rejected, and the deferral must be able to expire or be abandoned, since a malicious peer can reference a hash for which no data exists (overview, §"5. Mutual reference and its consequences"). The state concerns a node's knowledge of referenced data; validity, as a function of a block, proposal or certificate and of the blocks it commits to by hash (Remarks 4.10 and 4.11), stays two-valued. The expiry rule is marked to be confirmed.

Status: open problem.

Lemma 9.5 (Inclusion Depth as a finality-depth bound)**.** *In the Shielded Labs design let a bc-block H cite a certificate C (so $C \neq \mathcal{O}_{\mathrm{bft}}$), so that* $\mathsf{LF}(H) = C$ *(Remark 9.3). The Inclusion Depth rule, that H may cite C only if*

$$\mathsf{height}(H) \geq \mathsf{height}(\mathsf{snapshot}(C)) + \sigma + 1,$$

holds for H if and only if $\mathsf{finality\text{-}depth}(H) \geq \sigma + 1$.

Status: proved (this volume), of a designed-but-unspecified construction.

Proof. By Definition 4.6 and $\mathsf{LF}(H) = C$,

$$\mathsf{finality\text{-}depth}(H) = \mathsf{height}(H) - \mathsf{height}(\mathsf{snapshot}(\mathsf{LF}(H))) = \mathsf{height}(H) - \mathsf{height}(\mathsf{snapshot}(C)),$$

so the two inequalities coincide. □

Under the Last Final Snapshot rule the finality depth of H is the number of blocks of H above its final snapshot. In the book the bound $\mathsf{finality\text{-}depth}(H) \geq \sigma + 1$ holds without a rule whenever $\mathsf{LF}(H)$ is not $\mathcal{O}_{\mathrm{bft}}$ and the headers that $\mathsf{LF}(H)$ carries are ancestors of H (Proposition 5.14). The same holds for a cited certificate C other than $\mathcal{O}_{\mathrm{bft}}$: if the σ headers of C are ancestors of H, the newest is a strict ancestor, because the header of H commits to C, C commits to its headers, and a cycle of hash commitments is excluded (Remark 4.10); the deepest carried header then lies at height at most $\mathsf{height}(H) - \sigma$, and $\mathsf{snapshot}(C)$, its parent, at height at most $\mathsf{height}(H) - \sigma - 1$. Hence, among blocks that cite a certificate other than $\mathcal{O}_{\mathrm{bft}}$, the rule excludes exactly those whose certificate carries a header that is not an ancestor of H and whose final snapshot lies at most σ blocks below H: a decision whose tail lies on another branch. Neither Theorem 6.3 nor Theorem 6.5 uses the rule, nor any lower bound on finality depth.

9.4 Sticky fork choice

The accumulated work of a bc-chain is the sum of the work of its blocks (*Consensus Guide*, §"Work, and the best chain", Definition "Work of a block"), the score of Zcash (Remark 3.12).

Definition 9.6 (Sticky fork choice)**.** A node i follows *sticky fork choice* iff, whenever some bc-valid chain T of its view with $\mathsf{fin}_i^s \preceq_{\mathrm{bc}} T$ has more accumulated work than ch_i^s, or the same accumulated work and a tip hash that wins the tie-break, it replaces ch_i^s by the heaviest such chain, ties broken by tip hash, and it changes its best chain at no other time. The overview states the rule as "best = heaviest chain that contains fin": a heavier chain that forks below fin_i is refused, whatever its work (overview, §"9. Sticky fork choice"). The overview does not state the direction of the tie-break.

The rule departs from Definition 4.22, which chooses among bc-valid chains by score alone, and its tie-break departs from the first-received rule of the protocol specification (*Consensus Guide*, §"Work, and the best chain", Definition "Best valid block chain").

Proposition 9.7 (Absence of finalisation hazards under sticky fork choice)**.** *Let node i follow sticky fork choice and* update-fin*. Then $\mathsf{fin}_i^t \preceq_{\mathrm{bc}} \mathsf{ch}_i^t$ at every time t, and at every change of the best chain of i from ch_i^s to ch_i^t the chains $\mathsf{candidate}(\mathsf{ch}_i^t)$ and fin_i^s agree, so the finalisation safety hazard of* update-fin *is never recorded.*

Status: proved (this volume), of a designed-but-unspecified construction.

Proof. By induction over the sequence of best-chain changes of i, on which update-fin is defined (Definition 5.1). Initially $\mathsf{fin}_i^0 = \mathcal{O}_{\mathrm{bc}} = \mathsf{ch}_i^0$. Between changes neither chain changes. At a change from ch_i^s to ch_i^t, with $\mathsf{fin}_i^s \preceq_{\mathrm{bc}} \mathsf{ch}_i^s$ by the induction hypothesis, Definition 9.6 gives $\mathsf{fin}_i^s \preceq_{\mathrm{bc}} \mathsf{ch}_i^t$. By Definitions 4.5 and 3.8, $\mathsf{candidate}(\mathsf{ch}_i^t) \preceq_{\mathrm{bc}} \mathsf{ch}_i^t\lceil^{\sigma}$, and by Definition 3.5 $\mathsf{ch}_i^t\lceil^{\sigma} \preceq_{\mathrm{bc}} \mathsf{ch}_i^t$. Both fin_i^s and $\mathsf{candidate}(\mathsf{ch}_i^t)$ are thus prefixes of ch_i^t, and they agree by Lemma 3.10; the case of update-fin in which the candidate conflicts with fin_i^s does not arise, and no hazard is recorded. The new value fin_i^t is one of the two, hence a prefix of ch_i^t. □

The overview asserts the first claim without proof (§"10. Kinds of finality and which ledger state is current": local finality "always lies on the best chain"); the proof, and the absence of finalisation hazards, are this volume's.

Remark 9.8 (Consequences of sticky fork choice). The overview states the following consequences (§"9. Sticky fork choice"). The rule departs from most work only when a heavier chain excludes fin; while the BFT protocol is stalled, fin is frozen and selection above it is by work alone. A refused switch is "the only observable sign of a finality conflict". A partition in which only one side can finalise past the fork point leaves that side "permanently unwilling to switch to the other". Without Stalled Mode the gap between fin and the tip is unbounded during a stall, and ordinary spends keep landing above fin. A miner who dominates the best chain can withhold finality progress by never updating context_bft, at no cost in validity.

Status: argued (Shielded Labs).

Remark 9.9 (Status of sticky fork choice). Safety and liveness under sticky fork choice are unproved. The overview states: "The *TFL Book* has no safety or liveness proof for this rule. Its liveness argument relies on unmodified fork choice" (§"9. Sticky fork choice"). No source proves Prefix Consistency, Prefix Agreement (Definitions 3.15 and 3.16) or liveness of the best chain that the rule selects, and the book's arguments for Π_{bc} (Proposition 7.1, Remark 7.4) assume Definition 4.22.

Status: open problem.

9.5 Proposer behaviour

Remark 9.10 (Proposer deviations from the book). The design's proposer departs from Construction 4.13 in two ways, both behaviour and not validity, since a validator cannot tell whether carried headers are the proposer's best-chain tail (overview, §"4. The BFT side: TFCs and σ").

First, it bounds how far the snapshot of a proposal may lie above the snapshot of the last decided certificate; when the bound binds, the proposal carries a tail that ends below the proposer's tip, which still meets the Tail Confirmation rule. The overview calls the bound "a heuristic, not part of Crosslink 2", and its value is not given here. Second, where the book's honest proposer repeats the headers of its parent bft-block, the design's proposer makes no proposal; how often it should repeat instead is not decided. The second deviation removes the proposal on which the book's argument for Proposition 7.2 relies.

The overview is internally inconsistent on σ: its section "4. The BFT side: TFCs and σ" and its Part VI, "Open questions", state different values, the first described as unchecked. No value of σ is given here.

9.6 Finality vocabulary

Definition 9.11 (Shielded Labs finality quantities)**.** The overview distinguishes three quantities (§"10. Kinds of finality and which ledger state is current"), of which the volume keeps two, (i) and (ii); the third, a "database finalized tip", which the overview calls "Not finality", concerns a node's storage, not the protocol, and is omitted. Item (iii) is the quantity from which local finality advances (§"9. Sticky fork choice"):

(i) the *BFT final snapshot* of node i at time t, the snapshot of the newest certificate decided in the view of i: a function of the decided bft-chain, which need not be a prefix of ch_i^t, for any length of time;

(ii) *local finality*, the chain fin_i^t of Definition 5.1, of which the overview states "The node will not reorg past this block";

(iii) for each bc-block H, the chain $\mathsf{candidate}(H)$ of Definition 4.5, a function of H and of the chains it commits to.

Local finality is node-local, since it depends on the history of best chains of i; it is monotone (Lemma 5.2); under sticky fork choice it lies on the best chain (Proposition 9.7). It lies at or below the BFT final snapshot whenever the decided certificates form one bft-chain along which the Linearity rule holds: under Definition 5.1 alone, the chain fin_i^t is $\mathcal{O}_{\mathrm{bc}}$ or a prefix of $\mathsf{snapshot}(C)$ for a decided certificate C (Definitions 5.1 and 4.5), the newest decided certificate C^* extends C, and the Linearity rule along the path from C to C^* gives $\mathsf{snapshot}(C) \preceq_{\mathrm{bc}} \mathsf{snapshot}(C^*)$. The bootstrap's finalisation of the block at height h_1 (Definition 10.8) lies outside update-fin: from height h_2 until a decided certificate's snapshot extends that block, local finality need not lie at or below the BFT final snapshot, which before the first decision is $\mathcal{O}_{\mathrm{bc}}$, strictly below fin_i, if the genesis bft-block points at the best-chain genesis; where the genesis bft-block points is open. The overview requires that "Consensus must never read fin": every quantity that all nodes must compute identically, among them the roster and staking rewards, is derived from chain data. The overview states that honest nodes' values of fin are "only prefix-compatible, not equal"; prefix-compatibility across honest nodes is Assured Finality (Definition 6.1), which under the design rests on the unproved hypotheses of Proposition 9.12.

9.7 Theorems under the Shielded Labs design

The *Shielded Labs rules* below are the kept rules of §9.2, with the Valid context rule and honest view read through Definition 9.2, together with the Inclusion Depth rule and sticky fork choice. An honest node's best chain is bc-valid under these rules, since sticky fork choice chooses among bc-valid chains. The rules leave out the bootstrap of Definition 10.8: under them fin starts at $\mathcal{O}_{\mathrm{bc}}$ and changes only by update-fin. With the bootstrap, Proposition 9.12 needs the further hypothesis that all honest nodes finalise the same block at height h_1, which Proposition 10.9 derives, for the honest nodes whose best chains reach height h_2, from Prefix Agreement of Π_{bc} at depth $h_2 - h_1$; Proposition 9.13 needs $h_2 - h_1 \geq \sigma$; and in Proposition 9.7 the bootstrap value, a prefix of the best chain, is a third possible value of fin.

Proposition 9.12 (Final Agreement route under the Shielded Labs rules)**.** *Under the Shielded Labs rules, suppose that in an execution*

(a) any two certificates validly cited in honest view (Definition 9.2) lie on one bft-chain, that is, agree;

(b) for any two such certificates $C \preceq_{\mathrm{bft}} C'$, every certificate D with $C \prec_{\mathrm{bft}} D \preceq_{\mathrm{bft}} C'$ satisfies the Linearity rule relative to its parent: $\mathsf{snapshot}(D\lceil^{1}_{\mathrm{bft}}) \preceq_{\mathrm{bc}} \mathsf{snapshot}(D)$.

Then Assured Finality (Definition 6.1) holds in the execution.
Status: proved (this volume), of a designed-but-unspecified construction.

Proof. Let i be honest at t and j honest at u. If $\mathsf{fin}_i^t = \mathcal{O}_{\mathrm{bc}}$, it is a prefix of every bc-chain (Definition 3.4) and agrees with fin_j^u; likewise if $\mathsf{fin}_j^u = \mathcal{O}_{\mathrm{bc}}$. Otherwise, by update-fin (Definition 5.1) there is a best-chain change of i at a time $r \leq t$ with $\mathsf{fin}_i^t = \mathsf{candidate}(\mathsf{ch}_i^r)$, namely the last change up to t at which update-fin took its first case. With $H = \mathsf{ch}_i^r$, Definitions 4.5 and 3.8 give $\mathsf{fin}_i^t \preceq_{\mathrm{bc}} \mathsf{snapshot}(\mathsf{LF}(H))$. Since $\mathsf{fin}_i^t \neq \mathcal{O}_{\mathrm{bc}}$, the snapshot is not $\mathcal{O}_{\mathrm{bc}}$, so $\mathsf{LF}(H)$ is not $\mathcal{O}_{\mathrm{bft}}$ and H is not $\mathcal{O}_{\mathrm{bc}}$. Node i is honest at r, so H is bc-valid and its fat pointer meets the Valid context rule as read in Definition 9.2(i); as i observed H at r while honest, the certificate C_i that H cites is validly cited in honest view. By Remark 9.3, $\mathsf{LF}(H) = C_i$, so $\mathsf{fin}_i^t \preceq_{\mathrm{bc}} \mathsf{snapshot}(C_i)$. The same argument for j gives a certificate C_j, validly cited in honest view, with $\mathsf{fin}_j^u \preceq_{\mathrm{bc}} \mathsf{snapshot}(C_j)$.

By (a), C_i and C_j agree; by symmetry let $C_i \preceq_{\mathrm{bft}} C_j$, and let $C_i = D_0, D_1, \ldots, D_k = C_j$ be the certificates of C_j from C_i on, each the parent of the next. By (b), $\mathsf{snapshot}(D_{l-1}) \preceq_{\mathrm{bc}} \mathsf{snapshot}(D_l)$ for $1 \leq l \leq k$, and by transitivity of $\preceq_{\mathrm{bc}}$, $\mathsf{snapshot}(C_i) \preceq_{\mathrm{bc}} \mathsf{snapshot}(C_j)$. Hence fin_i^t and fin_j^u are both prefixes of $\mathsf{snapshot}(C_j)$, and they agree by Lemma 3.10. □

The proof is that of Theorem 6.5 with three substitutions: the Valid context rule and honest view are read through the fat pointer; Final Agreement (Definition 3.31) becomes hypothesis (a), because bft-last-final is the identity on decided certificates; and hypothesis (b) takes the place of the Parent rule's propagation of validity to every ancestor of a valid bft-block, since a certificate that no block cites is judged by no block (Definition 9.2). It uses neither the fork choice, nor Stalled Mode, nor an outer signature, nor the Inclusion Depth rule. Hypotheses (a) and (b) concern Tenderlink with its stake-weighted roster and are unproved (Remark 9.15).

Proposition 9.13 (Prefix Agreement route under the Shielded Labs rules)**.** *Under the Shielded Labs rules, in every execution in which the best chain selected by sticky fork choice has Prefix Agreement at depth σ (Definition 3.16, with ch_i^t that chain), Assured Finality holds.*

Status: proved (this volume), of a designed-but-unspecified construction.

Proof. Lemma 5.3 and its proof use update-fin and Definition 4.5 only, and no property of the fork choice, so they hold with ch_i^t the chain that sticky fork choice selects. Let i be honest at t and j honest at u. The lemma gives $r \le t$ and $r' \le u$ with $\mathsf{fin}_i^t \preceq_{\mathrm{bc}} \mathsf{ch}_i^r\lceil^\sigma$ and $\mathsf{fin}_j^u \preceq_{\mathrm{bc}} \mathsf{ch}_j^{r'}\lceil^\sigma$. Nodes i and j are honest at r and r', so by Prefix Agreement at σ the chains $\mathsf{ch}_i^r\lceil^\sigma$ and $\mathsf{ch}_j^{r'}\lceil^\sigma$ agree; let the first be a prefix of the second, the other case being symmetric. Then fin_i^t and fin_j^u are both prefixes of $\mathsf{ch}_j^{r'}\lceil^\sigma$ and agree by Lemma 3.10. □

No source proves Prefix Agreement of the sticky-choice chain (Remark 9.9); Manian's Quint suite witnesses its failure on a hand-built instance with more than one third of the BFT protocol's voting power subverted (Remark 12.4). In the overview's partition example (Remark 9.8), once both branches extend more than σ blocks above the fork point, Prefix Agreement at σ fails in the execution, as under any fork choice. What the sticky rule adds is that the conflict persists after the partition heals: the nodes of the side that finalised past the fork point never switch to the other branch, and the nodes of the other side, whose fin is at or below the fork point, keep their branch while it is the heavier. Assured Finality can still hold in that example, through Proposition 9.12, since the BFT protocol decided on one side only.

Proposition 9.14 (Unbounded run under the Shielded Labs rules)**.** *Under the Shielded Labs rules, which have no Finality depth rule, the number of blocks above fin_i^t on the best chain ch_i^t of an honest node i, none of which is stalled since the design has no Stalled Mode, is unbounded over executions: for every $d \in \mathbb{N}$ there is an execution in which $\mathsf{height}(\mathsf{ch}_i^t) - \mathsf{height}(\mathsf{fin}_i^t) \ge d$.*

Status: proved (this volume), of a designed-but-unspecified construction.

Proof. Let $H_0 \neq \mathcal{O}_{\mathrm{bc}}$ be a bc-block valid under the Shielded Labs rules, so that it carries a fat pointer (Definition 9.2) and meets the Inclusion Depth rule if it cites a certificate, and let $S = \mathsf{snapshot}(\mathsf{LF}(H_0))$. Consider an execution in which H_0 is the best chain of an honest node i at some time, and then blocks $H_1, \ldots, H_d$ are produced, each H_l a child of H_{l-1} that meets the Inherited origbc rules and repeats the fat pointer of H_{l-1}, as when no new certificate is decided or when producers never advance context_bft; node i adopts each H_l in turn and learns of no other chain. Each H_l is valid: the Valid context rule holds as for H_{l-1}, since the certificate, the signatures and the roster at its BFT height are the same; by Remark 9.3, $\mathsf{LF}(H_l) = \mathsf{LF}(H_{l-1})$, so the Extension rule holds with equality; the Last Final Snapshot rule holds because $S \preceq_{\mathrm{bc}} H_0 \preceq_{\mathrm{bc}} H_l$; and the Inclusion Depth rule holds because the height grows and the snapshot does not change. Each H_l extends the previous best chain, which contains fin_i (Proposition 9.7), and has more accumulated work, so sticky fork choice adopts it.

When H_0 is adopted, $\mathsf{fin}_i \preceq_{\mathrm{bc}} H_0$ (Proposition 9.7), so $\mathsf{height}(\mathsf{fin}_i) \le \mathsf{height}(H_0)$. At each later change, update-fin either keeps fin_i or sets it to $\mathsf{candidate}(H_l)$, which is a prefix of S (Definitions 4.5 and 3.8) and hence of H_0. Thus $\mathsf{height}(\mathsf{fin}_i) \le \mathsf{height}(H_0)$ throughout, and at the adoption of H_d, $\mathsf{height}(H_d) - \mathsf{height}(\mathsf{fin}_i) \ge d$. □

The proposition matches the overview's own statement that the gap between fin and the tip is unbounded during a stall (Remark 9.8); it is the failure, in the design, of Proposition 8.4, whose bound rests on the Finality depth rule.

Remark 9.15 (Final Agreement of Tenderlink). No analysis shows that Tenderlink, with a stake-weighted roster derived from bc-chain state, has Final Agreement, that the certificates it decides are valid, or that its notarisations are sound in the sense of Definition 3.21. Hypotheses (a) and (b) of Proposition 9.12 are therefore open, and with them the premise of the Final Agreement route for the design. A BFT height of Tendermint is decided in one or more rounds, and a validator that has voted to commit a value in one round is locked on it in later rounds of that height until a later round's quorum releases it. The book's argument for Proposition 7.2, that a proposer can always repeat its parent's snapshot, is not reconciled with these locks, and the design's proposer declines to repeat (Remark 9.10).

Status: open problem.

10 The finaliser set

This section states the book's abstract finaliser model and proves the power of a finaliser coalition in it, and then describes the finaliser set as Shielded Labs designs it, in the vocabulary of §9.1: the finalisers and their roster, bootstrap, delegation bonds, staking windows and the privacy of staking, rewards, slashing by user-configured hard forks, finaliser keys, and the mechanism-design review that shaped parts of the reward rule. Each component carries its class (Definition 1.1). The design documents are Shielded Labs' *Zcash Crosslink Design Overview*, draft 10 of 14 September 2026, last edited 5 October 2026 ("the overview" below), cited by section title, and Shielded Labs' *Rewards Distribution* (22 September 2026). Calendar values and bootstrap heights are unsettled, two current texts of the overview differing, and are given without numbers.

10.1 The abstract finaliser model

In the book's model the finaliser set of a BFT epoch is the set of holders of its n voting units (Definition 3.21); notarisation requires $q = \lceil 2n/3 \rceil$ units cast for a proposal (Definition 3.25), and fewer than one third of the units of each BFT epoch are ever cast non-honestly (Assumption 3.23). The model says nothing about how voting units are assigned. Among its "Major Missing Components" the book lists "PoS subprotocol selection" and "Issuance and supply mechanics, such as how much ZEC stakers may earn" (chapter "Status and Next Steps"). Every concrete decision about the finaliser set in the rest of this section therefore comes from Shielded Labs' documents, and each is designed but unspecified.

Remark 10.1 (Delegation goal). The book's chapter "Design Goals" states, as an ideal goal that binds no rule, that users of shielded mobile wallets can delegate ZEC to "PoS consensus providers" and earn a return "via ZEC issuance or fees", that delegation may expose them to a loss of delegated ZEC "(such as through 'slashing fees')", and that the providers, holders of voting units, "have no discretionary control over such delegated funds". The goal fixes no mechanism; the book defines neither delegation nor slashing.

Remark 10.2 (Units and threshold in the two designs). In the book each BFT epoch has a fixed number n of voting units, and the notarisation rule requires at least two thirds of them, in the integer form $q = \lceil 2n/3 \rceil$ (Definition 3.25). In Shielded Labs' design voting power is stake, and a certificate is decided when it is "approved by two thirds of stake-weighted power" of the finalisers whose votes count at its BFT height (overview, §"6. The BFT implementation"; §9.1). Read in the book's model, with the least amount of stake as the unit, the second is an instance of the first in which the units of a BFT epoch are the stake of the active roster at its BFT height; the overview does not state whether two thirds is attained with equality. Both are designed but unspecified.

A set of holders *withholds* in a BFT epoch when none of them casts any of its units of that BFT epoch for any proposal. The honest fin values are the chains fin_i^t of nodes i honest at t (Definition 5.1).

Proposition 10.3 (Power of a finaliser coalition)**.** *In an execution that satisfies Assumption 3.24 and in which notarisation proofs are sound (Definition 3.21), holders of more than one third of a BFT epoch's units can prevent every notarisation of that BFT epoch by withholding. In every execution, whatever the holders of voting units do, they cannot make honest* fin *values conflict while* Π_{bc} *has Prefix Agreement at* σ *(Definition 3.16).*

Status: proved (this volume), of a designed-but-unspecified construction.

Proof. Let the holders hold $c > n/3$ of the n units of the BFT epoch and withhold. By Assumption 3.24 no party other than its holder casts a unit, so for every proposal P of that BFT epoch the units cast for P are among the $n - c$ units of the other holders, and

$$n - c < 2n/3 \le \lceil 2n/3 \rceil = q.$$

No set of ballots for P therefore meets the notarisation rule (Definition 3.25), and by soundness of notarisation proofs (Definition 3.21) no valid notarisation proof proof_P is obtainable, so no bft-block (P, proof_P) is valid (Definition 3.26): no proposal of the BFT epoch is notarised.

If they hold more than one third of the units of every BFT epoch from some BFT epoch e on and withhold in each, every valid bft-block is $\mathcal{O}_{\mathrm{bft}}$ or belongs to a BFT epoch before e. The context of every block H of a valid bc-chain is valid (Valid context rule, Definition 4.17), so $\mathsf{LF}(H)$, a prefix of it, is valid (Definition 3.29) and is such a block. For i honest at t, either $\mathsf{fin}_i^t = \mathcal{O}_{\mathrm{bc}}$ or $\mathsf{fin}_i^t = \mathsf{candidate}(\mathsf{ch}_i^r)$ for some $r \le t$, a prefix of $\mathsf{snapshot}(\mathsf{LF}(\mathsf{ch}_i^r))$ (Definitions 5.1 and 4.5). Hence fin_i^t is a prefix of $\mathsf{snapshot}(B)$ for $B = \mathcal{O}_{\mathrm{bft}}$ or a valid bft-block B of a BFT epoch before e. The abstract model does not exclude that a proposal of such a BFT epoch is notarised after the withholding begins, so this bounds finalisation by the earlier BFT epochs without proving a stall.

The second clause is Corollary 6.4: under Prefix Agreement of Π_{bc} at σ, Assured Finality (Definition 6.1) holds whatever the holders of voting units do. □

10.2 Finalisers and the roster

The definitions of this subsection follow the overview, sections "10. Kinds of finality and which ledger state is current", "11. Finalizers and rosters" and "8. Bootstrap and activation".

Definition 10.4 (Voting power)**.** In the Shielded Labs design the finalisers are the participants of Tenderlink (§9.1); the term widens the meaning of Definition 3.21: the holders of the voting units of a BFT height are the finalisers of its active roster (Definition 10.6), and a finaliser of the inactive roster holds none. The *voting power* of a finaliser at a best-chain state is the stake delegated to it in that state, the total value of the bonds that target it (§9.1). Every fraction of finalisers is a fraction of stake, never of their number, and the notarisation threshold is two thirds of the stake of the active roster, the finalisers whose votes count at a BFT height. The definition is designed but unspecified (overview, §"11. Finalizers and rosters").

Remark 10.5 (Finaliser consent). A finaliser publishes an address that "acts as a capability carrying its own signature consenting to act as a finalizer", and a delegation to it requires that consent (overview, §"11. Finalizers and rosters"). The design has no registration transaction, no minimum self-stake and no web of trust inside the protocol: that a finaliser is run by a given operator is left to attestations outside it. The consent rule is designed but unspecified. No current design fixes a minimum stake for a finaliser.

Status: open problem.

Definition 10.6 (Active roster)**.** Let N be a fixed positive integer, the *active roster size*. At a best-chain state the *active roster* consists of the N finalisers of largest voting power at that state, or of all finalisers when there are at most N, and the *inactive roster* of the others. The split bounds the number of parties in a BFT round; it is neither a reward nor a penalty. The value of N and the selection rule, the order among finalisers of equal voting power included, are open: the overview lists "How the active roster is selected" among its open questions (Part VI). The definition is designed but unspecified (overview, §"11. Finalizers and rosters"), with N and the selection rule open.

Definition 10.7 (Roster derivation)**.** For a BFT height (§9.1) greater than 1, let C be the certificate decided at the preceding BFT height. The *roster* for the BFT height is the set of finalisers with the voting powers that the bonds in the best-chain state at $\mathsf{snapshot}(C)$ determine (Definitions 4.3 and 10.4), together with the active roster of that state (Definition 10.6); only the active roster's votes count. The roster for BFT height 1 is that of Definition 10.8, which this rule also gives when the genesis certificate's snapshot is the block of height h_1. Agreement of the BFT protocol on C fixes $\mathsf{snapshot}(C)$, the final snapshot of the newest decided certificate, which generally lies above fin and may lie on a chain the node does not take as best (Definition 9.11). The roster is required to be a pure function of the best-chain state at that snapshot and of the slashing configuration, the set of slash forks a node applies (§9.1); under a fixed slashing configuration it is frozen while no new certificate is decided, and a slash fork adopted during a stall changes it by jailing the finalisers it names (Definition 10.27). The derivation is designed but unspecified (overview, §"10. Kinds of finality and which ledger state is current", §"11. Finalizers and rosters").

Validators that hold the same certificate at the preceding BFT height and the same slashing configuration derive the same roster. Agreement on that certificate is a property of Tenderlink, which no analysis establishes (Remark 9.15).

Definition 10.8 (Bootstrap)**.** The bft-chain has no genesis received from outside the best chain: every node starts it from best-chain state. A *roster height* h_1 and a later *start height* h_2 of the best chain fix the initial roster and the start of BFT operation. In the numbering of §9.1, BFT height 0 is the genesis certificate, on which no roster votes. The roster for BFT height 1, the first voted height (the overview's BFT height 0), is derived, as in Definition 10.7, from the best-chain state at the block of height h_1. On reaching height h_2 a node finalises the block of its best chain at height h_1 and starts the BFT protocol; every best-chain block of height at most h_2 carries a nil fat pointer, which cites no certificate (Definition 9.2). The difference $h_2 - h_1$ is required to exceed the node's rollback limit, the depth beyond which it does not reverse blocks (*Consensus Guide*, §"Rollback limits, coinbase maturity, and finality"). The heights are unsettled: the two current texts of the overview differ, the second copy adding a third height, the first at which a staking action is valid. Whether the genesis bft-block points at the best-chain genesis or at the block of height h_1 is open (Part VI). The bootstrap is designed but unspecified (overview, §"8. Bootstrap and activation"), with the heights open.

Proposition 10.9 (Bootstrap safety)**.** *If* Π_{bc} *has Prefix Agreement at depth* $h_2 - h_1$ *(Definition 3.16), any two honest nodes whose best chains reach height* h_2 *hold the same block at height* h_1*, finalise it, and, with the same slashing configuration, derive the same initial roster.*

Status: proved (this volume), of a designed-but-unspecified construction.

Proof. Let nodes i and j reach height h_2 at times t and u at which they are honest, their best chains then having tips at height h_2 or above. Prefix Agreement at $h_2 - h_1$ makes $\mathsf{ch}_i^t\lceil^{h_2-h_1}$ and $\mathsf{ch}_j^u\lceil^{h_2-h_1}$ agree; both have tips at height at least h_1, so the shorter is a prefix of the longer and both hold the same block at height h_1. Each node finalises the block of its best chain at height h_1 at that time (Definition 10.8), and the initial roster is a function of the best-chain state at it and of the slashing configuration (Definition 10.7). □

Remark 10.10 (Rollback-limit argument)*.* The overview argues instead from rollback limits (§"8. Bootstrap and activation", "Safety argument"): when a node reaches h_2, its block at height h_1 lies below its rollback limit, "and therefore identical on every chain a node could be following", so every node computes the same roster from it. The argument rests on the rollback limit, which is node policy: consensus imposes no maximum reorganisation depth (*Consensus Guide*, §"Rollback limits, coinbase maturity, and finality"), and the rollback window that ZIP 218 recommends is a SHOULD on implementations (ZIP 218, "Block-count-based constants"). It is the only safety argument the design has for activation, and the overview calls the finality it relies on "reorg-depth finality rather than BFT finality". The clause "identical on every chain a node could be following" is a hypothesis on the best chains of distinct honest nodes, not a consequence of a node-local limit: a limit stops a node from abandoning its own block at height h_1, and does not stop two nodes whose best chains forked below h_1 from holding different ones. The argument thus shows only that a node does not abandon its own block at height h_1; Prefix Agreement of the best chain at depth $h_2 - h_1$ supplies the missing hypothesis (Proposition 10.9).

Status: argued (Shielded Labs).

10.3 Delegation bonds

Definition 10.11 (Bond)**.** A *bond* is created by a transaction that spends shielded funds and carries a staking action, the creation of a delegation bond, which states an amount, required to be a power of ten ZEC, and a target finaliser, named through its consent (Remark 10.5). The amount is quantised at creation only. The bond is a free-floating note, in the overview's term: a ledger object, not a note of a shielded pool, controlled by a signing key derived from the creating action and a salt, so that identical delegations yield distinct bonds. Holding the key is holding the bond, which is decoupled from the funds that created it. Bonds are atomic: no bond is split, and every later action applies to a whole bond. The definition is designed but unspecified (overview, §"12. Delegation bonds").

Definition 10.12 (Staking actions)**.** Besides its creation, a bond admits three staking actions. A *retarget* changes the bond's target finaliser; it may occur at any time, and its transaction records both the old and the new target, so that from the bond state at any time the targets at every earlier time are recovered by replaying the retargets backwards, which slashing requires. An *exit* is an *unbond* followed by a *withdraw*, in two different staking windows, the fixed height ranges to which creation, unbonding and withdrawal are confined. The unbond fixes the bond's numeric value, which depends on the block it lands in; the withdraw must state that value exactly, or it is invalid. The overview gives two reasons for the separation: the amount is an explicit number before the funds re-enter ordinary transactions, and the delay puts stake at risk, since "if an attack is discovered there is time to slash before the funds escape into the shielded pool". The actions are designed but unspecified (overview, §"12. Delegation bonds").

Remark 10.13 (Transaction contents). *Rewards Distribution*, "Staking Mechanics", requires that a transaction acting on bonds contain no inputs or outputs other than its bond actions, actions of the "latest/greatest shielded pool" and the payment of its fee. The document does not name the pool: whether it is the Ironwood pool (*Ironwood Guide*, §"The Orchard protocol and its two pools") or any shielded pool is open. In-protocol liquid staking derivatives are not a goal of the initial design: the book lists them among its out-of-scope goals, which it says "may be goals for future protocol improvements" (chapter "Design Goals"), and the overview defines none. ZIP 248 (Draft), the extensible transaction format, lists hypothetical bundle types "Staking" and "Unstaking (if it can't be combined with the Staking bundle)" with no defining ZIP, and no ZIP that ZIP 259 lists for NU7 provides the transaction format of staking actions or the header change that the cross-link fields require. The restriction is designed but unspecified; the ZIP 248 entries are specified text that defines no bundle.

Remark 10.14 (Open bond questions). Two questions about bonds are undecided. First, whether a withdraw may complete before the unbond it follows is finalised, and whether an exit may complete during a finalisation stall. Second, how a wallet derives bond keys from its seed: no design states a derivation, by ZIP 32 (*Wallet Guide*, §"Hierarchical key derivation (ZIP 32)") or otherwise, so the recovery of bonds from a seed is undesigned, and the overview lists as open the wallet's handling of the calendar: completing a withdraw without a live wallet and scheduling an action from outside a staking window (Part VI).

Status: open problem.

Remark 10.15 (Redelegation as censure). Short of a slash fork, a staker censures a finaliser by retargeting its bonds away from it, at any height, or by exiting them, in staking windows (Definitions 10.12 and 10.16). Shielded Labs' role allocation gives stakers the selection of finalisers (Remark 9.1), and the overview describes the threat of a slash fork as what moves stake, "the burn is the fallback" (§"16. Social slashing", "A big stick"). Two limits bound the means. A retarget or exit changes voting power from the block that contains it, but the votes that count only through a later roster, and, short of a slash fork, the roster changes only through decided certificates (Definition 10.7), so during a stall it is frozen and no redelegation reaches it. A party with more than half of the hash rate can exclude transactions at will (*Consensus Guide*, Remark "Majority is a different problem", in §"Reward-normalised exposure"), redelegations among them, and so disable the means. The two limits are this volume's consequences of Definition 10.7 and the *Consensus Guide*'s remark; the role allocation is Shielded Labs' argument.

Status: argued (Shielded Labs).

10.4 Staking windows and privacy

> **Definition 10.16** (Staking window)**.** A fixed calendar divides the best-chain heights into recurring *staking windows*, the overview's staking days, and the heights between them. A creation, unbond or withdraw takes effect only in a block whose height lies in a staking window (overview, §"13. Staking days"); a retarget takes effect at any height (§"12. Delegation bonds"). The overview gives the windows three roles: batching removes timing information that could link actions to users; the separation of unbond and withdraw into different windows gives the exit delay; and the slash window of a slash fork (Definition 10.27) is measured in staking windows. The period of the calendar and the length of a window are unsettled. The definition is designed but unspecified (overview, §"13. Staking days"), with the calendar open.

Remark 10.17 (Calendar divergence). The two current copies of the overview, its main text and a second copy whose staking-calendar and bootstrap text, revised on 29 September 2026, is newer than the main text's of 22 September, give different periods. *Rewards Distribution* gates a different set of actions: it confines creation and its own withdraw, which takes the place of the overview's unbond, to staking windows, and leaves unconstrained its transfer, which follows a "sufficient delay" that it leaves to be defined and which takes the place of the overview's withdraw (the volume's reading of the two lifecycles). Its text is inconsistent: item 2 of "Staking Mechanics" confines every transaction with bond actions to staking-day heights, while item 3 leaves redelegation and transfer unconstrained. The public *Crosslink FAQ* (modified 30 July 2026) describes an older scheme of fixed periods. The volume gives no calendar value.

Status: open problem.

Remark 10.18 (Privacy design). Public are each finaliser's total stake, since "that total is what voting weight is computed from, so it has to be", and, for each bond, its power-of-ten amount, its target, its retargets with old and new target, its withdrawal value and the staking window of each action. Hidden is the link from a bond to the shielded notes that funded it and to its holder's other

activity: "bonds are anonymous". Two devices serve this: quantisation of amounts, so that bonds cannot be fingerprinted by size, and batching into staking windows (overview, §"12. Delegation bonds", §"13. Staking days"). The exit delay of Definition 10.12 serves security. No design shields the delegation itself, which would hide a bond's amount and target. The two devices are designed but unspecified; shielded delegation is an open problem.

Remark 10.19 (Open privacy and expiry questions). Quantisation protects creation only: once a bond has earned rewards, which are added to it, its value at withdrawal is not a power of ten, and no source discusses the linkage that this value permits. No source couples transaction expiry to the calendar. A transaction carrying a creation, unbond or withdraw is minable only at heights inside a staking window (Definition 10.16) and, if its expiry height h_{exp} is not 0, at no height above h_{exp} (*Wallet Guide*, §"Expiry (ZIP 203)"). If it is not mined before its window ends and h_{exp} lies beyond that end, it cannot be mined until the next window opens although it has not expired, and it expires unmined if no later window opens at a height at most h_{exp}; with $h_{\text{exp}} = 0$ it never expires and waits for the next window (the volume's synthesis).

Status: open problem.

10.5 Rewards

Definition 10.20 (Reward source)**.** Staking rewards are paid from the block subsidy (*Consensus Guide*, §"Scheduled issuance with shorter blocks"), with no issuance beyond what the existing rules provide: in the words of Shielded Labs' *Crosslink FAQ*, "The 21 million coin cap remains unchanged". The distribution of transaction fees is unchanged: "Crosslink does not alter the distribution of transaction fees in any way" (*Rewards Distribution*, "Crosslink Rewards Distribution Rules"). The block subsidy remaining after the share of Zcash's development fund, the overview's post-dev-fund issuance, is divided between miners and staking (*Rewards Distribution* calls the remainder the Operational Consensus Rewards); the part paid for staking is the *staking share*. No source states whether the additional block subsidy by which ZIP 237 (Draft) reissues the reserve of the Network Sustainability Mechanism, the NSM (*Consensus Guide*, §"The NSM reserve and delayed reissuance"), is divided in the same way. The definition is designed but unspecified.

Remark 10.21 (The published split). The published split gives 40 per cent of the block subsidy to miners, 40 per cent to stakers and 20 per cent to Zcash's development fund (*Crosslink FAQ*). The 20 per cent is not a Crosslink share: it is ZIP 214's two funding streams, of 8 and 12 per cent of the block subsidy, which end at the third halving (*Consensus Guide*, §"Scheduled issuance with shorter blocks"). The 40 and 40 per cent of the subsidy are the 50 and 50 per cent of post-development-fund issuance of *Rewards Distribution*, since $0.8 \times 0.5 = 0.4$. Inside the staking share, 90 per cent goes to bonds pro rata and 10 per cent to finalisers as a commission fixed by the protocol; as fractions of the block subsidy these are 36 and 4 per cent. The split is designed but unspecified.

Remark 10.22 (Unsettled payment rule). The two texts disagree. The overview (§"14. Rewards", "Proposed issuance split" and "Proposed issuance rule") divides post-development-fund issuance

into 48 per cent miner subsidy, 48 per cent staking share, divided 90 to 10 as above, and a 4 per cent bounty to the miner that includes a new finality. It pays the staking share and the bounty only if a best-chain block "points at a new certificate" that finalises a best-chain block "close to the tip"; otherwise the 52 per cent is burned through the NSM. The overview measures that distance on the best chain, as the height of the citing block minus the height of the snapshot of the certificate it cites, which is the citing block's finality depth (Definition 4.6; Remark 9.3), and leaves undefined the bound that "close" denotes (Part VI). *Rewards Distribution* pays 50 and 50 per cent, then 90 and 10 per cent, in every block. The texts also disagree on the commission: the overview pays it to active finalisers "uniformly", *Rewards Distribution* in proportion to the total stake weight of each active finaliser.

Status: open problem.

> **Definition 10.23** (Commission and bond earnings)**.** The *commission* is 10 per cent of the staking share, fixed by the protocol and not set by each finaliser. Whether it is divided uniformly among active finalisers or in proportion to their stake, and whether its denominator is the active roster's stake or the total stake, are open (overview, §"14. Rewards"; *Rewards Distribution*, "Crosslink Rewards Distribution Rules"); the text of the *Crosslink FAQ* on "the commission rate charged by their selected finalizer" is stale. A bond earns from the other 90 per cent whether or not its finaliser is in the active roster: "Backing an up-and-coming finalizer costs nothing". A finaliser's commission accrues to its lockbox (§9.1), which a staking action converts into a bond and which counts, in the 90 per cent pool, as a virtual bond delegated to its finaliser. Rewards increase a bond's principal, and no yield is guaranteed: "Rewards come from a fixed pool". Quantisation applies at creation only (Definition 10.11), so this growth does not conflict with it. The definition is designed but unspecified (overview, §"14. Rewards"), with the division and the denominator open.

Remark 10.24 (Conditional rewards). The overview's single-shot rule (Remark 10.22) answers three recommendations of N. Della Penna's mechanism-design review (§1.3; *the review* below): a first-inclusion bounty for miners (R2); an anti-jackpot constraint (R4), in that nothing accrues while finalisation is stalled, each payment being made once, in its block; and, in the overview's words, certificate aging "by another route" (R6): where the review pays a bounty for any certificate that advances finality, optionally weighted by the blocks advanced, the overview withholds rewards from finalisations far behind the tip, to encourage catch-up on a shared prefix. The review's anti-jackpot results, its Theorem 3.3 and Proposition 3.5, belong to its own model (Remark 10.31). Any reward trigger must be objective, computable from chain data: consensus never reads the node-local fin (overview, §"10. Kinds of finality and which ledger state is current"; Definition 9.11). Rounding must be deterministic, every party agreeing exactly on the value paid to each recipient, and its rule is unfinished (overview, §"15. Accounting and the acceleration structure"; *Rewards Distribution*). These constraints are designed but unspecified; the rounding rule is open.

Remark 10.25 (Withheld rewards and the NSM). ZIP 237 (Draft; listed by ZIP 259 among the NU7 consensus changes) defines the NSM Value Balance in its Terminology as funds removed via ZIP 235 or ZIP 233, plus block subsidy and fees left unclaimed by coinbase transactions before NU6. Its recurrence for that balance, in "Changes to the Zcash Protocol Specification", adds

removed(height), "the total value removed from circulation in the block at height by any deployed mechanism (such as ZIP 233 or ZIP 235)". A Crosslink ZIP that removed withheld staking rewards would therefore enter the recurrence through removed(height), with an amendment of the Terminology only. Under NU7 as specified no such mechanism exists: ZIP 259 states that "NU7 does not deploy ZIP 233 or ZIP 234", and a coinbase cannot leave value unclaimed since ZIP 236 (*Consensus Guide*, §"The NSM reserve and delayed reissuance"). These statements about ZIP 237 are specified. The route by which withheld staking rewards would reach the NSM is designed by no text.

Status: open problem.

10.6 Slashing

The book defines no slashing. Its out-of-date chapter "Potential Changes to Crosslink" notes that a stake requirement for proposals "potentially allows parties that make invalid proposals to be slashed", and its chapter "Questions About Crosslink" sketches evidence of a safety violation, two bft-blocks final in some context with the same parent, or more than one third of stake voting for conflicting blocks of one BFT epoch, posted in a transaction to the best chain. Neither is adopted, and the "slashing fees" of the delegation goal (Remark 10.1) have no mechanism.

Remark 10.26 (No in-protocol slashing). The Shielded Labs design has no automatic slashing in the protocol. The stated reason is a property of Tenderlink: peers agree on the decision for a certificate, but different peers may hold different subsets of the votes that made up the two thirds, so "the protocol therefore cannot use 'who voted for what' as evidence for anything — not slashing, not per-vote payouts" (overview, §"6. The BFT implementation", §"11. Finalizers and rosters"). The property supports a narrower claim: the exact signature set is not agreed, so the absence of a finaliser's signature from a fat pointer is no evidence that it did not vote, whereas a signature that is present is attributable to its signer under Assumption 3.24. The absence of in-protocol slashing is designed but unspecified.

> **Definition 10.27** (Slash fork)**.** A *slash fork* is a user-configured hard fork. Node operators set a configuration that names one or more finalisers and a best-chain activation height; every slash is a hard fork, with no default switch. On the best-chain side every bond delegated to a named finaliser within a window reaching back two staking days, that is two staking windows (Definition 10.16), from the activation height is burned, bonds unbonding from it or retargeted away from it within the window included; the retarget records of Definition 10.12 make that set recoverable, and there is no partial slashing and no tunable amount. On the BFT side the named finalisers are jailed: they vote in no current or later round, and their voting power leaves the pool, so that the others can reach two thirds. Each certificate carries a best-chain height, "do not include by", below which no best-chain block may cite it, so that certificates of the reduced set reach the best chain no earlier than the activation height (the exact comparison and encoding are open, Part VI). The height is non-decreasing along the bft-chain and raised only by the first certificate of a forked bft-chain, which embeds the configuration; the forked finalisers thus resume at once while the best chain changes at the one activation height. Votes are cryptographically namespaced by configuration. The configuration obeys hard constraints:

> the roster stays a pure function of the best-chain state at the final snapshot of the BFT protocol and of the configuration (Definition 10.7); staking actions that have landed in the best chain are respected; and application is robust to users, miners and finalisers adopting the configuration at different times. The definition is designed but unspecified (overview, Part V, §"16. Social slashing").

Remark 10.28 (Open slashing details). Several interactions are marked as unsettled: slashing for misbehaviour other than stalls ("Other malicious behaviour"), the recognition of peers on another configuration as well behaved, and certificates produced between the creation of a configuration and its application (overview, §"16. Social slashing"; Part VI). A withdrawn bond cannot be burned. The exit and the slash window are both counted in staking windows: an exit needs two different windows (Definition 10.12), and the slash window reaches back two. A bond that unbonds from a named finaliser in the earlier window of the slash window may therefore withdraw in the later one, below the activation height, whatever the calendar. Such a bond belongs to the set of Definition 10.27 but escapes the burn, and no source states how the burn treats it.

Status: open problem.

10.7 Finaliser keys

Remark 10.29 (Keys and signatures). Finaliser keys, the signatures in fat pointers and bond keys are Ed25519 (overview, §"3. The proof-of-work side: the fat pointer and where it lives", §"12. Delegation bonds"). The signature set of a fat pointer has variable size. A fixed-size signature set for the fat pointer, by aggregate signatures or by a zero-knowledge proof over the signatures, is open, and the post-quantum story for the fat pointer, bond keys and finaliser keys is "not yet considered" (Part VI). In this design Assumption 3.24, on which Proposition 10.3 rests, is a hypothesis on the Ed25519 signatures of finalisers and on the custody of their keys. The use of Ed25519 is designed but unspecified; a fixed-size signature set and post-quantum keys are open problems.

Remark 10.30 (Key rotation). Assumption 3.23 counts every unit ever cast non-honestly, a cast with a compromised key included (Definition 3.22). The book therefore requires that "validator keys of honest nodes must remain secret indefinitely" and that, "whenever a key is rotated, the old key must be securely deleted" (chapter "The Crosslink 2 Construction", section "Model for BFT protocols"), and the book's issue on the point is open. Neither the book nor the overview has a mechanism to rotate or revoke a finaliser key, or a threshold-custody design for finaliser keys.

Status: open problem.

10.8 The mechanism-design review

Remark 10.31 (Scope of the review). The review examined a Shielded Labs design text of 11 December 2025, which predates the overview. Its results are stated in its own abstract model, with proofs of varying completeness: rational agents with a discount factor, a validator set frozen at the last finalised block, a finality function from chains to heights, and certificates valid when their signers hold stake above a fraction $\tau \geq 2/3$ of the total, τ the review's threshold fraction. They are not theorems about a Crosslink rule set, and the volume uses them as framing only. Its Theorem 6.4 ("Fork Choice Dichotomy") concerns a work-preferring and a finality-preferring rule. Sticky

fork choice (Definition 9.6) selects by work among the chains that contain the node-local fin, not among chains extending the highest known finalised block, and does not halt the best chain during a stall (Remark 9.8); it is neither case as stated, and the theorem is not read as constraining it. Its Theorem 2.3 ("Accountable Safety") is quorum intersection, the argument of Lemma 3.27 in units of stake.

Remark 10.32 (Results and failure modes). As framing, the review's results that bear on the finaliser set are these. Conflicting finality requires that stake of at least $2\tau - 1$ times the total have signed conflicting certificates (Theorem 2.3). Without slashing, the largest penalty for a deviation is at most the withheld rewards plus the time value of a delayed exit (Theorem 4.2); the overview's slash forks lie outside that assumption, and short of a slash fork the design is its case. A frozen validator set whose participating stake falls below the threshold during a stall cannot recover without intervention from outside the protocol (Theorem 5.4); the jailing of named finalisers by a slash fork (Definition 10.27), which removes their voting power so that the others can reach two thirds, is an intervention of that kind. If exits can complete during a stall, the cost of corrupting the frozen set decays while its voting weights stay fixed (Proposition 5.8), the first question of Remark 10.14. The review names three failure modes: the "Hostage Holdout", a blocking coalition delaying finality to increase future payouts; the "Zombie Set", a frozen set made permanently inoperable by attrition; and the "Liquid Exit Paradox", exits during a stall that leave the frozen set with "phantom security". Of its recommendations, the overview adopts the first-inclusion bounty (R2) and the anti-jackpot constraint (R4) and states that it meets certificate aging (R6) by another route (Remark 10.24); it does not adopt signer-conditioned commission (R1), since it pays active finalisers without regard to vote sets (Remark 10.26). Stated in the review's model, these results are not statements about Crosslink 2 or the overview's rules, and carry no status word of this volume.

11 The client contract

This section states what a client of a Crosslink 2 node may rely on. It defines the word *final* for a best-chain block and for a transaction, the only definition of the word for these objects in the volume, and proves what finality at an honest node guarantees, how the finality status of a block is decided, and that the finalised chain is valid. It then separates finality from confirmation depth, states which chain an application reads in each design and what a finalisation stall does to it, records what changes for the confirmation policy, anchors and expiry of the *Wallet Guide*, and states why no current design lets a client that checks headers only verify finality. The constructions the section uses are designed but unspecified (Definition 1.1); the deployed facts it cites are specified.

11.1 Finality as a status

Definition 11.1 (Final)**.** A bc-block X is *final at node i at time t* iff $X \preceq_{\mathrm{bc}} \mathsf{fin}_i^t$, where fin_i^t is the locally finalised chain of Definition 5.1. A transaction is *final at node i at time t* iff a block that contains it is final at i at t, that is, iff it is a transaction of a block of fin_i^t.

The notion is the book's assured finality: a final transaction cannot be reverted by the protocol, while a fork made outside the protocol can still reverse it (the book, chapter "Terminology", entries

"Final" and "Assured Finality"); a finality override is such a fork (Remark 8.8). It is designed but unspecified, and it is distinct from four other uses of the word.

- Finality in the context of a bft-block (Definition 3.30) is a relation between two bft-blocks; finality of a bc-block is relative to one node's fin_i^t.
- The final treestate of a block is the output treestate of its last transaction (*Ironwood Guide*, §"Anchors", Definition "Treestate and anchor"), an object and not a status.
- The final chain of the *Wallet Guide*, §"Reorganisation detection and recovery", Theorem "Correctness of synchronisation", is the server's best chain from the time at which it stops changing, a hypothesis on an execution; its Lemma "Expired unmined is final on its chain" (§"Status tracking and terminal states") concludes that an expired transaction lies in no chain that agrees with a given chain up to the expiry height. Neither is a status that a node computes.
- Confirmation by depth gives probabilistic assurance only: no finite confirmation count makes the modelled probability of reversal zero (*Consensus Guide*, §"Confirmation requirements", Proposition "No finite count is final").

Proposition 11.2 (Path to finality)**.** *Let node i be honest at time t and let $X \neq \mathcal{O}_{\mathrm{bc}}$ be a bc-block. Then $X \preceq_{\mathrm{bc}} \mathsf{fin}_i^t$ iff there are a proposal P, bft-blocks B and C and a bc-block H such that the following hold, in order.*

(1) $X \preceq_{\mathrm{bc}} \mathsf{snapshot}(P)$.

(2) $B = (P, \mathsf{proof}_P)$ *is a bft-block formed from P (Definition 3.26).*

(3) B *is final in the context of the valid bft-block C,* $B \preceq_{\mathrm{bft}} \mathsf{bft\text{-}last\text{-}final}(C)$ *(Definition 3.30).*

(4) H *is a valid bc-block whose context has a last final block that extends B,* $B \preceq_{\mathrm{bft}} \mathsf{LF}(H)$ *(Definition 4.4).*

(5) $X \preceq_{\mathrm{bc}} H\lceil^{\sigma}$.

(6) At an update of node i at a time at most t, H becomes its best tip with $\mathsf{fin}_i^s \preceq_{\mathrm{bc}} \mathsf{candidate}(H)$ *for the value* fin_i^s *before the update, so that* update-fin *sets* fin_i *to* $\mathsf{candidate}(H)$ *(Definition 5.1).*

Status: proved (this volume), of a designed-but-unspecified construction.

Proof. Suppose (1) to (6). By the Valid context rule and Definition 3.29, $\mathsf{LF}(H)$ is valid; every block of the bft path from B to $\mathsf{LF}(H)$ is then valid by the Parent rule, and the Linearity rule applied along the path gives $X \preceq_{\mathrm{bc}} \mathsf{snapshot}(B) \preceq_{\mathrm{bc}} \mathsf{snapshot}(\mathsf{LF}(H))$ (Definitions 4.17 and 4.9). With (5), X is a common prefix of $\mathsf{snapshot}(\mathsf{LF}(H))$ and $H\lceil^{\sigma}$, so $X \preceq_{\mathrm{bc}} \mathsf{candidate}(H)$ (Definitions 4.5 and 3.8). By (6) the update sets fin_i to $\mathsf{candidate}(H)$, and Lemma 5.2, node i having followed the update rule up to t (Definition 3.3), gives $X \preceq_{\mathrm{bc}} \mathsf{fin}_i^t$.

Conversely, if $X \preceq_{\mathrm{bc}} \mathsf{fin}_i^t$, then $\mathsf{fin}_i^t \neq \mathcal{O}_{\mathrm{bc}}$, so the last update up to t that changed fin_i set it, in the first case of update-fin, to $\mathsf{candidate}(H)$ for its new best tip H, which is (6). Since $\mathsf{candidate}(H) \neq \mathcal{O}_{\mathrm{bc}}$, $\mathsf{LF}(H) \neq \mathcal{O}_{\mathrm{bft}}$. With $B = \mathsf{LF}(H)$, $C = H.\mathsf{context_bft}$ and P the proposal of B, conditions (2) to (4) hold, H being valid as the best chain of an honest node, and $X \preceq_{\mathrm{bc}} \mathsf{candidate}(H)$ gives (5) and, through $\mathsf{candidate}(H) \preceq_{\mathrm{bc}} \mathsf{snapshot}(\mathsf{LF}(H)) = \mathsf{snapshot}(P)$, also (1). □

The steps are those of Construction 5.13, in which a BFT decision alone changes nothing locally, and Proposition 5.14 gives the distance from the tip to fin_i at such an update. By the Tail Confirmation rule (Definition 4.9), which B satisfies since it is valid by (4), and the paragraph after Definition 4.3, the block whose header is the newest that $P.\mathsf{headers_bc}$ carries lies σ blocks above $\mathsf{snapshot}(P)$, hence at least σ blocks above X. The valid notarisation proof of B requires, under soundness (Definition 3.21), ballots for P authenticated by the holders of at least two thirds of the voting units of P's BFT epoch, an honest holder casting only if $\mathsf{snapshot}(P)$ is a prefix of its own best chain truncated by σ (Construction 4.14). In adapted Streamlet (Example 3.33), condition (3) is a run in C of three adjacent blocks of consecutive BFT epochs whose middle block has B as a prefix; in a protocol that decides each block individually, $C = B$. A candidate that conflicts with fin_i^s leaves fin_i unchanged and is recorded as a finalisation safety hazard.

Proposition 11.3 (Guarantee of finality)**.** *Let a bc-block X be final at node i at time t, with i honest at t. Then X is final at i at every time $u \geq t$ at which i is honest. In every execution in which Π_{bc} has Prefix Agreement at depth σ (Definition 3.16) or Π_{bft} has Final Agreement (Definition 3.31), moreover, X and fin_j^u agree for every time u and every node j honest at u.*

Status: proved (this volume), of a designed-but-unspecified construction.

Proof. A node honest at u has followed the protocol, update-fin included, up to and including u (Definition 3.3). For $u \geq t$, Lemma 5.2 gives $\mathsf{fin}_i^t \preceq_{\mathrm{bc}} \mathsf{fin}_i^u$, so $X \preceq_{\mathrm{bc}} \mathsf{fin}_i^u$ by transitivity of the prefix order. Under either premise, Corollary 6.8 gives Assured Finality (Definition 6.1), so fin_i^t and fin_j^u agree. If $\mathsf{fin}_i^t \preceq_{\mathrm{bc}} \mathsf{fin}_j^u$, then $X \preceq_{\mathrm{bc}} \mathsf{fin}_j^u$. Otherwise $\mathsf{fin}_j^u \preceq_{\mathrm{bc}} \mathsf{fin}_i^t$, and X and fin_j^u are both prefixes of fin_i^t, so they agree by Lemma 3.10. □

Agreement of X with fin_j^u means that X is final at j at u or that fin_j^u is a strict prefix of X: no honest node's finalised chain conflicts with X. The proposition guarantees nothing to a node that is not honest in the model, which includes a node that the adversary has cut off from the rest of the network, for example by an eclipse (Remark 3.17), and nothing in it binds a fork made outside the protocol (Remark 8.8). Its premises are hypotheses on the sub-protocols: Prefix Agreement of Π_{bc} at σ rests on an assumption for Zcash proof of work (Remark 3.18) and on a transfer from Π_{origbc} to Π_{bc} that is unproved (Remark 7.4); Final Agreement of Π_{bft} is a hypothesis on Π_{origbft} carried over by an argued step (Proposition 7.3), and for the BFT protocol of Shielded Labs' design it is open (Remark 9.15).

Proposition 11.4 (Finality status)**.** *Let node i follow* update-fin*, let B be a bc-block and t a time. Exactly one of the following holds:*

(i) final*: $B \preceq_{\mathrm{bc}} \mathsf{fin}_i^t$;*

(ii) not yet final*: $\mathsf{fin}_i^t \prec_{\mathrm{bc}} B$;*

(iii) never final*: B and fin_i^t conflict, and then B is final at i at no time $u \geq t$.*

Status: proved (this volume), of a designed-but-unspecified construction.

Proof. By Definition 3.7, B and fin_i^t either agree or conflict. If they agree, then $B \preceq_{\mathrm{bc}} \mathsf{fin}_i^t$, case (i), or $\mathsf{fin}_i^t \preceq_{\mathrm{bc}} B$ with $\mathsf{fin}_i^t \neq B$, case (ii); the two exclude each other, since $B \preceq_{\mathrm{bc}} \mathsf{fin}_i^t$ and $\mathsf{fin}_i^t \preceq_{\mathrm{bc}} B$ together give $B = \mathsf{fin}_i^t$. Otherwise they conflict, case (iii), which excludes both. In case (iii), suppose $B \preceq_{\mathrm{bc}} \mathsf{fin}_i^u$ for some $u \geq t$. Lemma 5.2 gives $\mathsf{fin}_i^t \preceq_{\mathrm{bc}} \mathsf{fin}_i^u$, so B and fin_i^t are both prefixes of fin_i^u and agree by Lemma 3.10, a contradiction. □

The status is decided by the prefix relation, not by height. A block of height greater than $\mathsf{height}(\mathsf{fin}_i^t)$ is in case (ii) only if fin_i^t is its ancestor; a block on a branch that forks from fin_i^t below its tip is in case (iii) whatever its height, while a classification by height alone would place it in case (ii) if its height exceeds $\mathsf{height}(\mathsf{fin}_i^t)$ and in case (i) otherwise. A block of height at most $\mathsf{height}(\mathsf{fin}_i^t)$ is in case (i) iff it is the block of fin_i^t at its height, and in case (iii) otherwise. Case (ii) promises nothing: a later update may move fin_i to a chain that conflicts with B. A transaction contained in B has, relative to B, the status of B; a transaction contained in no block has none of the three statuses and is not final.

Proposition 11.5 (Validity of the finalised chain)**.** *For node i honest at t, the chain fin_i^t is bc-valid (Definition 4.17), and every transaction of fin_i^t, its anchors included, satisfies the transaction consensus rules in the context of its own chain, the prefix of fin_i^t that its block tips.*

Status: proved (this volume), of a designed-but-unspecified construction.

Proof. Lemma 5.3 gives a time $r \leq t$ with $\mathsf{fin}_i^t \preceq_{\mathrm{bc}} \mathsf{ch}_i^r\lceil^\sigma \preceq_{\mathrm{bc}} \mathsf{ch}_i^r$. Node i is honest at r, having followed the protocol up to $t \geq r$ (Definition 3.3), so its best chain ch_i^r is a bc-valid chain, a chain of bc-valid blocks (Definitions 3.11 and 4.17), and every prefix of ch_i^r, fin_i^t among them, is a bc-valid chain. Validity in Π_{bc} includes the Inherited origbc rules, under which the transaction consensus rules are those of Π_{origbc} unchanged, with no reordering or sanitisation and heights preserved (Definition 3.11). Each transaction of fin_i^t therefore satisfies them in the context of the ancestry of its block, which is the same in fin_i^t and in ch_i^r. Among them is the anchor rule: an anchor of a bundle equals the anchor, in its pool, of a block that precedes the transaction's block (*Ironwood Guide*, §"Anchors"), and that block is a block of fin_i^t. □

Since bc-validity is a function of a block and of the blocks it commits to by hash (Remark 4.10), never of a chain that contains it, the conclusion holds of every block of fin_i^t on every chain that extends fin_i^t. Snap-and-chat's finalised ledger is a sanitised sequence of transactions, whose two readings coincide when double spends and missing inputs are the only contextual invalidities and can differ in Zcash, where a transaction invalid for a missing anchor can become valid at a later position (Remark 2.17, item (3)).

Remark 11.6 (Local finality and global consistency). The finality status of a block is a function of one node's fin_i^t, and its value at one node is not evidence of global consistency. Two honest nodes, or one node at two times, can hold finalised chains of different lengths. The values of one honest node at two times agree in every execution (Proposition 6.2). Under the premises of Corollary 6.8

the values of any two honest nodes agree, one a prefix of the other (Definition 6.1); when both premises fail, no result of this volume excludes conflicting values at two distinct honest nodes, and a node that observes a candidate conflicting with its finalised chain records a finalisation safety hazard (Definition 5.1). Shielded Labs' *Zcash Crosslink Design Overview* (draft 10) states the same of its design: honest nodes reach one chain through different histories, so their values of fin are only prefix-compatible, not equal, and consensus never reads fin (the overview, §"10. Kinds of finality and which ledger state is current"; Definition 9.11).

11.2 Finality and confirmation depth

Remark 11.7 (Confirmation depth under Crosslink). The deployed facts are specified. No finite confirmation count is final: in the model of the *Consensus Guide*, for every attacker share of the hash rate that is positive and below the honest share, the probability that the attacker reverses a transaction is positive at every number of confirmations (§"Confirmation requirements", Proposition "No finite count is final"). Consensus imposes no maximum reorganisation depth; a node may limit how far it rolls back, and such a limit is node policy, under which different nodes may stop on different sides of a long split, not global agreement (§"Rollback limits, coinbase maturity, and finality", Remark "Effect of a rollback limit"). This section uses no rollback window. The *Consensus Guide* leaves to the layers above it the policy of when to treat a transaction as final (§"Layer boundary"). Under Crosslink 2, fin_i^t is a separate state, computed by Construction 5.13 from the BFT decisions that the best chain cites, while the best chain and its confirmation counts remain those of the best-chain protocol. A block deep in ch_i^t that is not a prefix of fin_i^t is not final, whatever its number of confirmations, and a block of fin_i^t is final whatever its depth. A displayed count of confirmations remains proof-of-work depth, as the overview also states of its design (§"10. Kinds of finality and which ledger state is current").

Corollary 11.8 (Confirmations of a newly final block)**.** *Let an update of node i set* fin_i *to* $\mathsf{candidate}(H)$ *for its new best tip H, where* $\mathsf{LF}(H) \neq \mathcal{O}_{\mathrm{bft}}$ *and the headers carried by* $\mathsf{LF}(H)$ *are ancestors of H. Every block that becomes final at i at that update has at least $\sigma + 2$ confirmations on the best chain H at that time.*

Status: proved (this volume), of a designed-but-unspecified construction.

Proof. A block X that becomes final at the update satisfies $X \preceq_{\mathrm{bc}} \mathsf{candidate}(H)$ (Definition 11.1). Under the hypotheses, Proposition 5.14 gives $\mathsf{candidate}(H) \preceq_{\mathrm{bc}} H\lceil^{\sigma} \preceq_{\mathrm{bc}} H$ and $\mathsf{height}(H) - \mathsf{height}(\mathsf{candidate}(H)) \geq \sigma + 1$, so X lies on H at depth $\mathsf{height}(H) - \mathsf{height}(X) \geq \sigma + 1$ (Definitions 3.4 and 3.5). In the inclusive count of Remark 3.6, a block at depth d has $d + 1$ confirmations, so X has at least $\sigma + 2$. □

The bound is attained by $\mathsf{candidate}(H)$ itself when the attainment condition of Proposition 5.14 holds and $\mathsf{candidate}(H)$ was not final at i before the update. The corollary is the only derivation of the count in the volume, and no argument rests on a value of σ (Definition 4.1).

Remark 11.9 (Terminal states). In the *Wallet Guide*, a mined transaction is terminal only relative to the wallet's rollback window, since consensus bounds no reorganisation depth (§"Status tracking and terminal states", Definition "Terminal states"; §"Reorganisation detection and recovery",

Remark "The rollback window"); the absence of a bound is specified, and the states are designed but unspecified there. Finality would strengthen this status: by Proposition 11.3, a transaction of a block final at an honest node stays final there whatever the depth of a later reorganisation of its best chain, and under the premises of that proposition no honest node's finalised chain conflicts with the block. No specification defines a terminal state by finality. The *Wallet Guide*'s states are taken on the server's best chain, which under unchanged fork choice need not contain fin_i^t (Remark 8.6).

11.3 Ledger views and stalls

Definition 11.10 (Application views)**.** An *application view* of node i at time t is one of the two chains fin_i^t (Definition 5.1) and $(\mathsf{ba}_\mu)_i^t$ (Definition 5.7), for the node's bounded-available depth μ (Definition 5.6). Either view is to be exposed only once the node is synced in the sense of Remark 5.12, a recommendation and not a consensus rule. The best chain ch_i^t is not an application view.

The definition is the book's (chapter "The Crosslink 2 Construction", sections "Stalled Mode", "Locally bounded-available chain" and "Syncing and checkpoints"), and it is designed but unspecified. By Proposition 5.8, at a node i honest at t, $\mathsf{fin}_i^t \preceq_{\mathrm{bc}} (\mathsf{ba}_\mu)_i^t$, so the second view extends the first. In the book, for $\mu = \sigma$, the main factor in an application's choice between them is not latency but its desired behaviour under a finalisation stall: "stall immediately, or keep processing user transactions until L blocks have passed" (section "Stalled Mode"); Remark 11.12 states the second behaviour precisely. The checkpoint and the freshness threshold of the syncing gate are open (Remark 5.12).

Remark 11.11 (Application views in the Shielded Labs design)*.* Shielded Labs' design has no Stalled Mode, no bound L and no bounded-available chain (§9.2). An application there either stops with fin_i^t during a finalisation stall or follows the best chain, which under sticky fork choice contains fin_i^t (Proposition 9.7), with no bound on how far it runs past fin_i^t (Proposition 9.14). The design offers an application no ledger view between the best tip and fin_i^t, although the BFT final snapshot of Definition 9.11 can lie on the best chain above fin_i^t; a displayed confirmation count is proof-of-work depth, never finality (the overview, §"10. Kinds of finality and which ledger state is current"). These views are designed but unspecified.

Remark 11.12 (Behaviour during a stall)*.* In the book, the run of non-stalled blocks above the finalised chain is bounded: under the hypotheses of Proposition 8.4, which include $\sigma \le L$ and $\mathsf{fin}_i^t \preceq_{\mathrm{bc}} \mathsf{ch}_i^t$, every block of ch_i^t at height greater than $\mathsf{height}(\mathsf{fin}_i^t) + L$ is stalled. Under the proposed Zcash policy, a stalled block is coinbase-only (Remark 4.25), so no ordinary spend enters a block more than L above fin_i^t, and the user transactions that $(\mathsf{ba}_\mu)_i^t$ carries above fin_i^t lie in blocks at most L above it. The case in which fin_i^t is not on the best chain is not covered (Remark 8.6). In Shielded Labs' design, the number of non-stalled blocks above fin on an honest node's best chain is unbounded over executions (Proposition 9.14), and ordinary spends keep landing above fin during a stall (the overview, §"9. Sticky fork choice"). Each behaviour is designed but unspecified in its design; which behaviour a specification adopts is an open problem.

Remark 11.13 (Design goals). The book's design goals bind nothing: it calls them ideal goals, some of which trade-offs may preclude (the book, chapter "Design Goals"). Three bear on clients: that users can rely on assured finality with an expected time to finality below 30 minutes; that the cost of attack for a one-hour rollback is not reduced; and that the cost of attack to halt the chain exceeds the 24-hour revenue of proof-of-work mining rewards. With no value of σ or L chosen (Definition 4.1), the time goal cannot be checked: under the hypotheses of Proposition 5.14, fin_i trails the tip by at least $\sigma + 1$ blocks at each update that sets fin_i to $\mathsf{candidate}(H)$, the latency of Π_{bft} adding to it, and no validated latency bound exists (Remark 5.15). A time goal fixes a block count only through the block target spacing, which NU7 changes (§1.1).

11.4 Confirmation policy, anchors and expiry

Remark 11.14 (Confirmation policy under finality). A confirmation policy is a triple $(c_{\mathrm{tr}}, c_{\mathrm{untr}}, z)$ of integers $1 \le c_{\mathrm{tr}} \le c_{\mathrm{untr}}$, the trusted and the untrusted confirmation depth, and a flag z that permits zero-confirmation shielding (*Wallet Guide*, §"The confirmation policy", Definition "Confirmation policy"). ZIP 315 recommends $c_{\mathrm{tr}} = 3$ and $c_{\mathrm{untr}} = 10$ ("Trusted and untrusted TXOs") and permits, without recommending, $z = \mathsf{true}$ ("Transparent UTXOs"); it says that wallets SHOULD NOT permit the spending of outputs with fewer than 3 confirmations, with exceptions for certain transparent outputs, among them those spent in shielding transactions ("Trusted and untrusted TXOs"). The floor that ZIP 315 sets is therefore 3 confirmations; the bound $1 \le c_{\mathrm{tr}}$ is a constraint of the definition. These rules are specified. ZIP 315 does not mention finality, and no source maps the policy onto the statuses of Proposition 11.4: a wallet could spend only final outputs, could combine a depth with the three statuses, or could keep its depth policy and display finality separately. Under the hypotheses of Corollary 11.8, an output of a block that becomes final has at least $\sigma + 2$ confirmations at that time. The choice of a policy under finality is an open problem.

Remark 11.15 (Anchors under finality). Transaction validity is unchanged by Crosslink 2 (Definition 3.11), so the anchor rule is inherited: the anchor of a bundle must be the anchor, in its pool, of a block that precedes the transaction's block in the chain, with no bound on how far back that block lies (*Ironwood Guide*, §"Anchors"), and the rule is evaluated against the ancestry of the block that contains the transaction. The recommended anchor depth is a recommendation to the builder of a transaction, not a consensus rule (*Ironwood Guide*, §"Anchors", Remark "Recommended anchor depth"; *Wallet Guide*, §"Target and anchor heights"). These rules are specified. No source restricts anchors to the finalised chain. The anchors of the transactions of fin_i^t at an honest node are valid in their own chains by Proposition 11.5.

Remark 11.16 (Anchor policy relative to fin). Whether consensus should restrict anchors relative to finality, and whether a wallet's default anchor should lie in fin_i^t, is an open problem; the trade-off below is this volume's analysis. A consensus rule can refer only to objective data of a block, such as its final snapshot $\mathsf{snapshot}(\mathsf{LF}(H))$ (Definition 4.4), since fin_i^t is node state that consensus does not read (Remark 11.6). An anchor at a block $X \preceq_{\mathrm{bc}} \mathsf{fin}_i^t$ satisfies the anchor rule in every chain that extends X and contains the transaction in a later block, so no reorganisation that keeps X invalidates the anchor, and X stays final at the node by Proposition 11.3. Against this, at each update that sets fin_i to $\mathsf{candidate}(H)$ under the hypotheses of Proposition 5.14, the distance from the tip H to fin_i is at least $\sigma + 1$ blocks and equals $\mathsf{finality\text{-}depth}(H)$, which varies from block to

block; an anchor in fin_i^t is therefore deeper than the recommended anchor whenever $\sigma + 1$ exceeds the recommended depth, and it has no fixed depth, unlike both conventions of the *Wallet Guide*, §"Target and anchor heights", Remark "The anchor convention". A note mined above X is not a leaf of the tree that the anchor of X roots (*Ironwood Guide*, §"Anchors", Remark "Old anchors"), so it cannot be spent against that anchor; the candidate set of the anchor is no larger than that of a later anchor; and an anchor at a depth that other builders do not use distinguishes its builder (*Ironwood Guide*, §"Anchors", Remark "Anchor choice as public data"). Since the finalised chains of honest nodes can differ in length, wallets that anchor at the finalised chains of different nodes can choose different anchors.

Remark 11.17 (Expiry under finality). Transaction expiry is a consensus rule: a non-coinbase transaction with expiry height $h_{\text{exp}} \neq 0$ must not be mined at a height greater than h_{exp}, and $h_{\text{exp}} = 0$ disables expiry (*Wallet Guide*, §"Expiry (ZIP 203)", Definition "Transaction expiry"). The default expiry delta at NU7 is that of the same section read together with ZIP 218, "Default expiry delta"; no argument of this volume rests on its value. The rule is specified and unchanged by Crosslink 2. Lemma 11.18 below is the analogue, for the finalised chain, of the *Wallet Guide*'s Lemma "Expired unmined is final on its chain". During Stalled Mode the best chain still grows, by stalled blocks, so an unmined transaction still expires by height; under the coinbase-only policy of Remark 4.25 no stalled block includes it, so a transaction whose expiry height is reached while every new block is stalled expires unmined. No source relates the expiry delta to L; the coupling is an open problem.

Lemma 11.18 (Expired unmined below fin)**.** *Let node i be honest at t and let a non-coinbase transaction have $h_{\text{exp}} \neq 0$. If $\mathsf{height}(\mathsf{fin}_i^t) \geq h_{\text{exp}}$ and no block of fin_i^t contains the transaction, then no bc-valid chain that extends fin_i^t contains it, and neither does fin_i^u for any time $u \geq t$ at which i is honest.*

Status: proved (this volume), of a designed-but-unspecified construction.

Proof. The blocks of a chain that extends fin_i^t at heights at most h_{exp} are those of fin_i^t, and the expiry rule, a consensus rule, excludes the transaction from every higher block of a bc-valid chain. For $u \geq t$ at which i is honest, Lemma 5.2 gives $\mathsf{fin}_i^t \preceq_{\text{bc}} \mathsf{fin}_i^u$ and Proposition 11.5 makes fin_i^u bc-valid, so fin_i^u is such a chain. □

Remark 11.19 (Downstream services). For a service that credits deposits, three questions are open problems: the meaning of a deposit received during a finalisation stall, when fin_i does not advance and, in the book, ordinary spends stop at a bounded distance above it; the treatment of a transaction already credited whose block falls into case (iii) of Proposition 11.4 at the service's node, and so can no longer become final there; and the protocol by which a client that runs no node learns finality status. A service that credits by finality status relies on the guarantee of Proposition 11.3 instead of a delay of its own choosing; no design states the obligations of such a service.

11.5 Light clients

Remark 11.20 (Commitment hook). Under ZIP 244, "Block Header Changes", the block commitment hash is a linked list of hashes terminated by arbitrary data, and a later network upgrade

may replace its all-zero terminator by the hash of a new structure; no upgrade through NU7 does (*FlyClient Guide*, §"The block commitment hash", Remark "Extension of the block commitment hash"). This is the specified place in the header at which a commitment to Crosslink data could enter. No ZIP or specification section specifies a consumer of the chain-history tree to which the header commits, a proof object or a verifier (*FlyClient Guide*, §"The specified commitment and its consumers"). These facts are specified.

Remark 11.21 (The light-client service). The light-client service of the *Wallet Guide* extracts its data, compact blocks included, from the node's view of the best chain, and carries no finality status (§"Architecture and authority", Definition "Light-client architecture"). Its Assumption "Honest-but-curious server" concerns answers about the current best chain and has no clause for finality status, and no light-client protocol for finality status is specified. A light wallet that learns the finality status of a block from a server therefore relies on the server for a status that the assumption does not cover. These facts concern specified objects.

Remark 11.22 (Verification of finality by light clients). A *header-chain client* is a light client that verifies block headers only, as the simplified-payment-verification client of the *FlyClient Guide*, §"Header-only verification", Definition "Simplified payment verification". Verification of finality by such a client is an open problem: no current design places in the header chain both the notarisation signatures and a commitment to the set of finalisers against which they are counted. In the book, each bc-header carries context_bft, a commitment to a bft-block (Definition 4.2), so a header-chain client sees the cross-link reference; validity of the referenced block depends on its notarisation proof, to which the header commits by hash through context_bft but which a header-chain client does not hold, and on the holders of the voting units of its BFT epoch (Definitions 3.21 and 3.26), whose determination the book does not fix and to which no header carries a commitment of its own. In Shielded Labs' design, the roster for a BFT height is derived from the bonds in the best-chain state at a snapshot and from the node's slashing configuration (Definition 10.7); no header or certificate carries a commitment to the roster itself, and a header-chain client, which holds no transactions, cannot evaluate that derivation, so it cannot check the stake weight behind a notarisation. The overview judges its storage option "commit only", which uses the hook of Remark 11.20, probably insufficient (§"3. The proof-of-work side: the fat pointer and where it lives"), and lists the choice of placement as open (Remark 4.8). Since a certificate does not contain its notarisation signatures, which lie in the fat pointer of a citing block, a copy of a certificate is not evidence of canonicity (Definition 9.2), which bears on every light-client design for that construction.

12 Formal models

This section states what each of the four formal models of Crosslink covers. The status word "machine-checked fragment" is that of Definition 1.1, which extends the *FlyClient Guide*'s Definition "Epistemic classes" (*FlyClient Guide*, §"Scope and status"); it records that a tool has checked a stated fragment, and claims nothing beyond that fragment. Two models are written in Quint, a specification language whose specifications translate into Lamport's specification language TLA+, so that the model checkers TLC, which enumerates the states of a finite instance,

and Apalache, which checks executions up to a bounded number of steps symbolically, apply to them. The third is written for Lean, a proof assistant that checks proofs of stated theorems. The fourth is written in TLA+ directly.

12.1 The Lean formalisation

Remark 12.1 (Lean formalisation). D.-E. Hopwood, "Add work-in-progress Lean formalization", an unmerged pull request to the TFL book (last commit 9 March 2026), states in Lean Theorem 6.3, with an honesty predicate that need not hold at earlier times, and a form of Theorem 6.5 whose last-final function is a placeholder, the identity, so that its premise is Agreement on the bft-chains and nothing ties those chains to the bc-chains. Both proofs are left unfinished, and both statements hold vacuously, because the parameter structure over which they are stated includes the hypothesis on tips stated below, which nothing satisfies. Lean accepts a proof left unfinished with a warning, so a successful build does not show that every stated theorem is proved. Machine-checked, per the pull request's continuous integration only and not rebuilt for this volume, is the bc-chain case of Lemma 3.10. The lemmas on the last common ancestor of bc-chains, that $\mathsf{lca}(A, B)$ is a prefix of both chains and that every common prefix is a prefix of it (Definition 3.8), and $\mathsf{candidate}(H) \preceq_{\mathrm{bc}} H\lceil^{\sigma}$, the first clause of Proposition 5.14, are checked under the hypothesis that two bc-chains with one tip are equal. The formalisation represents a bc-chain as a list of block hashes, and two different lists can share a tip, so that hypothesis fails and these lemmas hold vacuously. The last is moreover stated for an arbitrary LF, whose definition is left unfinished. Lemma 5.3 and both theorems are unproved in the formalisation. Its parameter structure encodes $L \geq 2\sigma$, which the book gives as advice for practice, as a hard constraint.

Status: machine-checked fragment.

12.2 The Informal Systems Quint model

Remark 12.2 (Informal Systems model). Informal Systems' Quint model of Crosslink (23 July 2025) is cited by author, kind and date, its title naming a function of the model. It models the proposal-validity check of Π_{bft} alone, for one node, with no adversary, voting, signatures or network. Its invariant states that the snapshots of consecutive accepted bft-blocks are ancestor-linked; its description reports that random simulation finds no violation and that TLC checks the invariant on an instance whose bc-block and bft-block stores hold up to eight blocks each, at confirmation depth one; neither run was repeated for this volume. The invariant follows in one step from the model's own linearity guard, since its store of bc-blocks only grows. That guard forbids the equal snapshot that the Linearity rule of Definition 4.9 permits and that the book's argument for Proposition 7.2 needs, and the model's tail is an unordered set of at least σ blocks rather than a sequence of exactly σ headers (Definition 4.2).

Status: machine-checked fragment.

12.3 The Manian Quint suite

Remark 12.3 (Manian suite). Z. Manian's Quint suite for Crosslink (18 May to 4 June 2026) is cited by author, kind and date, its only title being the name of its repository. Per its continuous

integration it runs bounded Apalache checks and TLC checks, on small instances and at bounded depth, of models in which every finaliser has equal voting power: a round machine shaped after Tendermint with a fixed roster, and separate models whose roster changes across BFT heights. "The induction itself is not closed by Apalache". It adds witness runs on hand-built fixtures. Its models aligned with the book are only witness-tested. The best-chain rules of Definition 4.17 and Stalled Mode (Definition 4.24) are absent from its core models, so the suite is not a check of the construction.

Status: machine-checked fragment.

Remark 12.4 (Witnessed findings). The suite witnesses the following on hand-built fixtures; the witnesses are instances, not general proofs.

(1) Under a partition, with more than one third of the BFT protocol's voting power subverted, a fork choice that rejects every chain conflicting with the node's finalised chain, whatever its work, as sticky fork choice does (Definition 9.6), turns a transient failure of Final Agreement into a split of the best chain that persists to the end of the witness run, after the partition heals and after a further block on the heavier branch; permanence is the model's argument that the rule ignores work, not a checked property. The unchanged fork choice of Definition 4.22 reconverges. The model records no hazard; under Definition 5.1 the node whose finalised block lies on the abandoned branch would record a finalisation safety hazard. When only one side finalises legitimately the comparison reverses: the rejecting fork choice keeps the finalised branch, and the unchanged one may abandon it.

(2) Verification of each certificate against the roster in force at its BFT height is well-founded, and dropping that check allows a takeover of the roster in one step.

(3) Leaked keys of former roster members forge an early certificate for a node that syncs without a checkpoint, and a checkpoint confines the forgery (Remark 5.12).

(4) Results for rosters of unit voting power transfer to stake-weighted thresholds only if the fault bound is read over stake: an instance of four finalisers whose one faulty finaliser, within the fault bound by count, holds more than one third of the stake yields conflicting certificates.

Status: machine-checked fragment.

12.4 The Garcia TLA+ specification

Remark 12.5 (Garcia specification). A. Garcia's "Crosslink2 TLA+ Specification" (18 April 2025 to 7 April 2026), which Informal Systems' model cites as a proposal for verification work, describes itself as a proof of concept. Its finalised chain is the longest bc-chain of the model, truncated, computed with no bft-chain, no context and no update rule update-fin, and its last-final function is the identity on a node's bft-chain. It submits invariants to TLC, one of them named Assured Finality but stated over that chain, and reports no completed run; it is therefore no machine-checked fragment.

12.5 Coverage of the formal models

Remark 12.6 (Scope of machine checking). No machine-checked artefact covers the book's Assured Finality (Definition 6.1) or either of its theorems, nor the ebb-and-flow paper's properties P1 and P2 and its prefix condition (Definition 2.7). Four formal models exist: the Lean formalisation (Remark 12.1), Informal Systems' model (Remark 12.2), Manian's suite (Remark 12.3) and Garcia's specification (Remark 12.5). The proofs of this volume are on paper; the scripts whose output the text quotes check their statements on random instances, by exhaustive enumeration over small parameter ranges, and on constructed counterexamples.

13 Status and open problems

This section states the components that a specification of Crosslink 2 for Zcash must fix and the one open problem that no earlier section states, activation on an existing chain. It then collects the status of the volume in three tables: every open problem, with the environment that states it and its source (Table 2); every goal of the construction, with the result that discharges it and the open problems on which its standing for Zcash depends (Table 3); and every rule, with the party that checks or follows it and the results that use it (Table 4). The tables cite environments by number and restate no statement.

13.1 Components a specification must fix

Remark 13.1 (Components left unfixed). A specification of Crosslink 2 for Zcash must fix each of the following components; no source fixes any of them.

(1) The concrete BFT protocol Π_{origbft}, the assignment of voting units to its BFT epochs and the format of its notarisation proofs (Definitions 3.19 and 3.21). The book uses adapted Streamlet as an example (Example 3.33); the Shielded Labs design uses Tenderlink (§9.1), which has no specification.

(2) The values of σ and L (Definition 4.1), with $\sigma + 1 \leq L$: $\sigma \leq L$ is needed, for some best-chain protocol of the interface, for the bounded non-stalled run (Proposition 8.5), and $L \geq \sigma + 1$ for Stalled Mode to end while Π_{bft} is live (Proposition 7.7); the depth μ is chosen by each node, by default σ (Definition 5.6). No source computes an adequate value of either (Remarks 3.18 and 7.8).

(3) The stalled-block predicate is-stalled-block (Definition 4.1); the book proposes coinbase-only blocks (Remark 4.25).

(4) The encoding of the cross-link fields and their placement in the block header or in the block (Definition 4.2, Remark 4.8).

(5) The mapping from stake to voting units and the selection of the roster (Definitions 10.4, 10.6 and 10.7).

(6) The staking calendar (Definition 10.16, Remark 10.17).

(7) The format of bond transactions, the range of quantised bond amounts and a minimum stake (Definition 10.11, Remarks 10.13 and 10.5).

(8) The reward rule (Remark 10.22), and the division of the commission among finalisers and its denominator (Definition 10.23).

(9) The fees of staking transactions and the number of staking actions that one transaction may carry (Remark 10.13).

(10) The rotation of finaliser keys and the initial roster (Remark 10.30, Definition 10.8).

(11) Activation on an existing chain (Remark 13.2).

No formal model checks the safety and liveness statements of the construction; the existing models check fragments only (Remark 12.6).

Status: open problem.

Remark 13.2 (Activation). No design activates Crosslink 2 on the existing Zcash chain with a safety argument that rests on consensus rules alone. The book models both chains from genesis: every x-chain starts at $\mathcal{O}_x$ (Definition 3.4), and the Genesis bc-block rule sets $\mathcal{O}_{\mathrm{bc}}.\mathsf{context_bft} = \mathcal{O}_{\mathrm{bft}}$ (Definition 4.17). Shielded Labs' design starts the BFT protocol from best-chain state by a deterministic bootstrap (Definition 10.8), whose roster height h_1 and start height h_2 are unsettled, the two current texts of the overview differing. The overview's safety argument (Remark 10.10) rests on rollback limits, which are node policy: consensus imposes no maximum reorganisation depth (*Consensus Guide*, §"Rollback limits, coinbase maturity, and finality"), and a node-local limit does not give its conclusion. Proposition 10.9 proves the conclusion under Prefix Agreement of the best chain at depth $h_2 - h_1$ instead, a prefix property that is an assumption for Zcash (Remark 3.18). The safety of the bootstrap therefore rests on a parameter that consensus does not fix or on that assumption.

Status: open problem.

13.2 Status by goal and by rule

Table 3 gives, for each goal of the construction, the result that discharges it, the status of that result, and the open problems on which its standing for Zcash depends. Table 4 lists every validity rule and honest-behaviour rule of the two sub-protocols and the update rule of the node state, with the party that checks or follows it and the results whose statement or proof uses it; it extends Remark 6.9 from the best-chain rules to the whole mechanism. A rule that no proved result uses is marked so.

Table 2. Open problems of the volume, in the order of their statements, each with the environment or subsection that states it and the source in which it arises. The book, the paper and the overview are the sources so named in "Sources and conventions" (§1.3); the stating environment cites the chapter or section. "This volume" marks a problem that no source states.

Open problem	Stated in	Source
The open question of the paper	Remark 2.16, §2.5	paper
Status of the prefix properties for Zcash	Remark 3.18, §3.3	*FlyClient Guide*; *Consensus Guide*; ZIP 218
Honestly submitted blocks	Remark 3.28, §3.4	book
Parameters	§4.1	book
Encoding	Remark 4.8, §4.2	overview
Proposed Zcash policy	Remark 4.25, §4.7	book
Candidate or final snapshot	Remark 5.5, §5.1	book
Syncing and checkpoints	§5.3	book
Necessity of the other rules	Remark 6.11, §6.4	book
Missing results for ba	Remark 6.16, §6.5	book
Transfer from Π_{origbc}	Remark 7.4, §7.1	book
Advance of finalisation	Remark 7.5, §7.2	book
Choice of L	Remark 7.8, §7.2	book
Necessity of $\sigma \leq L$ for Zcash	§8.2	this volume
Finalised chain off the best chain	Remark 8.6, §8.2	book
Open parts of the override design	Remark 8.9, §8.3	book
Deferred validity	Remark 9.4, §9.3	overview
Status of sticky fork choice	Remark 9.9, §9.4	overview
Final Agreement of Tenderlink	Remark 9.15, §9.7	this volume

Table 2. Open problems, continued.

Open problem	Stated in	Source
Finaliser consent	Remark 10.5, §10.2	this volume
Active roster	Definition 10.6, §10.2	overview
Bootstrap	Definition 10.8, §10.2	overview
Transaction contents	Remark 10.13, §10.3	*Rewards Distribution*
Open bond questions	Remark 10.14, §10.3	this volume; overview
Staking window	Definition 10.16, §10.4	overview
Calendar divergence	Remark 10.17, §10.4	Shielded Labs texts
Privacy design	Remark 10.18, §10.4	this volume
Open privacy and expiry questions	Remark 10.19, §10.4	this volume; *Wallet Guide*
Unsettled payment rule	Remark 10.22, §10.5	book; overview; *Rewards Distribution*
Commission and bond earnings	Definition 10.23, §10.5	overview; *Rewards Distribution*
Conditional rewards	Remark 10.24, §10.5	overview; *Rewards Distribution*
Withheld rewards and the NSM	Remark 10.25, §10.5	this volume
Slash fork	Definition 10.27, §10.6	overview
Open slashing details	Remark 10.28, §10.6	overview
Keys and signatures	Remark 10.29, §10.7	overview
Key rotation	Remark 10.30, §10.7	book
Behaviour during a stall	Remark 11.12, §11.3	this volume
Confirmation policy under finality	Remark 11.14, §11.4	*Wallet Guide*; ZIP 315
Anchor policy relative to fin	Remark 11.16, §11.4	*Wallet Guide*; *Ironwood Guide*
Expiry under finality	Remark 11.17, §11.4	this volume
Downstream services	Remark 11.19, §11.4	this volume
Verification of finality by light clients	Remark 11.22, §11.5	overview; *FlyClient Guide*
Components left unfixed	Remark 13.1, §13.1	this volume
Activation	Remark 13.2, §13.1	book; overview

Table 3. Status by goal: the result that discharges each goal, its status in the words of Definition 1.1, and the open problems on which its standing for Zcash depends. The liveness results are prose arguments of the book.

Goal	Result	Status	Depends on
Assured Finality	Corollary 6.8, Assured Finality under either premise; Final Agreement of Π_{bft} from that of $\Pi_{origbft}$ rests on Proposition 7.3, Transfer by erasure, argued (TFL book)	proved (this volume), of a designed-but-unspecified construction	Remarks 3.18 and 7.4 (Prefix Agreement of Π_{bc} for Zcash); Remark 9.15 (a concrete BFT protocol)
Safety under subversion of the BFT protocol	Corollary 6.4, Safety under subversion of the BFT protocol	proved (this volume), of a designed-but-unspecified construction	Remarks 3.18 and 7.4
Ledger prefix property	Proposition 5.8, Ledger prefix property	proved (TFL book), of a designed-but-unspecified construction	none
Bounded availability	Propositions 8.4, Bounded non-stalled run; 8.2, Bounded hazard-freeness	proved (this volume), of a designed-but-unspecified construction	Remarks 8.6 and 7.8
Liveness	Propositions 7.1, Liveness of Π_{bc}; 7.2, Liveness of Π_{bft}; 7.6, Stalled Mode only on failure of the liveness assumptions	argued (TFL book)	Remarks 7.5, 7.4 and 7.8
Agreement on ba	Corollary 6.15, Agreement on ba from Prefix Agreement	proved (this volume), of a designed-but-unspecified construction	Remarks 3.18 and 7.4 (Prefix Agreement of Π_{bc} at μ for Zcash); Remark 6.16
Client contract	Propositions 11.3, Guarantee of finality; 11.4, Finality status; 11.5, Validity of the finalised chain	proved (this volume), of a designed-but-unspecified construction	Remarks 11.22 and 11.14; for the second clause of Proposition 11.3, those of Assured Finality

Table 4. Rules: every validity rule and honest-behaviour rule of Π_{bft} and Π_{bc} and the update rule of the node state, with its defining environment, the party that checks or follows it, and the environments whose statement or proof uses it.

Rule	Defined in	Checked or followed by
Genesis bft-block rule	Definition 4.9	every node
Used by: Definition 4.3, Snapshot (genesis case); Proposition 4.21, Existence of a valid context; Proposition 4.30, Forced entry into Stalled Mode; Proposition 6.6, Necessity of the Linearity rule; Proposition 7.7, Stall when $L \le \sigma$; Proposition 8.5, Necessity of the constraint $\sigma \le L$; Proposition 9.12, Final Agreement route under the Shielded Labs rules; Proposition 11.2, Path to finality.		
Inherited origbft rules, with the Parent rule	Definition 4.9	every node
Used by: Lemma 3.27, Per-BFT-epoch uniqueness; Proposition 4.21, Existence of a valid context; Lemma 4.28, Growth of finality depth; Proposition 4.29, Branch dependence of exit; Proposition 4.30, Forced entry into Stalled Mode; Theorem 6.5, Final Agreement implies Assured Finality; Proposition 8.4, Bounded non-stalled run; Proposition 10.3, Power of a finaliser coalition; Proposition 11.2, Path to finality.		
Linearity rule	Definition 4.9	every node
Used by: Proposition 4.21, Existence of a valid context; Lemma 4.28, Growth of finality depth; Proposition 4.29, Branch dependence of exit; Theorem 6.5, Final Agreement implies Assured Finality; Proposition 6.6, Necessity of the Linearity rule; Proposition 7.2, Liveness of Π_{bft}; Proposition 8.4, Bounded non-stalled run; Proposition 8.5, Necessity of the constraint $\sigma \le L$; Proposition 9.12, Final Agreement route under the Shielded Labs rules; Proposition 11.2, Path to finality.		
Tail Confirmation rule	Definition 4.9	every node; a validator also downloads and checks the blocks before voting
Used by: Remark 4.7, Purpose of the additions, states its purpose; Proposition 4.21, Existence of a valid context; Proposition 5.14, Finalisation lag; Proposition 6.6, Necessity of the Linearity rule; Proposition 7.2, Liveness of Π_{bft}; Proposition 7.7, Stall when $L \le \sigma$; Proposition 8.5, Necessity of the constraint $\sigma \le L$.		
Honest proposal	Construction 4.13	proposer
Used by: Proposition 7.2, Liveness of Π_{bft}; Proposition 5.14, Finalisation lag (attainment of the bound).		
Honest voting: Valid proposal criterion, Confirmed best-chain criterion	Construction 4.14	validator
Used by: Definition 3.22, Non-honest casting; Proposition 6.6, Necessity of the Linearity rule; Proposition 7.7, Stall when $L \le \sigma$; Remark 4.15, Refusal to vote; Remark 8.7, Override argument.		

Table 4. Rules, continued.

Rule	Defined in	Checked or followed by
Genesis bc-block rule	Definition 4.17	every node
Used by: Definition 4.4, Last final context ($\mathsf{LF}(\mathcal{O}_{\mathrm{bc}}) = \mathcal{O}_{\mathrm{bft}}$); Proposition 4.21, Existence of a valid context; Theorem 6.5, Final Agreement implies Assured Finality; Proposition 7.1, Liveness of Π_{bc}; Proposition 8.2, Bounded hazard-freeness; Proposition 8.4, Bounded non-stalled run.		
Inherited origbc rules	Definition 4.17	every node
Used by: Lemma 4.19, Prefix Lemma; Proposition 4.21, Existence of a valid context; Proposition 9.14, Unbounded run under the Shielded Labs rules; Proposition 11.5, Validity of the finalised chain.		
Valid context rule	Definition 4.17	every node
Used by: Proposition 4.21, Existence of a valid context; Lemma 4.28, Growth of finality depth; Proposition 4.29, Branch dependence of exit; Lemma 5.3, Local fin-depth; Proposition 5.14, Finalisation lag; Theorem 6.3, Prefix Agreement implies Assured Finality; Corollary 6.4, Safety under subversion of the BFT protocol; Theorem 6.5, Final Agreement implies Assured Finality; Proposition 6.6, Necessity of the Linearity rule; Proposition 7.1, Liveness of Π_{bc}; Proposition 7.7, Stall when $L \le \sigma$; Proposition 8.5, Necessity of the constraint $\sigma \le L$; Proposition 9.12, Final Agreement route under the Shielded Labs rules; Proposition 9.14, Unbounded run under the Shielded Labs rules; Proposition 10.3, Power of a finaliser coalition; Proposition 11.2, Path to finality.		
Extension rule	Definition 4.17	every node
Used by: Lemma 4.19, Prefix Lemma; Proposition 4.21, Existence of a valid context; Lemma 4.28, Growth of finality depth; Proposition 6.6, Necessity of the Linearity rule; Proposition 7.1, Liveness of Π_{bc}; Proposition 8.4, Bounded non-stalled run; Proposition 8.5, Necessity of the constraint $\sigma \le L$; Proposition 9.14, Unbounded run under the Shielded Labs rules.		
Last Final Snapshot rule	Definition 4.17	every node
Used by: Lemma 4.18, Candidate under the Last Final Snapshot rule; Proposition 4.21, Existence of a valid context; Proposition 4.29, Branch dependence of exit; Proposition 4.30, Forced entry into Stalled Mode; Proposition 6.6, Necessity of the Linearity rule; Proposition 7.1, Liveness of Π_{bc}; Proposition 8.2, Bounded hazard-freeness; Proposition 8.4, Bounded non-stalled run; Proposition 8.5, Necessity of the constraint $\sigma \le L$; Proposition 9.14, Unbounded run under the Shielded Labs rules.		
Finality depth rule	Definition 4.17	every node
Used by: Proposition 4.21, Existence of a valid context; Proposition 4.29, Branch dependence of exit; Proposition 4.30, Forced entry into Stalled Mode; Proposition 6.6, Necessity of the Linearity rule; Proposition 7.1, Liveness of Π_{bc}; Proposition 8.2, Bounded hazard-freeness; Proposition 8.4, Bounded non-stalled run; Proposition 8.5, Necessity of the constraint $\sigma \le L$; Proposition 9.14, Unbounded run under the Shielded Labs rules (by its absence).		

Table 4. Rules, continued.

Rule	Defined in	Checked or followed by
Honest block production (choice of context)	Construction 4.20	block producer
Used by: Proposition 4.21, Existence of a valid context; Proposition 7.1, Liveness of Π_{bc}.		
Unchanged fork choice	Definition 4.22	every node
Used by: Proposition 7.1, Liveness of Π_{bc}; no proved result.		
Update rule update-fin	Definition 5.1	every node, as local state
Used by: Lemma 5.2, Local finalization linearity; Lemma 5.3, Local fin-depth; Lemma 5.9, Local ba-depth; Proposition 5.14, Finalisation lag; Proposition 6.2, The single-node case; Theorem 6.3, Prefix Agreement implies Assured Finality; Corollary 6.4, Safety under subversion of the BFT protocol; Theorem 6.5, Final Agreement implies Assured Finality; Proposition 6.6, Necessity of the Linearity rule; Theorem 6.12, Prefix Consistency implies Prefix Consistency of ba; Proposition 6.13, Retention of the finalised prefix in ba; Corollary 6.15, Agreement on ba from Prefix Agreement; Proposition 8.4, Bounded non-stalled run; Proposition 8.5, Necessity of the constraint $\sigma \leq L$; Proposition 9.7, Absence of finalisation hazards under sticky fork choice; Proposition 9.12, Final Agreement route under the Shielded Labs rules; Proposition 9.13, Prefix Agreement route under the Shielded Labs rules; Proposition 9.14, Unbounded run under the Shielded Labs rules; Proposition 10.3, Power of a finaliser coalition; Proposition 11.2, Path to finality; Proposition 11.3, Guarantee of finality; Proposition 11.4, Finality status; Proposition 11.5, Validity of the finalised chain; Corollary 11.8, Confirmations of a newly final block; Lemma 11.18, Expired unmined below fin.		
Syncing gate, a recommendation	Remark 5.12	every node
Used by: Definition 11.10, Application views; no proved result.		